Degree Project in the Field of Technology Fluid Mechanics and the Main Field of Study Engineering Mechanics

Second cycle, 30 credits

Inverse Design of an Intermediate Compressor Duct

SOHAN ROY, CONSTANTIN SEEMANN

Degree Program in Engineering Mechanics

Date: 2025.07.05

Superivsors: Jens H.M. Fransson, Jonathan Bergh, Blanka Tonsic

Examiner: Jens H. M Fransson, School of Engineering Sciences

Host company: GKN Aerospace

Key Words

Optimization, compressor duct, boundary layers, inverse design, metaheuristic algorithms, MGM, Differential Evolution

Abstract

The report at hand investigates the concept of a partially physics driven inverse design method. Starting point is a literature review as an introduction into the core concepts in fluid mechanics and optimization theory used to develop the tools used in the inverse design algorithm. The algorithm itself consists of two main parts: A metaheuristic algorithm which is aimed to generate a target pressure distribution (TPD), as well as a Modified Garabedian-McFadden (MGM) algorithm to create the respective geometry, that resembles the target pressure distribution. While this two–step process should be applicable to most internal flow cases, this study aims to establish and validate it in the context of aggressive inter compressor ducts. Furthermore, the influence of turbulence models in the generation of the wall pressure distributions is investigated. In particular, the Standard k-ω and Standard k-ϵ model's behavior is being compared, when facing different levels of aggressiveness. To further develop the established concept, a more advanced process in the TPD generation is examined: Including an analysis of the impact of the viscous forces and the nature of the model governing equation, qualitatively commenting on its scope. With the current implementation, the in-house inverse solver requires approximately 100 iterations of a 3D k-ε solver with standard wall functions—each taking 25 to 30 seconds—to reach a pressure residual level under or around 1%. With minor modifications to the model's governing equations the current 2D inverse design algorithm (IDA) framework shows potential for extension to three-dimensional problems.

Contents

Inverse Design of an Intermediate Compressor Duct 1
1 Introduction 10
1.1 Research question 10
1.2 Context of inter-compressor duct design 10
2 Technical Background 11
2.1 Fundamentals of gas turbines 11
2.1.1 Inter-compressor duct 11
2.2 Basics of Fluid Mechanics 12
2.2.1 Governing equations 12
2.2.2 Boundary layers 14
2.3 Laws of thermodynamics and entropy 22
2.3.1 Zeroth law of thermodynamics 22
2.3.2 First law of thermodynamics 22
2.3.3 Second law of thermodynamics 22
2.4 Ideal gas law and isentropic relation 23
2.4.1 Boyle's law 23
2.4.2 Charles's law 23
2.4.3 Avogadro's Law 23
2.4.4 Ideal gas law 23
2.4.5 Specific heat capacities and their ratio 24
2.4.6 Isentropic relation between gas states 25
2.5 Compressible flow 26
2.6 Isentropic flow relations 27
2.7 Computational Fluid Dynamics 29
2.7.1 Classification of governing equations 29
2.7.2 Classification of a 2D subsonic boundary layer equation 31
2.7.3 Discretization techniques 31
2.7.4 Turbulence models 34
2.8 Optimization Code 37
2.8.1 Relevance and challenges of optimization in engineering 37

2.8.2 Classification of Optimization Problems 38
2.8.3 Categorization of Optimization Methods 41
2.8.4 Deterministic Optimization Methods 42
2.8.5 Metaheuristic Methods 43
2.9 Purpose of an Inverse Design Process 44
3 Methodology 46
3.1 Differential Algorithm 46
3.1.1 Choice of algorithm 46
3.1.2 Algorithm Design 49
3.1.3 Constraints of wall pressure distribution 50
3.2 Wilcox EddyBl program 52
3.2.1 Boundary layer code design 52
3.2.2 Verification Wilcox EddyBl boundary layer code 53
3.2.3 Comparison / Choice of turbulence models 59
3.2.4 Example EddyBl program 60
3.3 Inverse design Framework 62
3.3.1 Mathematical formulation 63
3.3.2 Inlet slope correction 67
3.3.3 CFD solver 67
4 Results 69
4.1 Grid independence test and validation of *trollsol_0.5.1* 69
4.2 2D inverse design application for internal flows 70
4.2.1 Additional algorithm validation cases 75
4.2.2 Geometry generation issues detected and potential solutions 78
4.3 Geometries generated for different levels of aggressiveness 83
4.3.1 Low Aggressive duct design 83
4.3.2 Medium aggressive duct design 85
4.3.3 Highly aggressive Duct Design 87
4.3.4 Profile generation with axial outflow 90
4.3.5 Results discussion 93
4.4 Nature and scope of the model governing equation 94
4.4.1 Effect of including viscous effects 95
5 Conclusions and future work 100
5.1 Turbulence model investigation 100

5.2 Potential further improvements to TPD generation 100
5.3 Expanding the scope of the inverse design solver 101
6 Bibliography 102

List of Figures

Figure 1: Semi cross-sectional view of an aircraft engine in the axial-radial plane with the ICD (marked with a red circle) (Tonsic, Bala, Lejon, Deshpande, & Bergh, 2024) 12
Figure 2 : Schematic diagram depicting a laminar boundary layer profile development 15
Figure 3 : Schematic depiction of flow over a semi-infinite flat plate. Here, increasing x is synonymous with increasing Reynolds number 15
Figure 4: Streamlines and velocity profiles near a separation point 16
Figure 5 Displacement thickness 19
Figure 6 Transformation method (Bergh, 2018) 40
Figure 7 Differential Evolution algorithm (Bergh, 2018) 48
Figure 8 Setup optimization problem 49
Figure 9 Area equality constraint.....51
Figure 10 Comparison Falkner Skan and EddyBl numerical solutions for (a) m=0, (b) m>0 and (c) m<0.....55
Figure 11 Turbulence flow validation 58
Figure 12: Turbulence model comparison 59
Figure 13 Example EddyBl program with neutral pressure gradient.....61
Figure 14 Example EddyBl program with adverse pressure gradient.....61
Figure 15: Flowchart representing the inverse-design framework 62
Figure 16: Comparison between the pressure profiles solved by the in-house solver (Trollsol) and Ansys FLUENT.....70
Figure 17: (a) Pressure profile and (b) generated geometry using IDA without any outlet area constraint 71
Figure 18: (a) Absolute static pressure comparison between Target and Computed geometries (b) Geometry comparison for the same case 72
Figure 19 Axial velocity contour for the IDA generated geometries (a) without and (b) with the outlet area constraint 73
Figure 20: (a) Absolute static pressure comparison between Target and Computed geometries (b) Pressure residual in case 1 74
Figure 21: TPD and CPD comparison for TPD optimized using (a) k-ε (b) k-ω as the turbulence model in the boundary layer code.....75
Figure 22: Geometry corresponding to the optimized TPDs using k-ε and k-ω 76
Figure 23: Pressure contour of the geometry corresponding to (a) Figure 21(a) and (b) Figure 21(b).....77
Figure 24: Generated (a) TPD, including CPD and (b) Geometry using k-omega in the BL code.....79
Figure 25: Generated (a) TPD, including CPD and (b) Geometry using k-epsilon in the BL code.....80
Figure 26: (a) TPD achieved using the k-omega curve by manually added pressure points (b) Generated duct geometry from the mentioned TPD 81
Figure 27: Standard k-ω model, low aggressiveness.....83
Figure 28: Standard k-ε model, low aggressiveness 84

Figure 29: Standard k-ω model, medium aggressiveness 85
Figure 30: Standard k-ε model, medium aggressiveness 86
Figure 31: Standard k-ω model, high aggressiveness 87
Figure 32: Axial-velocity contour of the geometry achieved by using a k-ω model with high aggressiveness 88
Figure 33: Standard k-ε model, high aggressiveness.....89
Figure 34 (a) TPD vs CPD and (b) Final geometry achieved with a TPD using a constrained trapped area of 340 and 460 N/m 91
Figure 35 (a) Residual plot and (b) Axial-velocity contour plot achieved with a TPD using a constrained trapped area of 340 and 460 N/m 92
Figure 36: Axial velocity profile at the outlet of the generated duct in Figure 34 93
Figure 37: Free body diagram of a differential fluid element, curving under the effect of shear stress 95
Figure 38: Pressure residual, depicting the inviscid nature of the IDA governing equation.....97
Figure 39: : Pressure residual with a mass flow rate of (a) 24.22 kg/s (b) 19.38 kg/s98
Figure 40: Pressure residual with a mass flow rate (a) 16.35 kg/s and (b) 15 kg/s....99

Abbreviations and nomenclature

TPD	Target pressure distribution
CPD	Computed pressure distribution
RPM	Revolutions per minute
ICD	Inter-compressor duct
IDA	Inverse design algorithm
MGM	Modified Garabedian-McFadden
CFD	Computational fluid dynamics
DE	Differential evolution
SFC	Specific fuel consumption

1 Introduction

1.1 Research question

The aerodynamic design of S-shaped ducts is a challenging task as it involves curved geometries and, as a result, complex flow fields, including separation, secondary flows, or non-uniform outlet flows. Conventional "direct design" approaches rely on iterative CFD evaluations to find the optimal solution. These are not only time consuming and computationally expensive but can also yield suboptimal results – especially when considering multi-constraint scenarios.

Inverse design methodologies offer an attractive alternative by firstly specifying the desired performance, in the form of a pressure distribution for example, as an input, and then determining the duct geometry based on that. Among these, residual correction-based inverse methods, such as the modified Garabedian-McFadden (MGM) (Malone, Vadyak, & Sarkar, 1985) framework, have shown great potential in producing accurate geometries at relatively low computational costs. (Gubanova & Gubanova, 2014)

The thesis proposes an integrated inverse design framework combining a modified MGM algorithm with a metaheuristic optimization method to generate the target pressure distribution. Specifically, a differential evolution algorithm, subject to different constraints such as flow separation, is applied to optimize the wall pressure distribution. (Nili-Ahmadabadi, Aghabozorgi, Cho, & Kim, 2021) These optimized pressure profiles serve as an input to the MGM-based routine to create the corresponding geometries. This two-step process serves as a modular and robust architecture to design aerodynamic components. In particular, this study aims to further modify and apply the developed algorithms in the context of an inter-compressor duct (ICD).

1.2 Context of inter-compressor duct design

The inverse design approach is applicable for most non-separating internal flows. The current study only focuses on its application in designing the inter compressor S-shaped duct. As the name suggests, this component in the aero-engine intakes air from the low-pressure compressor to the high-pressure compressor, while ensuring good quality outflow and maintaining as high-pressure recovery as possible. Another optimization objective is increasing the aspect ratio of the duct as much as possible, ensuring reduction in length, weight and hence reducing the fuel consumption. Further details on this component is covered in Section 2.1.1.

2 Technical Background

2.1 Fundamentals of gas turbines

The fundamentals of an aero gas turbine revolves around the key concepts of fluid mechanics and thermodynamics. There are few important components in a gas turbine which are relevant to our study according to (Dan, Dixon, & Hall, 2025):

1. A rotating fan: It intakes as much air as possible, adding kinetic energy to the flow. Most of the air is bypassed as this is more efficient in generating thrust. Still, there is a need to use fuel to run the fan in the first place. Therefore, some of the airflow needs to be directed to the engine core. All the further downstream components are only applicable to the airflow in the engine core.
2. Compressor: There are two of these components, high- and low-pressure compressor (HPC and LPC respectively). The upstream component is the LPC, and it is bigger in diameter, hence it has a lower RPM to maintain its structural integrity. This is followed by the ICD, which directs the flow downstream, to the HPC. HPC is a lot smaller in diameter and has a much higher RPM doing most of the compression.
3. Combustion chamber: In the combustion chamber, the high-pressure air from the compressor is mixed with fuel and ignited. This combustion rapidly increases the temperature and energy of the air, creating high-velocity exhaust gases driving the turbine in the next stage.
4. Turbine: Similar to compressors, there are two turbines, high- and low-pressure turbines (HPT and LPT respectively), in order. Unlike the compressor, which adds energy to the flow by increasing the static pressure and temperature, the turbine is rotated by the flow and thus extracts energy from it. The turbine, connected by the shaft, rotates the compressors and the fan.

2.1.1 Inter-compressor duct

2.1.1.1 *Function*

Usually, an ICD has struts in it, which are similar to airfoils in shape. Struts serve the purpose of providing structural support, housing systems like lubrication tubes and ensuring minimal flow disruptions between the LPC and HPC. Struts were not considered in the scope of this problem, to make the flow axisymmetric, allowing the use for a 2D boundary layer code (refer to section 3.2).

As the main purpose of the compressors are to compress the air, i.e., increase the static pressure, it automatically becomes one of the important objectives of an ICD. Another important criterion is the quality of the outflow. The outflow should be as close to the ideal case as

possible, that means a flow, parallel to the axis. Most importantly, there should not be any separation bubble (refer to section 2.2.2 and Figure 4), to maintain an ideal velocity profile shape. Separation also means that loss in total pressure, usually as a reduction in static pressure. All these criteria are important to ensure the intended performance quality of all the downstream components, especially the HPC.

2.1.1.2 *What can be optimized?*

In addition to the constraints, the following ICD features can be optimized:

1. Increasing the pressure recovery or compressing as much as possible.
2. Reducing the length of the duct in the axial direction, keeping the inlet and outlet radial position, i.e., making the duct more aggressive.

Increasing the aggressiveness, with the same change in radial location from inlet to outlet, the duct length would reduce, translating to a reduction in mass and the overall length of the engine. This would decrease the specific fuel consumption (SFC) of the engine, which is defined by the following formula:

$$\mathrm{SFC} = \frac{Fuel\ flow\ rate\ (in \frac{kg}{s})}{Thrust\ (in\ Newton)} \left[\frac{s}{m}\right]$$

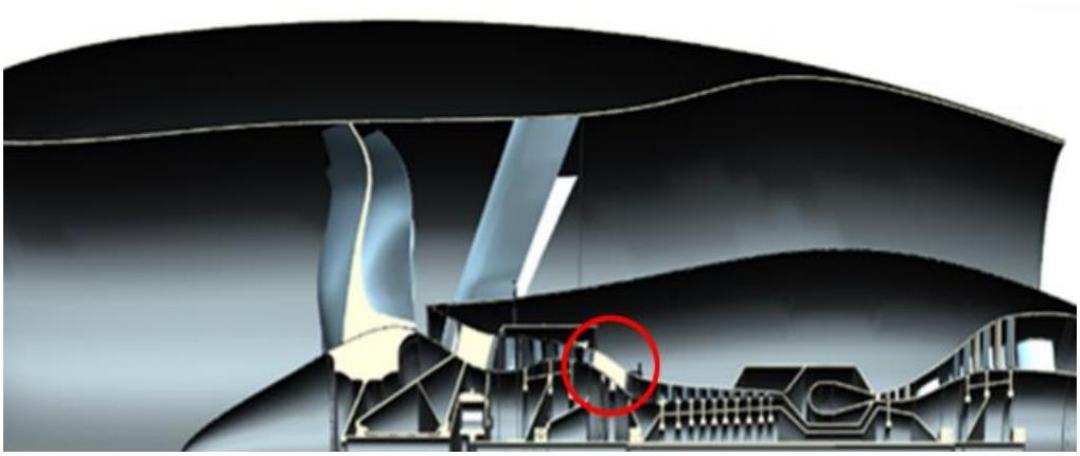


Figure 1: Semi cross-sectional view of an aircraft engine in the axial-radial plane with the ICD (marked with a red circle) (Tonsic, Bala, Lejon, Deshpande, & Bergh, 2024)

2.2 Basics of Fluid Mechanics

To assess the performance of an aero engine, the need to understand why fluid flows behave in a certain way is crucial. Therefore, it is important to comprehend the factors affecting the flow, covered in this sub-chapter.

2.2.1 Governing equations

The three governing equations are a representation of the law of conservation of mass, momentum and energy as a set of partial differential equations. For a detailed derivation, one can refer to (Kundu, Cohen, & Dowling, 2016)

Mass conservation

$$\frac{\partial \rho}{\partial t} + \frac{\partial \rho U_j}{\partial x_j} = 0$$

Equation 1 : Mass conservation equation

Also known as the continuity equation (Kundu, Cohen, & Dowling, 2016), where,

ρ = density of fluid

t = time

U_j = Velocity component in x_j direction

x_j = Three spatial cartesian coordinates (where j = 1, 2 or 3),

which can be interpreted in this way:

1st term: Imagine a flow particle (consisting of molecules in the order of 10^3) as a flowing control volume, with one unit volume. The temporal gradient term quantifies how the mass of the control volume changes with respect to time.

2nd term: This divergence term takes into account the rate at which mass is entering/leaving the control volume (flux), through its surfaces.

Hence, if there is more fluid entering than leaving (2nd term is negative), the mass of the control volume will increase (1st term is positive).

Momentum conservation

$$\rho \frac{DU_j}{Dt} = \rho g_j + \frac{\partial T_{ij}}{\partial x_j}$$

Equation 2 : Momentum conservation equations

In Equation 2 (Kundu, Cohen, & Dowling, 2016), g_j is gravity, including any other body force (force applicable on the entire fluid particle and not only control surfaces). T_{ij} is the total stress tensor, whose formulation is shown in Equation 3 (Kundu, Cohen, & Dowling, 2016). Note that the constitutive equation assumes the fluid to be Newtonian, isotropic and imposes Stokes assumption as well as the stress symmetry restriction, i.e., $T_{ij} = T_{ji}$.

$$T_{ij} = -P\delta_{ij} + 2\mu S_{ij} + \lambda S_{mm}\delta_{ij}$$

Equation 3 : Constitutive equation

Navier-Stokes for compressible flow

Replacing the stress tensor in Equation 2 with its formulation in Equation 3, yields in (Kundu, Cohen, & Dowling, 2016, s. 130)

$$\rho\frac{DU_j}{Dt} = -\frac{\partial P}{\partial x_j} + \rho g_j + \frac{\partial}{\partial x_i}\left[\mu\left(\frac{\partial U_j}{\partial x_i} + \frac{\partial U_i}{\partial x_j}\right) + \left(\mu_v - \frac{2}{3}\mu\right)\frac{\partial U_m}{\partial x_m}\delta_{ij}\right]$$

Equation 4 : Navier-Stokes equations for compressible flow

Equation 4 is an elaborate way of writing Newton's 2nd Law of Motion, i.e.,

Force = mass × acceleration. In Equation 4, the left-hand side equals the density multiplied with acceleration. The right-hand side represents the force per unit volume due to pressure gradient, body forces and viscosity respectively.

There is a simpler version of the Navier-Stokes equations for incompressible flows, but it is not relevant in our scope of work.

2.2.2 Boundary layers

Viscosity, also referred to as "Lack of slipperiness" by Sir Isaac Newton, was not properly introduced until 1929 by Jean Léonard Marie Poiseuille, i.e., after humans already learnt how to fly. Prior to that, the main way of studying fluid was potential flow. Fluid molecules keep on jumping around and colliding with other particles, hence exchanging momentum. This process of momentum exchange or diffusion is quantified using the material property of viscosity.

Let's assume that there is a uniform (no spatial variation in velocity) and laminar (no small-scale random fluctuations in velocity, both spatial and temporal) free stream flow on a flat plate. As seen in Figure 2, the flow first enters very uniformly, then it encounters the no-slip wall. This means that there is an incredibly high momentum gradient near the wall. Viscosity allows momentum exchange between the flow layers, eventually smoothening the flow profile over the distance. Once the flow is profile does not change anymore with position, it is considered fully developed.

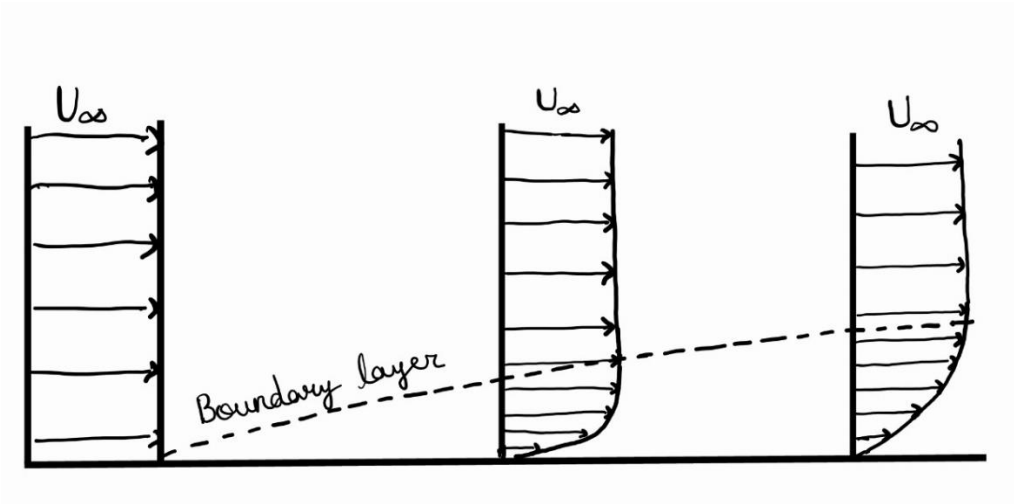


Figure 2 : Schematic diagram depicting a laminar boundary layer profile development

The above-mentioned profile is called a laminar boundary layer. Boundary layer (BL) denotes the region between the surface or the nearest point to the wall with positive velocity gradient and the point where the velocity almost reaches the free stream value. The thickness of this region is called the boundary layer thickness, δ. As one can see, δ increases with x. After a certain point, there is a transition from the laminar to the turbulent regime. This arises from instabilities created due adverse pressure gradient, surface roughness, blowing at surface and free-stream turbulence. These instabilities keep growing and after a certain point, the flow is considered fully turbulent as shown in Figure 3.

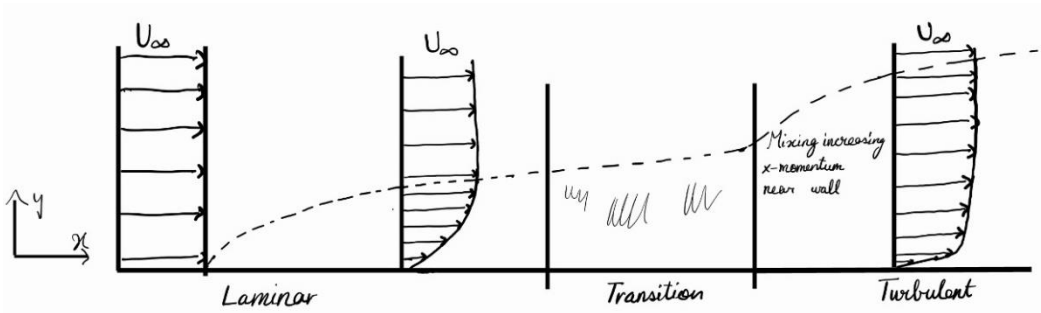


Figure 3 : Schematic depiction of flow over a semi-infinite flat plate. Here, increasing x is synonymous with increasing Reynolds number

A flow with a chaotic/random spatial and temporal variation in the flow characteristics is termed turbulent. In turbulent flows, increased mixing enhances the rate of momentum transfer. This in return can be represented as an additional contribution to the fluid's viscosity known as turbulent viscosity. Using Reynolds decomposition, modifying the flow characteristic terms and calculating a mean of the whole equation, a new equation is derived, called Reynolds Averaged Navier-Stokes equation or RANS (Equation 5). The extra diffusion term also appears in RANS;

but as stress, called Reynolds stress (Pope, 2000). Details on how to relate or model the Reynolds stress with the rest of the flow properties, has been show in section 2.7.4.

$$\rho \frac{DU_j}{Dt} = -\frac{\partial P}{\partial x_j} + \rho g_j$$
$$+ \frac{\partial}{\partial x_i}\left[\mu\left(\frac{\partial U_j}{\partial x_i} + \frac{\partial U_i}{\partial x_j}\right) + \left(\mu_v - \frac{2}{3}\mu\right)\frac{\partial U_m}{\partial x_m}\delta_{ij} - < u_i u_j >\right]$$

Equation 5 : RANS equations for compressible flow

Why does a boundary layer separate from the surface?

Consider a boundary layer that keeps on growing and is attached to the flat plate, but after a point, the plate starts curving down (Figure 4). This curvature in the wall will lead to an adverse pressure gradient (positive pressure gradient trying to decelerate the flow). The adverse pressure reduces the velocity gradient, and it eventually becomes zero and negative. This indicates that the flow is separated, leading to reverse flow in downstream.

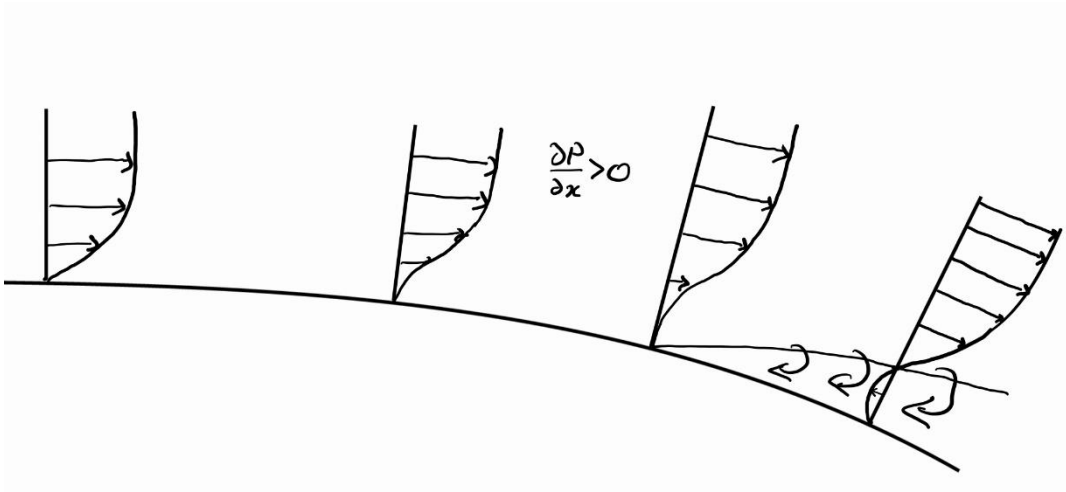


Figure 4: Streamlines and velocity profiles near a separation point

The relevance of separation has already been covered in section 2.1.

What is the Reynolds number?

The Reynolds number is a non-dimensional ratio of the inertial to the viscous forces and is formulated as:

$$Re = \frac{\rho U L}{\mu} = \frac{UL}{\nu}$$

Equation 6 : Reynolds number

where U is the velocity and L is the length scale of the flow respectively. Also, ρ is the density, μ is the dynamic viscosity and v is the kinematic viscosity of the fluid. A flow only remains laminar, if the viscous forces

dominate, keeping the streamlines in fixed layers. Hence, when Re increases, i.e., the inertial force gains dominance, the flow starts to go through transition and turns turbulent as shown in Figure 3. (Bertin, Cummings, & Reddy, 2014). Many transition CFD models use this concept; they only start using the turbulent models (sec. 2.7.4) after the flow has reached a certain Reynolds number.

2.2.2.1 *Laminar boundary layer equations*

The Navier-Stokes equations for a steady, 2-dimensional, incompressible and gravity-less flow are given by:

X:

$$\rho U \frac{\partial U}{\partial x} + \rho V \frac{\partial U}{\partial y} = -\frac{\partial p}{\partial x} + \mu \frac{\partial^2 U}{\partial x^2} + \mu \frac{\partial^2 U}{\partial y^2}$$

Y:

$$\rho U \frac{\partial V}{\partial x} + \rho V \frac{\partial V}{\partial y} = -\frac{\partial p}{\partial y} + \mu \frac{\partial^2 V}{\partial x^2} + \mu \frac{\partial^2 V}{\partial y^2}$$

Equation 7: Navier-Stokes equations for a steady, incompressible and body force-less flow

From Equation 1, it can be deduced that:

$$\frac{O(U)}{O(x)} = \frac{O(V)}{O(y)}$$

Equation 8 : Order of magnitude comparison from continuity equation

where O represents the order of magnitude and these values inside the BL are:

- O(U) = U_∞ (Free stream velocity)
- O(x) = L (Length of flat plate)
- O(y) = δ (BL thickness)

Hence,

$$O(V) = U_\infty \delta / \mathrm{L}$$

Equation 9 : Order of magnitude of the cross-stream velocity

The order of magnitude of the pressure depends on which forces dominate. Near the wall, as the velocities are low but their gradients are high, the viscous forces dominate. Then one can assume that the order of magnitude of the dominant viscous term and the pressure term in the momentum equation are the same.

This yields in,

$$\frac{O(P)}{L} = \mu \frac{U_\infty}{\delta^2}$$

$$=> \left(\frac{\delta}{\mathrm{L}} = \frac{1}{\sqrt{\mathrm{Re}}} \text{ from Equation 17 : Solution to the Blasius equation}\right)$$

$$=> O(P) = \frac{\mu U_\infty}{\mathrm{L}^2} \mathrm{Re}$$

Equation 10 : Order of magnitude of pressure

Each term in the two momentum equations can be scaled using its respective order of magnitude. Applying the definition of the Reynolds number (Equation 6) and using the result of Equation 17, the non-dimensionalized equations yield in:

X:

$$\rho U \frac{\partial U}{\partial x} + \rho V \frac{\partial U}{\partial y} = -\frac{\partial p}{\partial x} + \frac{1}{Re} \mu \frac{\partial^2 U}{\partial x^2} + \mu \frac{\partial^2 U}{\partial y^2}$$

Y:

$$\rho U \frac{\partial V}{\partial x} + \rho V \frac{\partial V}{\partial y} = -Re \frac{\partial p}{\partial y} + \frac{1}{Re} \mu \frac{\partial^2 V}{\partial x^2} + \mu \frac{\partial^2 V}{\partial y^2}$$

Equation 11 : Non-dimensionalized Navier Stokes equations for 2D incompressible and steady flow with Blasius flow profile assumption

As $Re \gg 1$, the equations approximate to:

X:

$$\rho U \frac{\partial U}{\partial x} + \rho V \frac{\partial U}{\partial y} = -\frac{\partial p}{\partial x} + \mu \frac{\partial^2 U}{\partial y^2}$$

Y:

$$\frac{\partial p}{\partial y} = 0$$

Equation 12 : Boundary layer equations for 2D incompressible and steady flow

2.2.2.2 *Quantified characteristics*

Displacement thickness:

As shown in Figure 5, the boundary layer causes a loss in the mass flow rate. This can be utilized to define a characteristic length scale. The so called displacement thickness (δ^*) represents the length by which the surface should be shifted in the cross-stream direction, to compensate for the mass deficit caused by the presence of a boundary layer (Figure 5). Hence,

$$\delta^* = \int_0^{y\to\infty} \left(1 - \frac{U}{U_\infty}\right) dy$$

Equation 13: Formula for displacement thickness (Equation 7.126, (Pope, 2000))

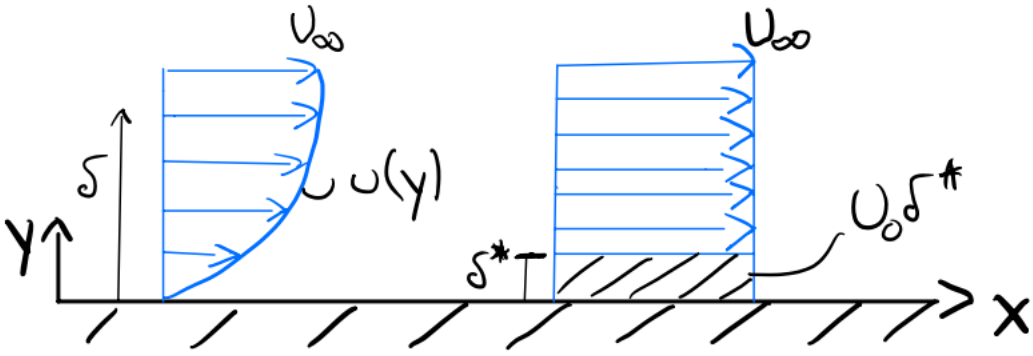


Figure 5 Displacement thickness

Momentum thickness

Similarly, momentum thickness (θ) is the momentum deficit in the streamwise direction caused by the presence of a boundary layer. As there is no cross-stream velocity outside the boundary layer, the momentum deficit in the streamwise direction is only due to the streamwise velocity. Hence,

$$\theta = \int_0^{y\to\infty} \frac{U}{U_\infty}\left(1 - \frac{U}{U_\infty}\right) dy$$

Equation 14: Formula for momentum thickness (Equation 7.127, (Pope, 2000))

For a flat plate or a flow with no pressure gradient, the drag can be directly calculated from the momentum thickness by multiplying it with ρU^2_∞.

Shape factor

Shape factor (H) is the ratio of the displacement thickness to the momentum thickness

$$H = \frac{\delta^*}{\theta}$$

Equation 15: Formula for shape-factor (Equation 7.139, (Pope, 2000))

This non-dimensionalized number is independent of the flow or body dimensions but only takes into account if the boundary layer is very thick, i.e. if there is a presence of adverse pressure gradient, risk of separation, or the boundary layer is fully developed. Thus, it is used as one of the measurements to detect, whether a flow is prone to separation or not.

These are some typical values for H**:**

Flow type	Shape factor (*H*)
Laminar boundary layer (Blasius)	$H \approx 2.59$
Turbulent boundary layer	$H \approx 1.3-1.4$
Near separation for laminar boundary layer	$H > 3.515$

The values for the laminar and the turbulent boundary layer (equation 7.176, (Pope, 2000)) can be derived from the analytical solutions in the following sections and the formula for H. The value for near separation was derived using a semi -analytical approach by assuming a quartic polynomial form for the velocity profile. (Pohlhausen, 1921)

2.2.2.3 *Analytical solutions to laminar boundary layer*

All well-known analytical solutions use the assumption of self-similarity. Taking another look at Figure 2, this phenomenon can be further explained: $\delta(x)$, the non-dimensionalized velocity, $f'(\eta)$, versus this new transformed coordinate, η, will not change with x.

Implementing this concept, two solutions can be derived for an incompressible laminar boundary layer. By assuming a self-similar velocity profile, Blasius reduced the boundary-layer equations to a third order nonlinear ODE.

$$f''' + \frac{1}{2} f f'' = 0$$

Equation 16 : Self-similar governing equation (Blasius, 1908) (Bertin & Cummings, Aerodynamics for Engineers, 2014)

Equation 16 can be solved numerically by imposing the following boundary conditions: The no-slip and impermeability condition at the wall, and the assumption that the velocity reaches its free stream value at the edge of the boundary layer. The solution provides information about the velocity profile and boundary layer thickness for a two-dimensional and steady flow over a flat plate, not being exposed to any pressure gradient. (Blasius, 1908)

$$\delta_{99}(x) \approx 5.0 \sqrt{\frac{\vartheta x}{U_e}}$$

Equation 17 : Solution to the Blasius equation (Equation 4.23 in (Bertin & Cummings, Aerodynamics for Engineers, 2014))

where $= \mu/\rho$, is known as the kinematic viscosity and the 99 in the subscript means that the cross-stream coordinate (y) at which the streamwise velocity (U) becomes 99% of the free stream velocity at the boundary layer edge (U_e).

Falkner-Skan *(Bertin & Cummings, Aerodynamics for Engineers, 2014)*

The Blasius equation considered an assumption of constant external streamwise velocity. Instead, if one generalizes it using an extra parameter, the pressure gradient can vary from zero yielding in a generalized form of Equation 17, the so-called Falkner-Skan solution:

$$\delta_{99}(x) \approx 5.0\sqrt{\frac{\vartheta x}{U_e}}, where\ U_e \propto (x - x_0)^n$$

Equation 18: Generalized Blasius solution for non-zero pressure gradient

In Equation 18,

- *n = 0 means no pressure gradient, hence constant free stream velocity*
- *n > 1* means negative or favourable pressure gradient and
- *n < 1* means positive or adverse pressure gradient
- x_0 *is the shifted origin*

This clearly shows that by applying a more adverse pressure gradient, the BL will grow, thus reducing the velocity gradient, $\partial U/\partial y$, at the wall. This will eventually lead to flow separation.

2.2.2.4 *Analytical solutions to turbulent boundary layers*

As the presence of Reynolds stresses acts as an extra unknown variable, the governing equations are not enough to solve them. One needs to use turbulence models as well as the self-similar approach to close the equations. (Turbulence models)

Prandtl's 1/7th Power Law (section 7.21, (Pope, 2000))

$$\frac{U}{U_\infty} = \left(\frac{y}{\delta}\right)^{1/7}$$

Equation 19: Prandtl's power law

Application: Mostly High Reynolds number

Spalding's Law (section 12.9, (Kundu, Cohen, & Dowling, 2016))

Spalding provided a continuous analytical expression for the velocity profile that covers both the log-law and viscous sublayer:

$$y^+ = u^+ + e^{-\kappa B}\left[e^{\kappa u^+} - 1 - \kappa u^+ - \frac{(\kappa u^+)^2}{2!} - \frac{(\kappa u^+)^3}{3!}\right]$$

Equation 20: Spalding's law

where:

- $y^+ = yu_\tau/v$ (non-dimensional wall distance)

- $u^+ = U/u_\tau$ (velocity in wall units)
- κ = von Kármán constant (~0.41)
- B = empirical constant (~5.0)

2.3 Laws of thermodynamics and entropy

2.3.1 Zeroth law of thermodynamics

The zeroth law of thermodynamics states that if two thermodynamic systems are both in thermal equilibrium with a third system, then the two systems are in thermal equilibrium with each other.

2.3.2 First law of thermodynamics

The first law of thermodynamics states that energy can neither be created nor destroyed, only altered in form. For any system, energy transfer is associated with mass crossing the control boundary, external work, or heat transfer across the boundary. These produce a change of stored energy within the control volume. Quantifying the statement into a mathematical relation:

$$dE = dQ - dW$$

Equation 21: 1st law of thermodynamics

where *E* is the internal energy stored in a thermodynamics system, *Q* is the heat added to the system and *W* is work done by the system.

2.3.3 Second law of thermodynamics

Entropy is a measure of randomness in a system, which can be only increased. For an isolated system to maintain constant entropy, two conditions must be satisfied: there must be no net heat transfer into or out of the system, and no internal irreversible processes (such as viscous dissipation) should occur.

$$dS = \frac{dQ_{rev}}{T}$$

Equation 22: Entropy formula for reversible process

This is also known as the 2nd law of thermodynamics stating that the state of entropy of the entire universe, as an **isolated system**, will always increase over time. Consequently, the changes in the entropy can never be negative.

A common example of this is the heat transfer between a hot and cold metal block only connected by a metal rod. Over time, heat flows from the hot block to the cold block until an equilibrium is reached, but it will never occur vice versa. The result arises from the hot block loosing heat at a higher temperature, while the cold block gains the same amount of heat. As the entropy change is defined as the ratio of heat transfer and temperature, the entropy decrease in the hot block is smaller in

magnitude than the entropy increase in the cold block. This creates a net positive change in the total entropy of the system:

$$dS = dS_{hot} + dS_{cold} = \frac{-|dQ|}{T_{hot}} + \frac{|dQ|}{T_{cold}} = |dQ|\left[-\frac{1}{T_{hot}} + \frac{1}{T_{cold}}\right] > 0, as\ T_{hot} > T_{cold}$$

Equation 23: Example showing entropy change can only be positive

Hence, it is the 2nd law that drives the heat from hot to cold systems.

2.4 Ideal gas law and isentropic relation

The ideal gas law is derived from three simpler gas laws:

2.4.1 Boyle's law

Boyle's law states describes the inverse proportionality relationship between pressure (*p*) and volume (*V*) of a fixed amount of gas at a constant temperature.

$$p \propto \frac{1}{V}$$

Equation 24: Boyle's law

2.4.2 Charles's law

Charles's Law describes the directly proportional relationship between the volume (*V*) and temperature (*T*, in Kelvin) of a fixed amount of gas at a constant pressure.

$$V \propto T$$

Equation 25: Charle's law

2.4.3 Avogadro's Law

Avogadro's law describes the direct proportionality between volume (*V*) and the amount of gas (n, in the number of molecules)

$$V \propto n$$

Equation 26: Avogadro's law

2.4.4 Ideal gas law

All the fore-mentioned three laws can be combined into one proportionality:

$$PV \propto nT$$

Equation 27: Ideal gas law proportionality

Replacing the proportionality sign yields in the ideal gas law

$$PV = n\mathcal{R}T$$

Equation 28: Ideal gas law with universal gas constant

The universal gas constant ($\mathcal{R}$) was introduced independently by August Friedrich Horstmann in 1873 and Dmitri Mendeleev in 1874. Mendeleev calculated its value with remarkable precision, within 0.3% of its modern value. The symbol ($\mathcal{R}$) itself was first used by Benoît Clapeyron in his work on the Carnot cycle in 1834.

The same value for $\mathcal{R}$, which is 8.314 J/mol.K, is valid for any gas, and this formula can be further converted into:

$$P = \rho\mathcal{R}T$$

Or

$$P\nu = RT,$$

Equation 29: Ideal gas law with specific gas constant

where *R* is the specific gas constant and ν is the specific volume or *1/ρ*.

The relation between *R* and $\mathcal{R}$ is:

$$R = \frac{\mathcal{R}}{M},$$

Equation 30: Relation between universal and specific gas constant

where *M* is the molar mass of the gas, which is described as the mass of 1 mole or $6.023*10^{23}$ molecules of a gas.

The only relevant *R* for our study is *R* of air, which is *287 J/KgK*

2.4.5 Specific heat capacities and their ratio

Heat added to the system can be represented as:

$$dq = cdT,$$

Equation 31: Formula for added heat

where *c* is the specific heat capacity. The definition of *c* changes depending upon the process.

Work done by the system can be defined as

$$dw = pd\nu = 0 \; at \; constant \; volume.$$

Equation 32: Work done at constant volume

Therefore,

$$de = dq = c_\nu dT,$$

Equation 33: Change in internal energy at constant volume

where c_v is the specific heat capacity at constant volume.

Enthalpy, which is the measure of total static energy in the system, is given by:

$$h = e + pv$$

Equation 34: Static enthalpy formula

and at constant pressure:

$$dh = de + pdv = de + dw = dq = c_P dT,$$

Equation 35: Static enthalpy relation to temperature

where c_p is the specific heat capacity at constant pressure.

Therefore, in a general process, using the ideal gas law:

$$dh = de + d(pv) = de + d(RT) = c_v dT + RdT$$

Hence,

$$c_v dT + RdT = c_P dT$$

$$\Rightarrow c_P - c_v = R$$

Equation 36: Relation between specific heat capacities and specific gas constant

c_P and c_v can also be related using γ, also known as the heat capacity ratio:

$$\frac{c_P}{c_v} = \gamma$$

Using the last two relations:

$$c_P = \frac{\gamma R}{\gamma - 1} \text{ and } c_v = \frac{R}{\gamma - 1}$$

Equation 37: Formula for the specific heat capacities

2.4.6 Isentropic relation between gas states

$$ds = \frac{dq}{T} = \frac{de + dw}{T} = \frac{c_v dT + pdv}{T} = \frac{c_v dT + d(pv) - vdp}{T}$$
$$= \frac{(c_v dT + RdT) - \frac{RTdp}{p}}{T} = \frac{(c_P dT)}{T} - \frac{Rdp}{p}$$

Therefore,

$$s_2 - s_1 = \int_1^2 \rho \left[\frac{(c_P dT)}{T} - \frac{Rdp}{p}\right] = \int_1^2 \rho \left[\frac{(c_P dT)}{T} - \frac{Rdp}{p}\right] = c_P \ln\left(\frac{T_2}{T_1}\right) - R \ln\left(\frac{p_2}{p_1}\right)$$

Equation 38: Formula for entropy change

As $s_2 - s_1 = 0$ for isentropic process,

$$c_P \ln\left(\frac{T_2}{T_1}\right) = R \ln\left(\frac{p_2}{p_1}\right)$$

Using the formula for c_P and cancelling γ from both sides

$$\left(\frac{T_2}{T_1}\right)^{\frac{\gamma}{\gamma-1}} = \left(\frac{p_2}{p_1}\right)$$

Using the ideal gas proportionality:

$$\left(\frac{T_2}{T_1}\right)^{\frac{\gamma}{\gamma-1}} = \left(\frac{p_2}{p_1}\right) = \left(\frac{\rho_2}{\rho_1}\right)^{\gamma}$$

Equation 39: Isentropic gas state ratio relations

2.5 Compressible flow

Till before this section, all the prior explanations assume that the flow is incompressible. Hence, it is important to understand what incompressible flow means. Air is compressible meaning it changes in volume under higher pressure, while, according to Boyle's Law, keeping the same temperature. This does not mean that all airflow is always compressible: A Compressible flow is different from a compressible fluid. A fluid being incompressible means something which would not change its volume per unit mass, i.e., specific volume, under higher pressure, keeping temperature constant; a very common example here is water. On contrary, a flow being incompressible means that any fluid particle in a fluid flow, does not change its specific volume but different fluid particles can have different specific volumes (if the FLUID is compressible). A very common example of an incompressible flow with a compressible fluid is airflow with a speed much lower than that of sound.

Condition for considering a gas-flow to be compressible:

On modifying Equation 4 by neglecting the viscous and the body forces and assuming a steady flow:

$$UdU + \frac{dp}{\rho} = 0$$

Equation 40: Momentum conservation for a flow, free of external forces and viscosity, over a differential element of length dx

If there is a free-stream flow of U_∞, the maximum change in that can occur to the speed is $-U_\infty$. Hence, the maximum change in dp/ρ is:

$$\frac{dp}{\rho} = U_\infty^2$$

Equation 41: Momentum conservation depicting the maximum change in static pressure possible due to stagnation

Replacing p with an expression including ρ using the ideal gas law:

$$\frac{RTd\rho}{\rho} = U_\infty^2$$

Equation 42: Modified Equation 41, using the ideal gas law

It is considered as a rule of thumb that a change of more than 5% is considered significant enough to not neglect it. Therefore, for compressible flow:

$$\frac{d\rho}{\rho} > 0.05$$

Hence,

$$U_\infty^2 > 0.05RT \approx (0.2a)^2,$$

where a is the speed of sound.

Therefore, the condition for considering a flow to be compressible is:

$$U_\infty \gtrsim 0.2a$$

Or

$$M \gtrsim 0.2$$

Equation 43: Condition for compressibility in gas flow

where M is known as the Mach number, quantifying the ratio of velocity magnitude to the speed of sound.

2.6 Isentropic flow relations

Total temperature is a measure of the total enthalpy in terms of temperature:

$$T_0 = T + \rho\frac{U^2}{2c_p}$$

Using $U^2 = M^2a = M^2\gamma RT \; and \; c_P = \frac{\gamma}{\gamma-1}R$, the following is achieved:

$$\frac{T_0}{T} = 1 + \frac{\gamma - 1}{2} M^2$$

Equation 44: Total temperature and Mach number relation

This can be converted into a ratio of two static temperatures:

$$\frac{T_2}{T_1} = \frac{1 + \frac{\gamma - 1}{2} M_1^2}{1 + \frac{\gamma - 1}{2} M_2^2}$$

Equation 45: Temperature ratio relation to Mach number

Furthermore, Equation 39 can be used to derive similar ratios for ρ and p.

Another important relation is the relation between the cross-sectional area of flow and the Mach number.

$$A = \frac{\dot{m}}{\rho U}$$

Equation 46: Area as a function of mass flow rate, density and velocity

Using

$$\frac{\rho_0}{\rho} = \left(1 + \frac{\gamma - 1}{2} M^2\right)^{\frac{1}{\gamma - 1}}, \qquad U = M\sqrt{\gamma R T} \text{ and Equation 44:}$$

$$A = \frac{\dot{m}}{\rho_0 \left(1 + \frac{\gamma - 1}{2} M^2\right)^{\frac{-1}{\gamma - 1}} M \sqrt{\gamma R T_0 \left(1 + \frac{\gamma - 1}{2} M^2\right)^{-1}}}$$

$$\Rightarrow A = \frac{\dot{m}}{\rho_0 M \sqrt{\gamma R T_0}} \left(1 + \frac{\gamma - 1}{2} M^2\right)^{\frac{\gamma + 1}{2(\gamma - 1)}}$$

Equation 47: Area as a function of Mach number, mass flow rate, stagnation temperature and density, and gas properties

A^* being the critical area, is defined as the area where $M = 1$:

$$A^* = \frac{\dot{m}}{\rho_0 \sqrt{\gamma R T_0}} \left(1 + \frac{\gamma - 1}{2}\right)^{\frac{\gamma + 1}{2(\gamma - 1)}}$$

Equation 48: Critical area

Therefore,

$$\frac{A}{A^*} = M^{-1}\left(\frac{1 + \frac{\gamma - 1}{2}M^2}{1 + \frac{\gamma - 1}{2}}\right)^{\frac{\gamma+1}{2(\gamma-1)}}$$

Equation 49: Area to critical area ratio as a function of Mach number

$$\frac{A_1}{A_2} = \frac{M_1^{-1}\left(1 + \frac{\gamma - 1}{2}M_1^2\right)^{\frac{\gamma+1}{2(\gamma-1)}}}{M_2^{-1}\left(1 + \frac{\gamma - 1}{2}M_2^2\right)^{\frac{\gamma+1}{2(\gamma-1)}}}$$

Equation 50: Area ratio relation with Mach number

2.7 Computational Fluid Dynamics

In the early stages of the development of flow solvers, it was realized that the governing equations had too many variables to be solved analytically without any assumptions or simplifications. Usually, an option is to perform experiments capturing the flow characteristics, but it is often very expensive and time consuming to setup and conduct these analyses. Therefore, another option is to use numerical techniques to solve the equations. They are very cheap and fast but sometimes lack in accuracy. With the advent of modern computers and more research, computational methods have become faster and more accurate. This tool or scope of study is called computational fluid dynamics (CFD). Researchers have also started applying machine learning to the realm of CFD but that topic is not relevant to our study.

In CFD, a flow domain is divided into multiple smaller units or cells (can range from thousands to millions). The governing equations are numerically discretized and applied to the flow field in each cell. As there are not enough governing equations, assumptions or extra relations (model) are assumed to close the set of equations. This set of equations is then solved for every cell using an iterative technique. The choice or the order, an iterative technique follows, and how the values at a certain cell should be related to that of the neighboring cell, depends a lot on the nature of the governing equation, translating to the physics of the flow.

2.7.1 Classification of governing equations

How information propagates in a flow field is very important as the CFD algorithm should follow the same order of solving the equations in the cells. Broadly, there are three types of partial differential equations (PDEs): Elliptic, hyperbolic and parabolic. Although this classification can only be applied to linear PDEs, which is not the case with the Navier-Stokes equations because of the convective term, their nature can be understood physically, or by modifying the original equation.

Considering a 2nd order linear PDE in Equation 51, the nature can be mathematically identified by calculating the discriminant, $\Delta = B^2 - 4AC$ (section 4.2, (Anderson, et al., 2009))

$$A\frac{\partial^2 U}{\partial x^2} + B\frac{\partial^2 U}{\partial x \partial y} + C\frac{\partial^2 U}{\partial y^2} + D\frac{\partial U}{\partial x} + E\frac{\partial U}{\partial y} + FU = G$$

Equation 51: 2nd order linear partial differential equation

2.7.1.1 *Elliptic (section 4.3.3, (Anderson, et al., 2009))*

Condition: $\Delta < 0$

These problems are usually steady diffusion processes, with no marching properties (in neither space nor time). Hence, every point in the domain affects (*domain of influence)* and is affected (*domain of dependence)* by change in flow characteristics at all other points in the domain. One such example is the 2D Poisson equation, mentioned in Equation 52.

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = f$$

Equation 52: Poisson equation

2.7.1.2 *Parabolic (section 4.3.2, (Anderson, et al., 2009))*

Condition: $\Delta = 0$

They are usually time-dependent diffusion processes.

For example, the 1D heat equation, mentioned in Equation 53, is parabolic in time, i.e., the flow characteristics at every point is affected by the same at all the point in the domain, in the previous time instant (one way dependence, same as upstream but in a temporal point of view).

$$\frac{\partial u}{\partial t} = \vartheta \frac{\partial^2 u}{\partial x^2},$$

Equation 53: 1D heat equation

where ϑ is the diffusion coefficient, equivalent to the kinematic viscosity for momentum diffusion.

2.7.1.3 *Hyperbolic (4.3.1, (Anderson, et al., 2009))*

Condition: $\Delta > 0$

In case of hyperbolic problems, information only propagates along certain lines, called the characteristic lines, like an acoustic wave. Therefore, the simplest example is the 1D advection equation, mentioned in the following:

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0,$$

where c is real speed.

Another common example is supersonic flow, where the shock waves are the characteristic lines.

2.7.2 Classification of a 2D subsonic boundary layer equation

As the boundary layer equation, mentioned in Equation 12, is not linear, classifying by calculating the discriminant is not possible. In that case, it is possible to identify the nature physically.

In the boundary layer, a point affects all the point in its downstream and is affected by all the points in its upstream. The typical example of this are the boundary layer equations without any separation. As Equation 12 is non-linear, it is not possible to answer the nature mathematically, but it is physically.

In Equation 12, there is no momentum diffusion term in the x direction, only in y along with a convection term in x. This means that information only propagates downstream because of the convection, i.e. due to the flow itself, making it parabolic in x or streamwise direction. The diffusion term makes it elliptic in y or the cross-stream direction.

Hence, it is ideal to march through space in the downstream direction, starting from the inlet boundary condition and solve for the equations at all points with the same cross-stream coordinate simultaneously. This type of problem, where the ideal process is to start from an initial value or boundary condition(s) and march through time or space is also called an initial value problem.

In case of separation under high adverse pressure gradients, the diffusion term in x would no longer be considered negligible, making the flow also elliptic in x.

2.7.3 Discretization techniques

There are three techniques to discretize the domain and equations as well as solve them: Finite difference method (FDM), finite element method (FEM) and finite volume method (FVM).

The finite element method divides the domain into multiple *elements* and applies a weighted residual method on it. FEM can assign varying material properties to different elements. The formulation is energy-based, an ideal choice for solving elasticity problems. On contrary, these equations are also weakly conservative, so it is not an ideal choice for solving fluid flow.

Finite difference method involves an easier formulation and hence it was the choice for CFD, till 1970/80s. As, the computation power started growing at an incredible pace, researchers shifted their focus on FVM, as it naturally ensures conservation and it is also useful for complex geometries.

2.7.3.1 *Finite difference method*

In this discretization technique, the domain is broken into a structured grid with each common point of the grid lines acting as the point of calculation. The strong form or the governing equation as a set of partial

differential equations are discretized numerically using suitable schemes as per the problem in concern. Some model equations are added to close the problem and iterative techniques are used to solve this set of numerically discretized partial differential equation.

The discretized partial differential equations are solved by calculating differences in particular directions (x, y or z). Thus, it is very important for the grid to be structured. Therefore, this approach cannot be used to solve a fluid flow field in a complex geometry.

2.7.3.1.1 Explicit vs implicit scheme

How the flow characteristics in each cell are related to the same in the neighboring cells, is determined by the choice of scheme used: explicit, implicit or semi-implicit.

In explicit schemes, the flow characteristics at a certain point entirely depend upon the values in the previous time step,

$$u_n^i = u_{n-1}^i + h.f(u^{n-1})$$

Whereas in implicit, the structure looks as follows:

$$u_n^i = u_{n-1}^i + h.f(u^n)$$

It should be noted that $f(u^n)$ is a function, consisting of values at the nth time step, at one or multiple points, depending on the order of the numerical discretization chosen.

Let's consider a very low Re and a 1D incompressible flow with no pressure gradient:

$$\frac{\partial U}{\partial t} = \vartheta \frac{\partial^2 U}{\partial x^2}$$

Equation 54: Governing equation for a 1D incompressible flow at very low Reynolds number regime

The same equation when numerically discretized using a:

Explicit scheme or Forward in time and central difference in space (FTCS) at the n-1th time step becomes:

$$\frac{(U_i^n - U^{n-1}{}_i)}{\Delta t} = \vartheta \frac{U^{n-1}{}_{i+1} - 2U^{n-1}{}_i + U^{n-1}{}_{i-1}}{2\Delta x^2}$$

Equation 55: Example of an explicit scheme

Implicit scheme or Backward in time and central difference in space (BTCS) at the nth time step:

$$\frac{(U_i^n - U^{n-1}{}_i)}{\Delta t} = \vartheta \frac{U^n{}_{i+1} - 2U^n{}_i + U^n{}_{i-1}}{2\Delta x^2}$$

Equation 56: Example of an implicit scheme

Semi-implicit or a Crank-Nicolson scheme:

This can be found by just taking an average of the previous two schemes

$$\frac{(U_i^n - U^{n-1}{}_i)}{\Delta t} = \frac{1}{2}\left(\vartheta \frac{U^{n-1}{}_{i+1} - 2U^{n-1}{}_i + U^{n-1}{}_{i-1}}{2\Delta x^2} + \vartheta \frac{U^n{}_{i+1} - 2U^n{}_i + U^n{}_{i-1}}{2\Delta x^2} \right)$$

Equation 57: Crank-Nicolson scheme

The choice of which type of scheme to use, entirely depends on the type of problem, i.e., how the information propagates. How the information propagates physically is determined by the governing equations (section 2.7.1), hence the domain of influence or dependence of a point. The chosen scheme also has its own domain of dependence or influence, which should at least include the physical domain of dependence or influence, respectively, to ensure stability of the solver. Refer to section 5.3, (Anderson, et al., 2009) for more details.

2.7.3.1.2 Dissipative vs dispersive error

A solver might be stable but still not converge to a unique solution. To ensure convergence, consistency is also an essential factor to consider. The error of a solver (difference between the analytical and the numerical solution) needs to decrease with reduction in grid size (or time step size). Along with these conditions, the problem must also be well-posed. A well-posed problem has a unique solution, which depends on the boundary and the initial conditions.

There are two types of error (section 9.4.2.1, (Anderson, et al., 2009))

Dissipative

The difference between the analytical and the numerical solution is only the magnitude. This difference might be a function of the frequency of the solution as well as the time step or grid size.

Example: - Implicit scheme mentioned in Equation 56

Dispersive

In this case, the error is in terms of a phase difference instead, which, in most cases, depends on the same factors, frequency of the solution and the choice of the stepping value. If an analytical solution is supposed to be comprised of multiple frequency harmonic components (which is usually the case), the phase error can vary with the frequency, which might lead to wiggles in the otherwise smooth solution.

Example: - Explicit scheme mentioned in Equation 55

2.7.3.2 *Finite volume method*

Here, the domain is again divided using a grid and each unit control volume is called a cell (refer to chapter 11, (Anderson, et al., 2009) for details). On these control volumes, the numerically discretized weak form or integral of the well-known governing equations is applied. As, the

scheme directly calculates the flux through the surfaces of a control volume, the conservation laws are satisfied by default. Lax-Wendroff scheme (1960) and the Godunov scheme (1959), which mainly focused on satisfying the conservation laws, laid the foundation of the Finite Volume Method. Along with the development of the better turbulent models, FVM was also being revised. Later, it got merged with the RANS solver (covered in section 2.7.4) and started being used in commercial software programs.

They also have similar characteristics as the FDM, with some minor changes. In case of the flux calculation, the discretization can be central difference as well, but in this case it just means that the flux is calculated using the flow properties at the cell centre (by using linear interpolation). One should always keep in mind the numerical dissipation/dispersion that comes along with the scheme we choose. Upwind schemes (information in the computational domain propagates in the same direction as in the physical domain. Example: FTBS for positive velocity) are preferred in case of shocks or discontinuities because of their dissipative properties. The same thing is not preferred for smooth and laminar flows. In our context, second order upwind is preferred as the flow experience high adverse pressure gradients and will often separate. As mentioned, it has dissipative properties, so the accuracy won't be as high in the smaller length scales. Since we only care about capturing accurately the larger scale flows and not the fluctuations, it is deemed to be acceptable.

The iterative techniques used with FVM are: BiCGStab, GMRES, Jacobi and Gauss-Seidal or multigrid methods (AMG), especially for complex geometries and larger systems. All these iterative techniques are used by commercial software programs like Ansys Fluent. The choice entirely depends on the type of cell, if it is small, dense or highly stiff system, or the concern lies on coarser grids. It also depends on which governing equation is solved. For details on some of the mentioned iterative techniques, refer to section 9.4.4, (Anderson, et al., 2009).

2.7.4 Turbulence models

As understood from Equation 5, there is an extra term, (Reynold's stress term) that appears in the Navier-Stokes-equations (9 extra terms in a 3D problem). Hence, there is a need to introduce extra equations to close the problem. This extra equation(s) that directly or indirectly relates the Reynolds stress term to the mean flow characteristics is termed turbulence model.

The most common models are as follows, where the ones relevant to this study (Standard k-ω, Standard k-ϵ) are discussed further later on.

Spalart-Allmaras (Spalart & Allmaras, 1994)

This is a one-equation low-Re model (applicable in regions where the viscous force dominate).

Pros: Low computational cost and chances of convergence is better with coarser mesh, compared to the low-Re k-epsilon model. Can be used for external aerodynamics, attached boundary layer flows as well as high Re wall-bounded flow because of its low cost.

Cons: Shear flows, under-prediction of separation, transition, limited compressibility effects, isotropic turbulence assumption (cannot capture anisotropy: Rotating flows and swirling flows) and decaying turbulence.

Standard k-ϵ *(free stream)* (Launder & Spalding, 1974)

This model introduces two transport equations for the turbulent kinetic energy (k) and its dissipation rate (ϵ).

Pros: Relatively accurate for high Reynolds number flows in which the turbulence behaviour is close to homogeneous, and the turbulence production is nearly balanced by dissipation

Cons: Not useful for predicting BL under strong adverse pressure gradient, separation, shear layers and jets (Versteeg and Malalasekera, 2007), high curvature flows, diverging passage.

Standard k-ω (Wilcox, 1998)

Two equation model, solving for turbulent kinetic energy (k) and specific dissipation rate (ω).

Pros: Good near wall treatment, and suitable for high adverse pressure gradients

Cons: Sensitive to free stream conditions, and less performant in free shear flows in comparison to k-ω SST or k-ϵ.

Realizable k-ϵ (Shih, Liou, Shabbir, Yang, & Zhu, 1995)

Eddy viscosity formulation and the transport equation for the dissipation is modified compared to the standard one.

Improvements over the standard one: (1) rotating homogeneous shear flows; (2) boundary-free shear flows including a mixing layer, planar and round jets; (3) a channel flow, and flat plate boundary layers with and without a pressure gradient; and (4) backward facing step separated flows.

k-ω SST (Menter, 1994)

Combines the strengths of k-omega (for near-wall regions) and k-epsilon (for free-stream regions)

Pro: Excellent for BL flows, separation, and adverse pressure gradient; and can also be coupled with transition models (like the next one) to increase the applicability of the model.

Cons: High computation cost compared to the simpler models as it requires fine meshing, over predicts turbulence in free shear layers (jets and wakes). Without any coupling with other solvers, it cannot predict the

transition. It is also less accurate for flows with anisotropy. [Valid sources needed for the pros and cons]

$\boldsymbol{\gamma Re - \theta}$ (Langtry R. , et al., 2006)

This is explicitly used for transition

Pros: Great at predicting transition; applicable to a wider range for Re; great for BL flows; does not over-predict turbulence in laminar region (compared to the purely turbulence models)

Cons: Relies on empirical correlation for Re_θ; Hence, it might not always capture the physics accurately in non-standard conditions. Sensitive to free stream turbulence intensity and other input parameters. Computational expensive. Struggles with flow with strong adverse pressure gradients and separation (non-standard condition).

One can couple the two models (Transition SST 4 equation model in Ansys Fluent), as the strength of SST k-omega, i.e., predicting turbulence, and that of Gamma Re-theta, i.e., predicting transition, complements each other. (Langtry & Menter, 2009)

Alternatively, the empirical correlations for the Re_θcan be made better by including the effects of APG or separation.

2.8 Optimization Code

2.8.1 Relevance and challenges of optimization in engineering

Optimization plays a central role in engineering, enabling the systematic improvement of designs and systems based on mathematical principles. In a wide range of engineering applications, from structural mechanics to fluid dynamics, optimization provides a structured framework to enhance performance, reduce costs, and meet required constraints. (Rao, 2009) Specifically, in aerodynamic and turbomachinery contexts, such as compressor diffuser ducts, optimization can lead to significant performance gains by tailoring geometries for individual needs.

A compressor diffuser duct serves the purpose of decelerating and diffusing high-speed flow, converting kinetic energy into static pressure. The quality of this pressure recovery directly impacts the efficiency of compressors and, by extension, the overall engine or system performance. (Madavan, 2004) Traditional design methods for such components often rely on empirical rules or iterative trial-and-error processes. However, these approaches are increasingly being replaced or augmented by optimization-based techniques exploring the design space more systematically and can reveal non-intuitive but high-performance solutions. (Abhishek, o.a., 2020)

Generally, optimization in engineering refers to the process of identifying the most effective configuration of a system or component to meet specified objectives, subject to real-world constraints. It involves the mathematical formulation and solution of problems where a set of design variables are adjusted to optimize an objective function, albeit minimizing or maximizing it while satisfying certain constraints. Generally, an optimization problem can be formulated as follows:

$$\min_{x \in \mathbb{R}^n} f(x)$$

$$subject\ to\ g_i(x) \leq 0, i = 1,2, \dots, m$$

$$and\ h_j(x) = 0, j = 1,2, \dots, n$$

Equation 58 Basic optimization problem

In this equation, $x \in \mathbb{R}^n$ is the vector of decision variables, $f(x)$ the decision variable, which is to be minimized in this case, $g_i(x)$ are inequality constraints and $h_i(x)$ are equality constraints. The solution is

then to be found within the feasible region, which is being defined by the two constraints at hand. (Nocedal & Wright, 2006)

In engineering applications, typical objectives might include minimizing weight, drag or costs, or on the other hand maximizing efficiency, power output, or pressure recovery. Constraints may include geometric, structural, thermal, operational, or physical limitations, reflecting the realistic physical boundaries in which a design must exist (Rao, 2009)

Applying these concepts to the example at hand of maximizing the pressure recovery through a diffuser duct might lead to the following problem setup: The objective function in this case is to maximize the pressure recovery, while it is constrained by a maximum curvature radius of the walls or by certain characteristics of the internal flow field.

Furthermore, one of the main challenges in engineering related optimization problems is the non-convexity of the problems. Non-convex functions can have multiple different local optima making it challenging for an algorithm to identify the actual global optimum, while still being in the feasible region. (Boyd & Vandenberghe, 2004) The pressure recovery in a diffuser, for example, can vary non - linearly with the geometry due to flow defining characteristics of boundary layer separation and turbulence.

Consequently, it is essential to be aware of the problem one is facing, and to properly address and classify it. As a result it is possible to select the most appropriate algorithm for each problem setup. The following chapters give an introduction into the most important considerations to be made for a general optimization problem and the resulting suitable algorithms.

2.8.2 Classification of Optimization Problems

Engineering optimization problems can be classified along several axes based on their constraints, number of objectives, type of decision variables, and the structure of the mathematical model. This classification is essential for selecting appropriate optimization techniques and understanding the underlying complexity of the problem.

An unconstrained optimization problem implies, that the design variables may vary freely in the domain of the objective function without any constraints restricting the solution space. Hence, from a mathematical point of view, this simply yields in:

$$\min_{x \epsilon \mathbb{R}^n} f(x)$$

This type of optimization problem is typically simpler and can be solved using gradient based methods such as steepest descent or Newton's method, if the function is differentiable in $\mathbb{R}$. (Nocedal & Wright, 2006)

However, in real world applications the lack of constraints, such as physical limits or performance related thresholds is unlikely. Hence, it is essential to consider a constrained optimization.

As aforementioned, in a constrained optimization problem the optimal solution to the objective function must satisfy one or more constraints as well. This requirement yields in

$$\min_{x \epsilon \mathbb{R}^n} f(x)$$

$$subject\ to\ g_i(x) \leq 0, i = 1,2, \dots, m$$

$$and\ h_j(x) = 0, j = 1,2, \dots, n$$

Equation 59 Basic optimization problem (constrained)

If a candidate fulfills the present constraints, it is deemed a feasible solution for the objective functions. After having established possible constraints to the objective function it is essential to consider, how they can be integrated efficiently in the algorithm process. (Nocedal & Wright, 2006) (Rao, 2009) The handling of those constraints can be split into two main methods: Direct constraint handling and transformation methods. In the case of direct constraint handling, the optimization proceeds with the objective function, if the candidate is within the feasible solution space. If he is not, the optimization proceeds with the constraint function directly aiming to bring it back in the feasible solution space.

Transformation methods on the other hand, transform a constrained problem to an unconstrained one by adding a penalty to the objective function. Hence, the addition of the penalty method or function transforms the objective function into a modified version of the original. Mathematically, as stated by (Bergh, 2018), this can be described as:

$$\tilde{f}(x) = f(x) + \sum_{m=1}^{M} w_m \cdot p_m(x)$$

Equation 60 Transformation method

where $\tilde{f}(x)$ represents the modified or so called pseudo-objective function, $f(x)$ the original one, w_m the weighting factor applied to the m-th penalty and $p_m(x)$ the penalty function itself. As a result the pseudo-objective function is being optimized and each violation of the constraint simply yields in a less suitable candidate. Figure 6 shows how the transformation method is represented graphically (Bergh, 2018) Penalty functions can be divided into two groups: Interior methods are active, when the candidate is within the feasible solution space. They are small when he is well within the solution space, and increase when he is moving towards the constraint boundary. Exterior methods on the other hand, are only applied outside the feasible region and aim to steer the candidates back to the feasible solution space. (Fletcher, 1981)

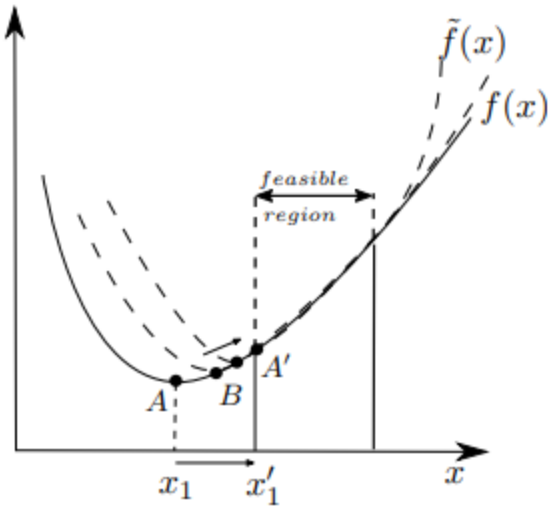


Figure 6 Transformation method (Bergh, 2018)

After having established the basic solution space for an optimization problem it is also essential to define the number of objectives that need to be considered in the process. In a single objective optimization only one objective or variable is maximized or minimized. The previous examples all make use of this classical formulation, where the decision making of the optimization algorithm is driven by a single quantity, e.g. maximize pressure recovery. This approach results in a solution space which is totally ordered. Hence, it is always possible to derive which solution is best. Nevertheless, this simplified problem setup may overlook possible trade-offs occurring in a practical design process. (Kalyanmoy, 2001)

To widen the search and account for possible trade-offs, one can extend the single objective function to a multi objective one. Mathematically, a multi objective problem can be described as

$$\min_{x \epsilon \mathbb{R}^n} \boldsymbol{f}(x) = \left(f_1(x), f_2(x), \dots, f_k(x)\right)$$

In multi objective optimization problems the concept of Pareto-optimality plays a fundamental role. A solution x^* is Pareto-optimal, if no other solution exists that improves any of the objectives. The combined set of all possible solutions is called a Pareto front. It illustrates the trade-offs between different solutions, and one can choose individually what objective to prioritize and hence what solution to choose. (Deb, 2001)

Optimization problems can be further classified based on the type of variables and functions involved. One of the important classifications lies between continuous and discrete optimization. In continuous optimization, the variables can take any value within a given interval, while in a discrete optimization the variables can only take a certain set of given values.

Secondly, it is possible to classify a problem based on its linearity. In a linear problem for instance, the relationship between variables behaves linear. This is a key consideration, since this allows the use of faster converging method such as the Simplex-algorithm. In non-linear optimization problems on the other hand, objective functions or constraints behave in a non-linear way and thus requiring more complex algorithms. (Rao, 2009)

Finally, optimization problems can also be classified whether they are convex or not. In a convex problem or function, the local optimum does also represent a global optimum. Hence, a gradient based algorithm for example, could not return a local minimum as a result of its optimization process. Non-convex problems, however, consist of different local optima and one global one. Thus, they require a different optimization approach, analyzing the solution space globally. (Boyd & Vandenberghe, 2004)

2.8.3 Categorization of Optimization Methods

Generally spoken, one can classify optimization algorithms into deterministic or metaheuristic (stochastic) methods. Deterministic methods follow a predefined path and typically yield in the same solution for a given problem setup every time they are executed. Metaheuristic algorithms, on the other hand, incorporate a dose of randomness and are

particularly useful and effective for complex or poorly understood problem setups.

2.8.4 Deterministic Optimization Methods

As the name already suggests, deterministic algorithms follow a predefined mathematical-based procedure and thus yield in the same optimized solution for a set of initial conditions every time the algorithm is executed. Since their procedure is based on mathematical concepts guiding towards the optima, they rely on analytical properties of the objective and constraint functions such as continuity or differentiability. Hence, these methods are particularly effective when facing problems with a smooth design space. (Nocedal & Wright, 2006)

Among the most widely used deterministic techniques are gradient-based methods, utilizing the first and sometimes second derivatives of the objective function to determine the direction of improvement. Classical methods such as steepest descent, Newton's method, and quasi-Newton methods fall into this very category. These methods are well-suited for solving unconstrained or mildly constrained problems where the objective function needs to be differentiable and convex. If those properties of the objective and possibly constraint function are given, they provide an approach with a rapid convergence towards the optimum especially when exploiting second-order information. (Nocedal & Wright, 2006)

In constrained optimization, deterministic methods need to become more sophisticated. Techniques such as Sequential Quadratic Programming and interior-point methods are commonly used for problems involving nonlinear constraint functions. Sequential Quadratic Programming, for example, solves a sequence of quadratic sub-problems approximating the original nonlinear problem. These approaches are highly efficient for small to medium-scale problems and have a strong theoretical foundation based on the Karush–Kuhn–Tucker optimality conditions. (Rao, 2009) (Nocedal & Wright, 2006)

The major advantage of deterministic methods is their efficiency and convergence speed in well-structured problems. They provide strong theoretical guarantees, particularly in convex settings. However, a key limitation is their tendency to converge to local optima in non-convex problems, which are typical in complex engineering applications. To avoid the local optima for these scenarios they would heavily rely on the initial conditions guiding the algorithm to a global optimum not a local one. Additionally, these methods require gradient information, which can

be difficult to obtain or unreliable when using high-fidelity simulations like Computational Fluid Dynamics. (Jameson, 1988)

2.8.5 Metaheuristic Methods

In contrast to deterministic approaches, metaheuristic algorithms are high-level strategies guiding a usually stochastic search process. They are designed to be robust and general-purpose, making minimal assumptions about the nature of the objective function. These methods incorporate randomization to explore the search space as globally as possible and are hence especially useful for non-convex, discontinuous, or black-box problems where deterministic gradient based methods struggle.

The Genetic Algorithm is one of the earliest metaheuristic algorithm to be developed. It is inspired by the concept of Darwinian evolution. It starts and operates with a population representing candidate solutions. From there it applies the evolutionary concepts off selection, crossover and mutation to further evolve new generations. The fitness-based selection encourages the survival of better-performing candidates ("Survival of the Fittest"), while genetic operators introduce diversity and allow for a global exploration of the design space. Genetic Algorithms are particularly effective for discrete and combinatorial optimization and can also be extended to multi-objective formulations, such as in the Non-dominated Sorting Genetic Algorithm II (NSGA-II). (Kalyanmoy, 2001) (Deb, Pratap, Agarwal, & Meyarivan, 2002; Falkner & Skan, 1931; Falkner & Skan, 1931)

Furthermore, the Particle Swarm Optimization algorithm is another commonly used representative of metaheuristic algorithms. As the name suggests it models the social behavior of a swarm from the animal kingdom such as birds or fish. Each particle in the algorithm will adjust its position in the search space based on two factors: Its previous experience and the behavior of its neighbors. The main advantages of the algorithm lie in its fast convergence and simplicity. (Kennedy & Eberhart, 1995; Kundu, Cohen, & Dowling, 2016)

Among the most effective metaheuristics for continuous engineering optimization is Differential Evolution, which advantages and functionality will be discussed in more detail in the following chapter. Differential Evolution firstly generates an initial population of vectors lying within the feasible region. From there, each member of the population is mutated in order to create a new mutated vector. It is generated by combining one individual vector with two randomly selected ones, thus introducing a diversity to the population. In the next step a crossover between the

original and mutant vector is performed forming the so called trial vector. Whether the components of the trial vector are made up by the mutant or the original depends on a random probability and crossover rate. Finally, the trial vector undergoes a selection process: If it yields in a better result than the original vector it replaces him in the next generation. If not, the original vector is kept. This process is iterated until a predefined convergence criterion is met or the maximum number of generations is surpassed. (Madavan, 2004) (Storn & Price, 1997) (Bergh, 2018)

Generally, metaheuristics are particularly attractive due to their derivative-free nature. Unlike gradient-based methods, they treat the objective function as a black box, requiring only function evaluations. This makes them ideal for problems where the cost function arises from complex simulations and analytical gradients are unavailable or infeasible to compute. (Rashed et al., 2023)

However, metaheuristics are not without drawbacks. They tend to require many function evaluations, which can be computationally expensive, especially when each evaluation of the objective function involves running a high-fidelity simulation. Their convergence to an optimum is slower compared to gradient-based methods, since they don't follow a predefined mathematical pattern. While they are designed to avoid local optima, they cannot guarantee that the global optimum will be found. Hence, they are defined as stochastically algorithms. Furthermore, their performance can be sensitive to algorithmic parameters such as the population size, mutation rate, or inertia weight, which thus often require further tuning. (Price, Storn, & Lampinen, 2005)

Even though metaheuristic algorithms have the presented disadvantages or limitations they are still to be found in various engineering applications, since they incorporate the ability to handle non-convex and high dimensional problems, while having non-differentiable objective functions as well.

2.9 Purpose of an Inverse Design Process

Traditional "direct" aerospace design methodologies revolve around defining a geometrical shape, computing the aerodynamic performance of this shape and then analyzing and usually re-designing the shape until the outcome meets a given set of criteria (say a suitable pressure profile). Inverse design routines invert this process by first defining the preferred outcome of the design process first (say the preferred pressure profile), and then performing an inverse design process to generate the geometry which results in these outcomes. That is for example, while in direct

design, the pressure profile for an airfoil is the output of the design process, in the inverse approach, the preferred (usually optimized) pressure profile is an input. Using target aerodynamic profiles of pressure, loss and other quantities adds the advantage that these profiles can be designed and optimized ahead of time subject to constraints such as boundary layer thickness and shape factor, to provide the maximum possible aerodynamic performance for a component, for which the associated geometry can be generated. Recent work in the design of axisymmetric S- or Swan Neck diffuser ducts (SNDs) has shown that predesigning an optimized pressure recovery profile can result in significantly shortened geometries, with high but robust performance across operating ranges, resulting in reductions in engine length and specific fuel consumption (SFC). Design of such a framework also has benefits for other components, such as struts and rotating / stationary blade rows, where highly performant airfoils can then be designed, whilst avoid costly separations and overly thickened boundary layers. The design and implementation of such a system has the potential to be integrated into a number of existing design tools and therefore have wide ranging impact of the next generation of aerodynamic components for flying engines. Refer to section 3.3 for a detailed analysis of the algorithm.

3 Methodology

3.1 Differential Algorithm

3.1.1 Choice of algorithm

The design space for a duct optimization problem is often high-dimensional and complex, especially when multiple parameters are involved, such as duct shape and flow control variables. The response surface, which relates pressure to design variables, is likely to be nonlinear and may contain multiple local optima. For instance, certain shapes might create localized recirculation zones that improve pressure locally, but another shape might yield a larger global pressure increase. In such multimodal landscapes, deterministic methods struggle, as they are prone to getting stuck in local optima. (Sagonda & Folly, 2022)

Moreover, duct optimization typically involves various constraints, such as limits on duct length, diameter, pressure drop restrictions elsewhere in the system, or fabrication constraints. Differential Evolution has been shown to handle such constraints effectively. It incorporates penalty functions and follows those general iterative steps in the optimization process as described by (Bergh, 2018):

1. Initial population of candidate vectors $(\bar{x})$ is generated within in the feasible solution
2. In the second step each candidate is mutated to produce the so called mutant vector $\bar{v}$, where

$$v_g = x_{g,r0} + F \cdot \left(x_{g,r1} - x_{g,r2}\right).$$

Equation 61 Mutation step

$x_{g,r0|r1|r2}$ present three different candidate vectors and F some weighting factor. Based on the three vectors the mutant vector is created and by adjusting F it is possible to vary the distance from the mutant vector to the initial vector $x_{g,r0}$. An increase in the distance yields in an higher exploration of the solution space, since more vectors are created further away from the initial set of candidates.

3. After mutation the cross-over step is executed. It involves producing a so called trial vector based on a component-wise crossover between mutation and initial vector. This can be described as:

$$u_i = \begin{cases} v_i \; if \; rand_i(0,1) \leq Cr \; or \; i = \; i_{rand} \\ x_i \; otherwise \end{cases}$$

Equation 62 Crossover step

Cr represents another weighting factor. An increase yields again in a more aggressive search of the solution space, while a decrease helps

refining existing solutions, especially when the candidates already converge to the optimum. Hence, it is important to find the balance between exploration and exploitation.

4. Based on the creation of the trial vector, the selection process is as follows:

$$x_{g+1} = \begin{cases} u_g \; if \; f(\bar{u}_g) \leq f(\bar{x}_g) \\ x_g \; otherwise \end{cases}$$

Equation 63 Selection process

where x_{g+1} represents the next generation of candidate vectors

5. The steps of initialization, mutation, crossover and selection are repeated until some convergence criterion is met. (Bergh, 2018)

Figure 7 by (Bergh, 2018) visualizes the described process from initialization to mutation and crossover until selection.

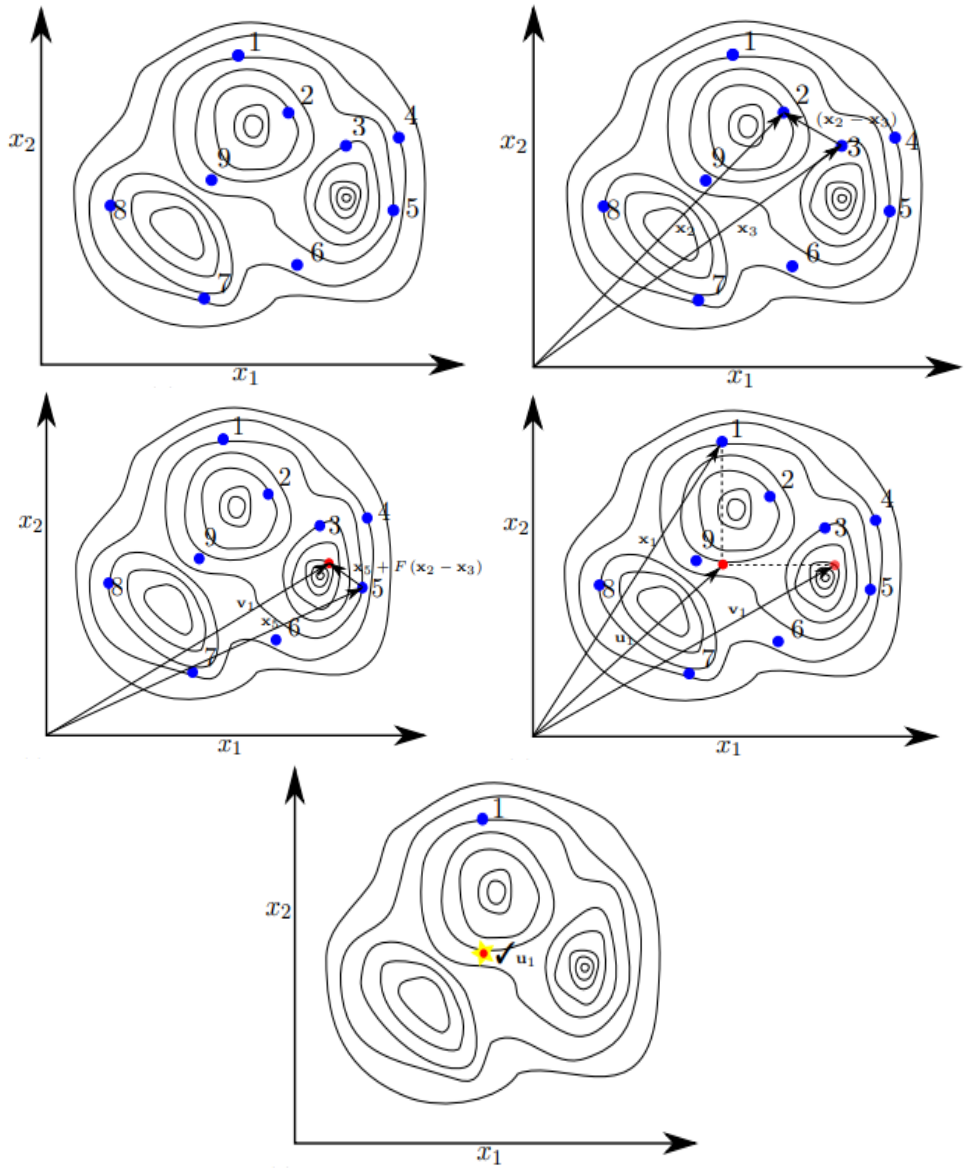


Figure 7 Differential Evolution algorithm (Bergh, 2018)

The ultimate goal in the process of diffuser optimization is to maximize the pressure at the outlet, thus finding the best possible design in a challenging and noisy design space. Given the potential for noisy evaluations (such as those resulting from CFD simulations) and the presence of multiple local optima due to varying flow patterns, a metaheuristic approach offers the robustness needed. Differential Evolution, in particular, is easy to implement and tune, while still maintaining population diversity through its differential mutation and crossover mechanisms. This makes it less sensitive to initial parameter guesses, providing a reliable approach to finding the global optimum. (Madavan, 2004)

Lastly, the use of metaheuristics like DE is not untested, as they have a successful track record in fluid dynamics problems, including HVAC duct design and aerodynamic duct shape optimization, further supporting their applicability in this context. (Hall, Ahuja, Hartfield, Shelton, & Ahmed, 2012) (Nili-Ahmadabadi, Aghabozorgi, Cho, & Kim, 2021)

3.1.2 Algorithm Design

As part of this study, a custom optimizer was developed to generate physically consistent wall pressure distributions which can for example be used in designing a diffuser duct through the inverse design method. The optimizer is based on a Differential Evolution algorithm being, as previously explained, a population-based global optimization method well-suited for handling those nonlinear and constrained problems. The goal of the optimizer is to determine the optimal vertical coordinates of control points defining the upper and lower wall pressure profiles, represented via interpolated splines of the single coordinates. In the optimization process the first point of both curves is fixed, since it forms the initial condition. The other points can move freely in the vertical direction. (Figure 8)

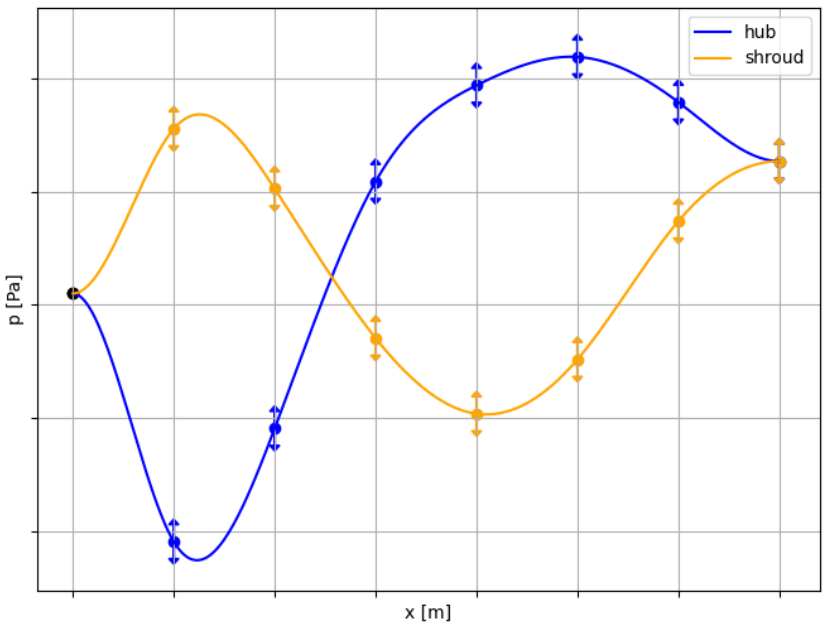


Figure 8 Setup optimization problem

By adjusting those spline points the optimization aims to maximize (or minimize, based on configuration) the outlet parameter of the diffuser,

while simultaneously satisfying multiple design constraints. These constraints are enforced using the transformation penalty method, in which violations are numerically penalized and added to the objective function value creating the aforementioned pseudo-objective function. The specific constraints are further explained and justified in the following subsections. The optimizer is fully configurable through an input file. This modular structure allows for a flexible experimentation with design scenarios and objective configurations, not only bound to an optimal pressure distribution of an ICD.

3.1.3 Constraints of wall pressure distribution

To ensure the physical feasibility of an s-shaped diffuser in the inverse design process, it is crucial to consider the conservation of mass and momentum in the transverse direction. Assuming the inlet and outlet flow angles are both zero, the net change in y-direction momentum within the control volume must be zero as well. Applying and simplifying Newton's second law in the y-direction yields in

$$\sum F_y = 0$$

$$\int_0^L (P_{up} - P_{low})\, dx + \int_0^{\Delta H_{up}} \tau_{up}\, dy + \int_0^{\Delta H_{low}} \tau_{low}\, dy = 0$$

Equation 64 Area constraint

where P_{up} and P_{low} represent the upper and lower static wall pressures. In the idealized case where the viscous forces are neglected the equation above simplifies to

$$\int_0^L (P_{up} - P_{low})\, dx = 0$$

Equation 65 Area constraint without viscous forces

This equation implies that the net pressure force in the y-direction must vanish. Geometrically this translates to the fact, that the areas enclosed by the upper and lower pressure wall pressure distributions add up to zero. Hence, area 1 and area 2 in Figure 9 must be equal to ensure zero inflow and outflow angles.

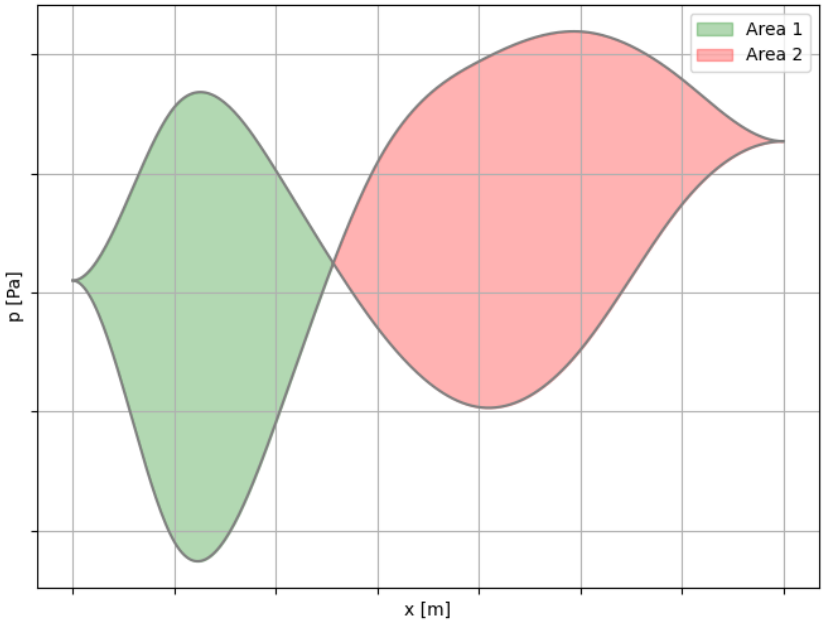


Figure 9 Area equality constraint

Secondly, the initial guess plays a crucial role in defining the constraint conditions for the wall pressure distribution during the optimization process, since a higher “trapped” area yields in a more aggressive duct design. Usually an initial the pressure distribution along the upper and lower walls is given and used to evaluate the “trapped area” of this first guess. This area is then treated as a constraint ensuring that the resulting optimized pressure distribution corresponds to an s-shaped duct with the same $\Delta r/_L$ ratio of the initial duct. Mathematically, this constraint is implemented as follows:

$$Area_1 = Area_2 = Area_{initial}$$

If one takes for example a ratio of $\Delta r/L = 0.3$, the initial pressure distributions define an enclosed area that is then tried to be matched throughout the optimization process. This not only preserves the target duct curvature but also ensures that the final pressure profile conforms to the physical constraints of the system. (Nili-Ahmadabadi, Aghabozorgi, Cho, & Kim, 2021)

Lastly, it is essential to ensure convergence in the process of geometry generation. Hence, the derivative of the mean pressure distribution from hub and shroud must be zero in the region near the inlet and outlet. This has been enforced by applying another constraint to the optimization problem focusing on the derivative being zero in those respective areas.

3.2 Wilcox EddyBl program

To evaluate the aerodynamic feasibility of candidate pressure distributions during optimization, a boundary layer code is employed to predict flow separation. Rather than simulating the full 2D or 3D duct geometry at each iteration—which would be computationally expensive—the approach taken here, projects the wall pressure distributions onto a flat plate. The upper and lower wall pressure curves are treated as externally imposed pressure gradient acting along the surface of said flat plate. This transforms the original problem into one of a boundary layer development over a flat surface under a prescribed pressure gradient. By solving the compressible turbulent boundary layer equations along this flat plate using an integration-based method, it becomes possible to efficiently evaluate if the imposed adverse pressure gradient causes separation or not.

3.2.1 Boundary layer code design

The EddyBl program developed by Wilcox is being used to evaluate the separation criterion for each pressure profile. EddyBl solves the compressible boundary-layer equations using the RANS formulations with the boundary layer approximations. These equations are transformed, so they are suitable for numerical marching in the streamwise direction. The program supports the use of different turbulence models such as the k-ω model or the k-ϵ one. Their usage can be configured via the input files or the Windows input interface. As additional part of the input file, the grid and geometry is preset by the user. In this case the grid should be fine enough near the wall to resolve the boundary layer $(y^+ \approx 1)$. Furthermore, the freestream velocity, temperature and pressure as well as the initial velocity profiles are set as an input. As boundary conditions, the static pressure is given at different x-positions along the flat plate.

After having specified the input file, EddyBl uses a parabolic marching algorithm propagating in the streamwise direction to solve the boundary layer equations. Hence, it solves the flow field sequentially station-by-station from the leading edge downstream. This implies that no upstream influence is being assumed in the calculations (ellipticity). As a result, EddyBl cannot capture separating boundary layer flows. When separation occurs, the wall shear stress approaches zero or becomes negative leading to a potentially reverse flow, and consequently non monotonic velocity profiles. This results in a significant upstream influence, which cannot be captured by the purely parabolic marching scheme. Hence, the EddyBl program terminates due to this mathematically ill-posed solution.

Since a flow over a flat plate is being investigated the use of a structured grid is justified. To discretize the equations in the wall-normal direction

the aforementioned implicit Crank-Nicolson scheme is being applied. It is second-order accurate and semi-implicit. Thus, it offers a robust compromise between the stability of fully implicit schemes and the more simplistic approach of the explicit ones. Crank-Nicolson can be illustrated considering this basic diffusion-equation (representative of the ones solved in the Wilcox EddyBl program)

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}$$

Discretizing the equation according to Crank-Nicolson, where the right hand side becomes the average of the second derivative at the current and next point, yields in:

$$\frac{(u_i^n - u_i^{n-1})}{\Delta t} = \frac{1}{2}\left(\nu \frac{u_{i+1}^{n-1} - 2u_i^{n-1} + u_{i-1}^{n-1}}{\Delta x^2} + \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2} \right)$$

Equation 66 Crank-Nicolson scheme

This forms a tridiagonal system, which is being solved by deploying the Thomas algorithm.

3.2.2 Verification Wilcox EddyBl boundary layer code

3.2.2.1 *Falkner-Skan verification*

In the process of choosing an appropriate boundary layer solver it is very important to test its results against proven experimental or numerical results. As a first step, the boundary layer solver was used to compare it to well established numerical solutions. One such class of solutions are the Falkner-Skan similarity solutions, which describe laminar, steady, two-dimensional boundary layers over wedge-shaped bodies with possibly varying pressure gradients. These solutions are particularly suitable for the verification process, since they generalize the Blasius solution for a flow over a flat plate. Hence it is possible to not only include a zero pressure gradient scenario in the comparison, but also one using favorable and adverse pressure gradients.

The starting point to derive the Falkner-Skan equations from the incompressible two dimensional boundary layer equations, which have already been derived in the previous chapter:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

$$u\frac{\partial y}{\partial x} + v\frac{\partial y}{\partial x} = -\frac{1}{\rho}\frac{dP}{dx} + \nu\frac{\partial^2 u}{\partial y^2}$$

Equation 67 2D boundary layer equations

where u and v represent the velocity components in the x - (streamwise) and y - (wall normal) direction respectively. ν represents the kinematic viscosity and $P(x)$ the pressure outside the boundary layer. For the Falkner-Skan equations it is assumed, that the external inviscid flow follows a power law profile of the following form:

$$U_e(x) = Cx^m$$

where the constant $C > 0$ and m represents a pressure gradient parameter. Using the stream function we can derive the similarity solutions:

$$\psi(x,y) = \sqrt{\nu C}x^{\frac{m+1}{2}}f(\eta), \eta = y\sqrt{\frac{C(m+1)}{2\nu}}\,x^{\frac{m-1}{2}}$$

This transformation yields in the Falkner-Skan equations:

$$f^{'''} + ff^{''} + \beta\left(1 - f^{'2}\right) = 0$$

Equation 68 Falkner-Skan equation

where the following boundary conditions apply:

$$f(0) = f'(0) = 0, f'(\propto) = 1$$

In this case, β is related to the pressure gradient parameter m in the following way:

$$\beta = \frac{2m}{m+1}$$

The Falkner-Skan solution, can generally divided in the following three use cases, depending on the pressure gradient parameter:

- $m > 0$ favorable pressure gradient
- $m = 0$ zero pressure gradient, Blasius solution
- $m < 0$ adverse pressure gradient

The Falkner-Skan equations are a third order non-linear differential equation and must be solved numerically. (Falkner & Skan, 1931) (Schlichting & Gersten, 2016)

To validate the EddyBl boundary layer code by Wilcox, the Falkner-Skan equations have been solved for three different use cases: Zero ($m = 0$), favorable ($m = 3.5$) and adverse pressure gradient ($m = -0.08$).

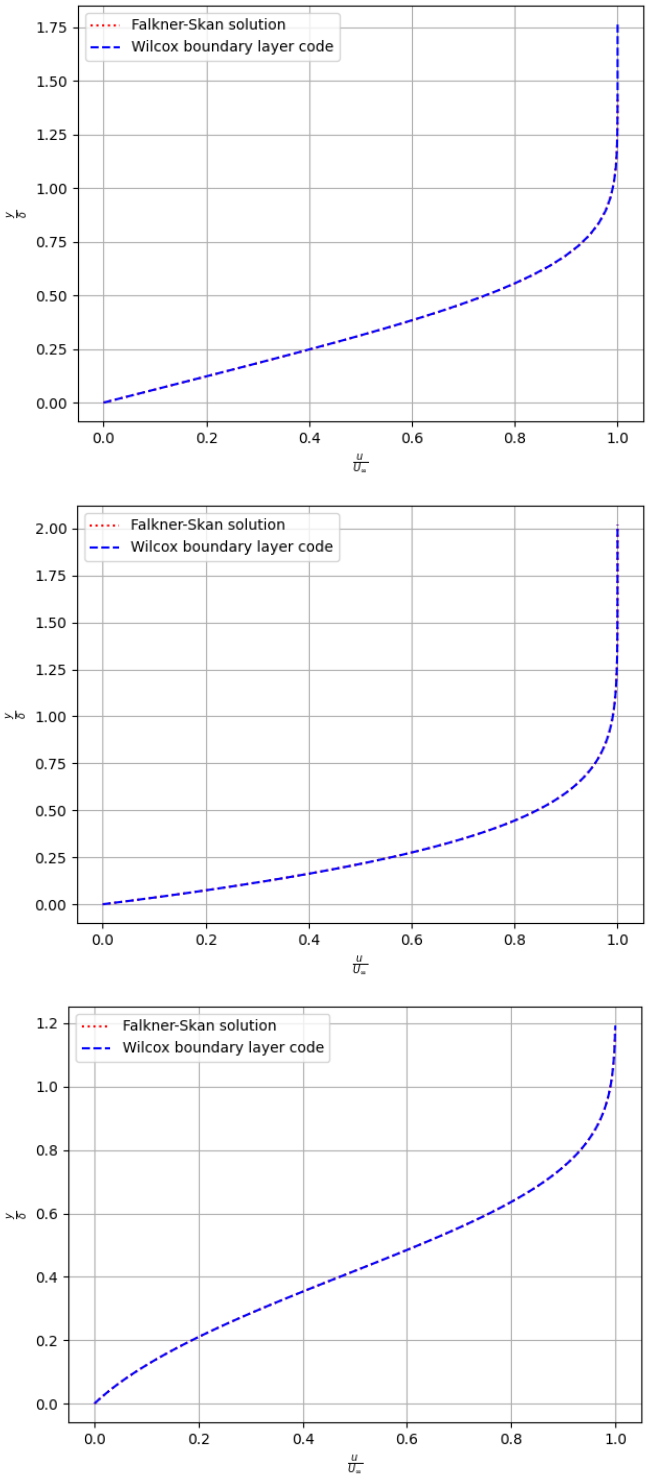


Figure 10 Comparison Falkner Skan and EddyBl numerical solutions for (a) m=0, (b) m>0 and (c) m<0

The three plots (Figure 10) show the comparison between the numerical boundary layer solver (EddyBl) and the numerical Falkner-Skan similarity profiles for different pressure gradient conditions. The different conditions can be split in the three typical cases: Favorable pressure gradient, neutral pressure gradient, and adverse pressure gradient. It is possible to observe that both numerical solutions collapse well on top of each other and showing a good agreement between the two. (Schlichting & Gersten, 2016)

3.2.2.2 *Turbulent boundary layer verification*

The second verification step of the numerical boundary layer code is performed in the turbulent regime. In particular, the computed mean velocity profiles are compared to the empirical and semi-analytical results derived from Spalding's Law of the Wall (1961), a well-known composite formulation that bridges the viscous sublayer, buffer layer, and log-law region of turbulent boundary layers. Unlike the classical logarithmic law, Spalding's expression offers a continuous and differentiable velocity profile valid throughout the inner boundary layer, making it a suitable reference model to validate the numerical solution.

The Law of the Wall, as it was proposed by Prandtl presents the base of Spalding's Law of the Wall. Prandtl's analysis of turbulent boundary layers led to the conclusion that the near-wall velocity u depends on the wall shear stress τ_w, the distance from the wall y, the fluid density and viscosity ρ, μ respectively, resulting in the general form:

$$u = f(\tau_w, y, \mu, \rho)$$

By using dimensional analysis as well as defining the friction velocity as $u_* = \sqrt{\frac{\tau_w}{\rho}}$ and the viscous length scale as $\nu = \frac{\mu}{\rho}$, this relationship yields in the following non-dimensional form:

$$u^+ = f(y^+) \; with \; u^+ = \frac{u}{u_*}, \; y^+ = \frac{y\, u_*}{\nu}$$

When considering a logarithmic region this gives the well-known form of the so called "log-law":

$$u^+ = \frac{1}{\kappa}\ln(y^+) + B$$

Where $\kappa \approx 0.41$ is the von Kárman constant and $B \approx 5.0$ is an empirical integration constant. However, the equation above presents two

problems: Firstly, it does not satisfy the no slip condition and secondly, it fails to have “zero slope” of the velocity profile at the outer edge of the boundary layer.

To address these issues Spalding proposed a single composite formula capturing the entire near-wall velocity profile. The formula defines y^+as a function of u^+ as follows:

$$y^+ = u^+ + A\left[e^{\kappa u^+} - 1 - \kappa u^+ - \frac{(\kappa u^+)^2}{2!} - \frac{(\kappa u^+)^3}{3!} - \frac{(\kappa u^+)^4}{4!}\right],$$

Equation 69 Spalding's law of the wall

where the recommended constants are $\kappa = 0.4$ and $A = 0.1108$.

Spalding’s formulation bridges all wall regions and satisfies the no-slip condition at the wall. Even though it does not enforce the zero-gradient condition of the velocity profile at the boundary layer edge, it offers a substantial improvement in accuracy and smoothness, especially for computational implementations. A notable feature of Spalding’s law is, that it inverts the typical relationship between y^+ and u^+ as it expresses y^+ in terms of u^+ and thus facilitating the numerical integration for turbulent boundary layers. Additionally, this approach avoids the piecewise definitions of the viscous sublayer $(y^+ < 5)$, the buffer layer $(5 < y^+ < 30)$ and the logarithmic region $(y^+ > 30)$, as it includes all into one single formula. (Mandal & Mazumdar, 2015)

To verify the accuracy of the numerical boundary layer solver, the computed inner-scaled velocity profiles (u^+ vs y^+) are compared to Spalding’s law in canonical turbulent boundary layer conditions (zero pressure gradient over a flat plate). The solver is expected to reproduce the composite curve within accepted tolerances, particularly in the near-wall region where the gradient is steep and resolving accuracy is crucial.

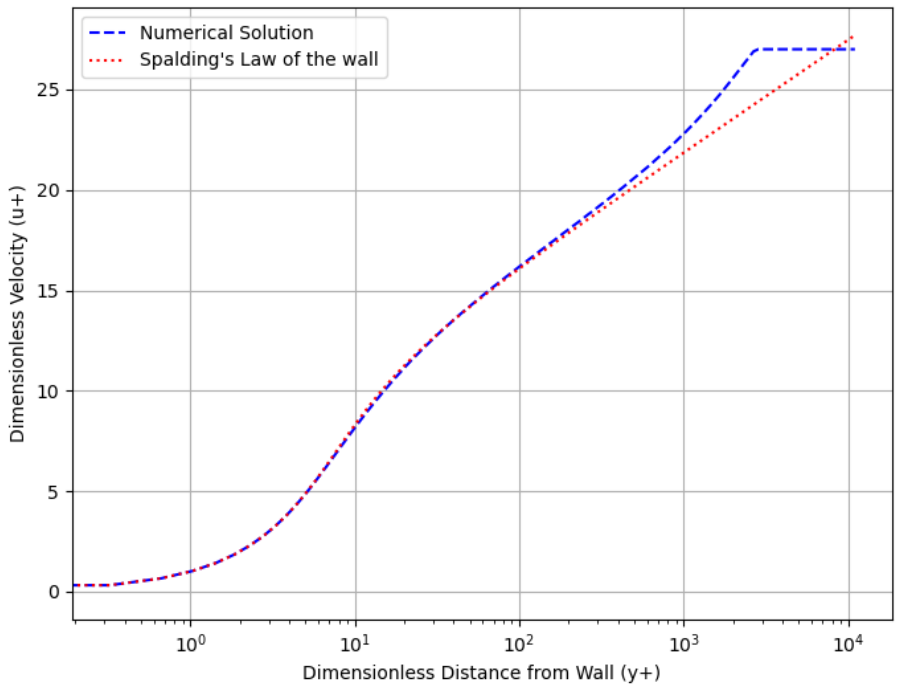


Figure 11 Turbulence flow validation

In Figure 12, this dimensionless velocity profile (u^+) is plotted against the inner-scaled wall distance (y^+) on a logarithmic axis. The numerical solution by the EddyBl program shows excellent agreement with Spalding's law in the viscous sublayer ($y^+ < 5$), the buffer region ($5 < y^+ < 30$), and the logarithmic layer ($30 < y^+ < 300$). However, in the outer region ($y^+ > 1000$), the numerical profile begins to deviate from Spalding's curve, showing higher velocities.

This discrepancy arises from the fact that Spalding's law is a composite formulation primarily designed to fit the inner boundary layer. It lacks a wake component and thus tends to under predict velocity in the outer region, where the effects of turbulence structures and freestream interaction become dominant. Additionally, the flat portion of the numerical profile indicates that the flow has approached the freestream velocity $U_\propto$, beyond which u^+ remains constant. This plateau is however expected and physically represents the boundary layer edge. (Mandal & Mazumdar, 2015)

3.2.3 Comparison / Choice of turbulence models

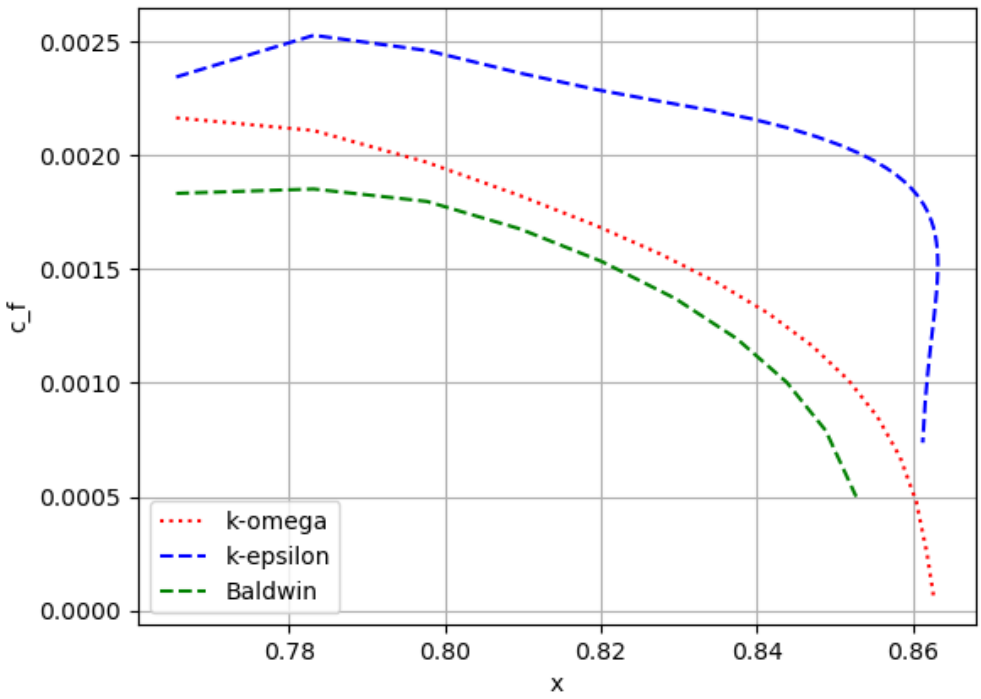


Figure 12: Turbulence model comparison

In Figure 12 above, the skin friction coefficient c_f is plotted as a function of the stream wise location x on a flat plate exposed to a strong adverse pressure gradient forcing flow separation. The graph compares the performance of three different turbulence models: the k-ϵ model the k-ω model and the Baldwin one-equation model. As one expects in an adverse pressure gradient environment, the skin friction decreases along the plate and eventually becomes negative, signaling the onset of boundary layer separation.

During the numerical simulation each of the three models predicts separation at a different stream wise location:

- Baldwin one-equation model at $x = 0.630$
- Standard k-ϵ model at $x = 0.712$
- Standard k-ω model at $x = 0.677$

This ordering is consistent with the known characteristics of these turbulence models in an adverse pressure gradient environment.

The Baldwin one-equation model is an algebraic model with a single transport equation for the eddy viscosity. While it is computationally efficient, it lacks detailed treatment of turbulent transport phenomena and is highly sensitive towards an adverse pressure gradient. This often results in premature separation predictions, as seen in the results above. (Baldwin & Lomax, 1978)

The Standard k-ω model solves two partial differential equations: one for the turbulent kinetic energy k and another for the specific dissipation rate ω. It is known for its superior performance in near-wall regions and flows with strong adverse pressure gradients. The model is more responsive to changes in boundary layer conditions, making it more reliable for capturing separation onset compared to the k-epsilon model, though it still predicts separation somewhat earlier than observed in experimental data (Launder & Spalding, 1974) (Wilcox, 1998)

The k-epsilon solves transport equations for turbulent kinetic energy k, as the k-omega does, and its dissipation rate ε. It performs best in free shear flows such as jets and wakes, but is less accurate in wall-bounded and separated flows. Under the influence of a strong adverse pressure gradient, it often over predicts the turbulent mixing, hence artificially stabilizing the boundary layer and as a result delaying separation. (Launder & Spalding, 1974) (Pope, 2000)

To summarize, as one can observe from Figure 12, the numerical solver, shows a consistent behavior with what can be expected from each of the presented turbulence models. The Baldwin model, being more sensitive towards an adverse pressure gradient, predicts as a result the earliest separation, while the k-epsilon model—known to be more resistant to separation—delays it the most. The k-omega model lies in between, both in terms of complexity and sensitivity. (These results suggest that the boundary layer code not only implements the models correctly but also incorporates their characteristic behavior with respect to adverse pressure gradients and separation tendencies accurately as one might expect. (Mandal & Mazumdar, 2015)

3.2.4 Example EddyBl program

After having established the accuracy and reliability of the results produced by the numerical solver, the Figures below (Figure 13, Figure 14) show a possible outcome after executing the Wilcox boundary layer solver for the laminar flow case. In every computational step the basic values such as free stream velocity or skin friction, are written to an output file. Additionally, it is possible to write the full velocity profiles in a certain interval to the output file, as well. The Figures below illustrate the results after generating the velocity profiles at four different x-positions.

The second set of images represents a strong adverse pressure gradient applied to flow over a flat plate. Hence, it is possible to observe, how the velocity gradient near the wall reduces over distance, until it almost becomes negative, indicating flow separation. The first set of images is facing a neutral pressure gradient, hence the velocity profiles remain constant over the different x-positions.

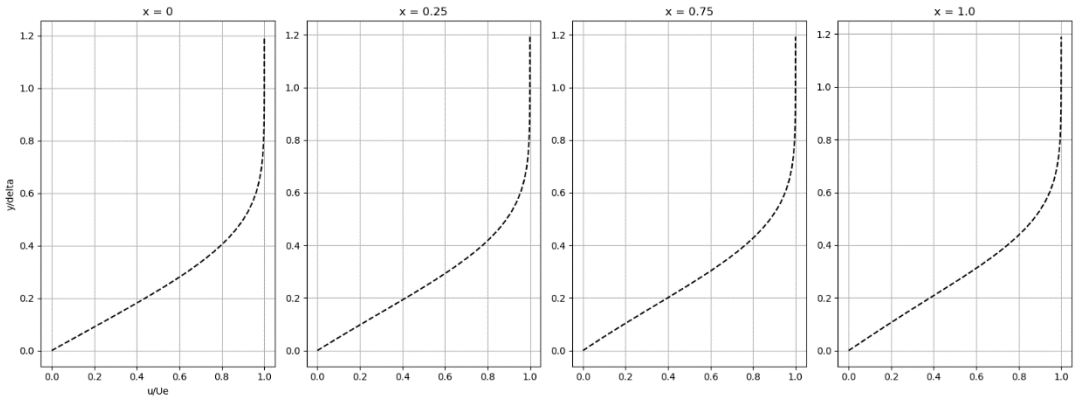


Figure 13 Example EddyBl program with neutral pressure gradient

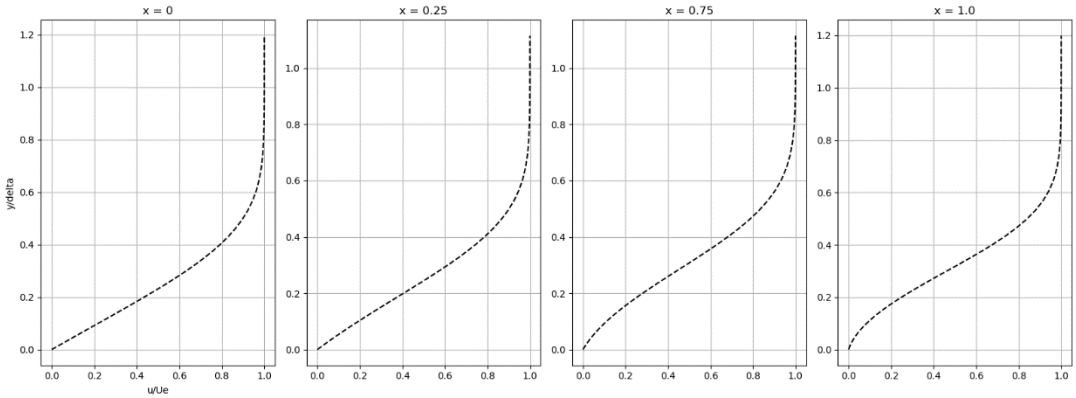


Figure 14 Example EddyBl program with adverse pressure gradient

3.3 Inverse design Framework

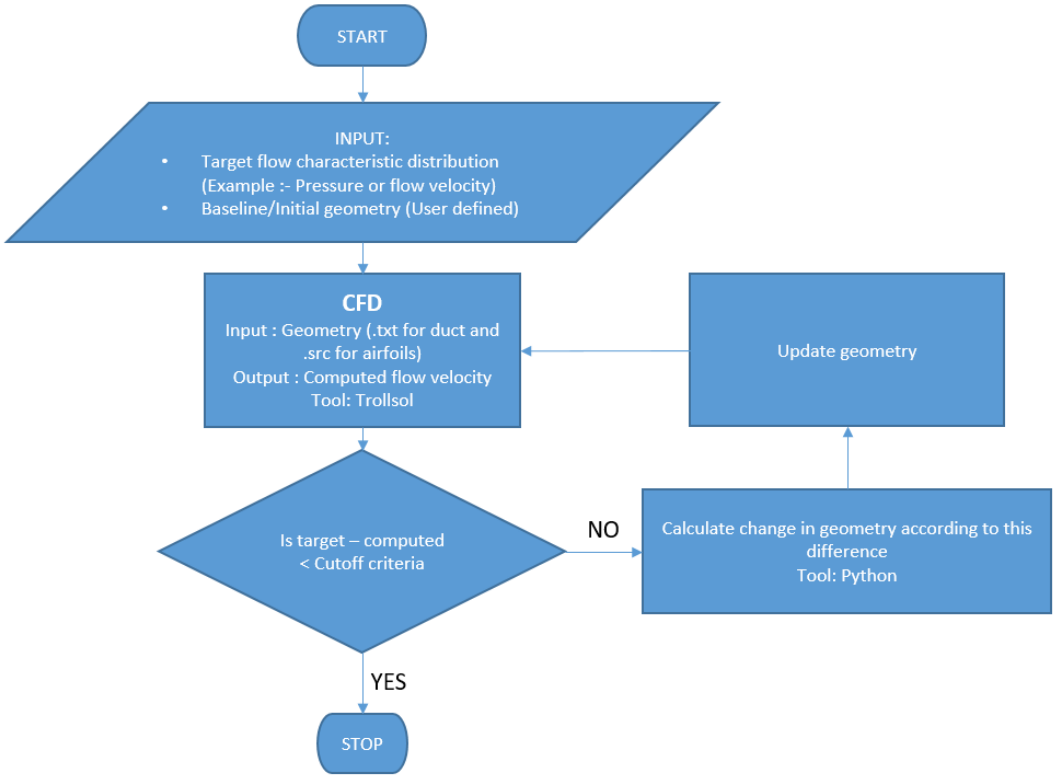


Figure 15: Flowchart representing the inverse-design framework

As seen in the algorithm requires the target flow characteristic distribution, most importantly static pressure, which is enough if there is no separation. Along with that, the inlet conditions and the inlet coordinates of the 2 lines are needed. To account for separation, other velocity distribution quantifiers like shape factor, also need to be included with the pressure distribution.

In case of internal flow, the isentropic flow relation, as described in section 2.4.6, is used to find the outlet area from the static pressures and the inlet area. Using the inlet coordinates and the outlet area, a straight diffuser is created, which is considered as the base geometry.

The inlet boundary condition is already provided by the user, and the outlet static pressure boundary condition is created from the target pressure values at the outlet.

With all these files, the base geometry is first solved, and the difference between the solved pressure distribution and the target pressure distribution is calculated. This translated to the change in geometry in terms of cross stream coordinates at different streamwise locations. For the mathematical relation between the pressure difference and the cross-stream coordinate difference, refer to section 3.3.1.

The new geometry found is again solved, and the whole algorithm is repeated until it reaches a certain cutoff criterion.

3.3.1 Mathematical formulation

$$\beta_0 \frac{\partial r}{\partial t} + \beta_1 \frac{\partial^2 r}{\partial x \partial t} + \beta_2 \frac{\partial^3 r}{\partial x^2 \partial t} = \Delta P$$

Equation 70: Governing equation to calculated geometry modification

The physics based analytical formula relates the change in pressure to the change in the radial (or cross-stream) coordinates. Numerically discretizing using backward difference for slope and central difference for double derivative, leads to Equation 71:

$$\beta_0 \Delta r_i + \beta_1 \frac{\Delta r_i - \Delta r_{i-1}}{x_i - x_{i-1}} + \beta_2 \frac{\frac{\Delta r_{i+1} - \Delta r_i}{x_{i+1} - x_i} - \frac{\Delta r_i - \Delta r_{i-1}}{x_i - x_{i-1}}}{\frac{x_{i+1} - x_{i-1}}{2}} = \Delta P_i \times \Delta t$$

Equation 71: Discretized governing equation

where $\Delta r_i = r_i^{t+1} - r_i^t$, $\Delta P_i = TPD_i - CPD_i$ and Δt is a pseudo time-step.

As there is no upstream coordinate for the inlet point, forward difference is implemented to calculate the 1st derivative instead. The 2nd derivative is assumed 0 for both the inlet and outlet points. Therefore, β_2 is 0 for the extreme points and the first point governing equation is as follows:

$$\beta_0 \Delta r_0 + \beta_1 \frac{\Delta r_1 - \Delta r_0}{x_1 - x_0} = \Delta P_0 \times \Delta t$$

Equation 72: Discretized governing equation for the inlet point (i = 0)

On rearranging the terms in Equation 71,

$$A_i \Delta r_{i-1} + B_i \Delta r_i + C_i \Delta r_{i+1} = \Delta P_i \times \Delta t$$

Equation 73: Re-arranged governing equation

In Equation 73, for i = 1 to n-2 where i = 0 is inlet and n-1 is outlet (similar formulae were used in (Malone, Vadyak, & Sarkar, 1985))

$$A_i = \frac{-\beta_1}{x_i - x_{i-1}} + \frac{2\beta_2}{(x_i - x_{i-1})(x_{i+1} - x_{i-1})}$$

$$B_i = \beta_0 + \frac{\beta_1}{x_{i+1} - x_i} - \frac{2\beta_2}{(x_{i+1} - x_i)(x_i - x_{i-1})}$$

$$C_i = \frac{2\beta_2}{(x_{i+1} - x_i)(x_{i+1} - x_{i-1})}$$

For i = 0 and n-1,

$$B_0 = \beta_0 - \frac{\beta_1}{x_1 - x_0}$$
$$C_0 = \frac{\beta_1}{x_1 - x_0}$$
$$A_{n-1} = \frac{-\beta_1}{x_{n-1} - x_{n-2}}$$
$$B_{n-1} = \beta_0 + \frac{\beta_1}{x_{n-1} - x_{n-2}}$$

Equation 73, being an implicit formulation, needs to be solved using a matrix formulation, as shown in

$$\begin{bmatrix} B_0 & C_0 & & \dots & & 0 & & \dots \\ A_! & B_! & C_1 & & \dots & & 0 & & \dots \\ 0 & A_2 & B_2 & C_2 & & \dots & 0 & & \dots \\ & & & . & & & & & \\ & & & & . & & & & \\ & & & & & . & & & \\ & & & & & & . & & \\ \dots & 0 & & \dots & & A_{n-2} & B_{n-2} & C_{n-2} \\ \dots & 0 & & \dots & & & A_{n-1} & B_{n-1} \end{bmatrix} \begin{bmatrix} \Delta r_0 \\ \Delta r_1 \\ . \\ . \\ . \\ . \\ . \\ \Delta r_{n-2} \\ \Delta r_{n-1} \end{bmatrix} = \begin{bmatrix} \Delta P_0 \\ \Delta P_1 \\ . \\ . \\ . \\ . \\ . \\ \Delta P_{n-2} \\ \Delta P_{n-1} \end{bmatrix} \times \Delta t$$

Equation 74: Matrix formulation of the governing equation

B_0 is set to a very high number compared to the other coefficient values, this converts the equation at i= 0 to $\Delta r_0 = 0$. This constraint is necessary, otherwise the algorithm might diverge leading to the lines moving apart or closing in with an unnecessarily high *Δr.*

This formulation was followed by testing of the algorithm, and it was found that the outlet area also had to be constrained to the required value. If the outlet points moves apart, keeping the same inlet conditions, the outlet static pressure should rise, and if it rises too much, the algorithm detecting very high outlet pressure, will try to bring the lines close at the outlet. Unfortunately, a typical CFD algorithm imposes the outlet static pressure strictly and adjust the inlet static pressure accordingly. Hence, if the lines move apart by an amount more than it should, the outlet pressure condition will still remain the same, hence the same outlet static pressure. Instead, the inlet static pressure will be lowered down. Eventually, the algorithm never recovers from such a scenario and diverges.

To introduce a constant outlet area constraint, one more equation was added and 2 original equations were combined to keep the total number of equations the same.

$$\Delta\left(\pi * r_{n-1,shroud}^2 - \pi * r_{n-1,hub}^2\right) = 0$$

Assuming Δr<<r, and on linearizing, the equation modifies to the following,

$$r_{n-1,shroud}\Delta r_{n-1,shroud} - r_{n-1,hub}\Delta r_{n-1,hub} = 0$$

Equation 75: Outlet area constraint

The operator in between the terms in Equation 75: Outlet area constraint is change to negative in Equation 76, as positive change in hub line means reduction in radial coordinate, on contrary to the same for the shroud line.

$$r_{n-1,shroud}\Delta r_{n-1,shroud} + r_{n-1,hub}\Delta r_{n-1,hub} = 0$$

Equation 76: Outlet area constraint corrected

The two original equations combined were that of the last points of both the hub and shroud line.

$$A_{n-1,h}\Delta r_{n-2,h} + B_{n-1,h}\Delta r_{n-1,h} - A_{n-2,s}\Delta r_{n-1,s} - B_{n-1,s}\Delta r_{n-1,s} = (\Delta P_{n-1,h} - \Delta P_{n-1,s}) \times \Delta t$$

Equation 77: Combined outlet governing equation

As the hub and shroud outlet position is related, the coefficient matrices of both the hub and shroud line has to be combined into one and solved for simultaneously.
Hence,

$$[C][\Delta r] = [\Delta P] \times \Delta t \quad \text{, where}$$

$$[C] = \begin{bmatrix} B_{0,h} & C_{0,h} & \dots & 0 & \dots & \dots & 0 & \dots \\ A_{1,h} & B_{1,h} & C_{1,h} & \dots & 0 & \dots & \dots & 0 & \dots \\ 0 & . & . & . & \dots & 0 & \dots & \dots & . & \dots \\ \dots 0 \dots & . & . & . & \dots & 0 & \dots & \dots & . & \dots \\ \dots 0 \dots & . & . & . & \dots 0 & \dots & \dots & . & \dots \\ \dots 0 \dots & A_{n-2,h} & B_{n-2,h} & C_{n-2,h} & \dots & 0 & \dots \\ \dots 0 \dots & A_{n-1,h} & B_{n-1,h} & \dots & 0 & \dots & -A_{n-1,s} & -B_{n-1,s} \\ \dots & 0 & \dots & B_{0,s} & C_{0,s} & \dots & 0 & \dots \\ \dots & 0 & \dots & A_{1,s} & B_{1,s} & C_{1,s} & \dots & 0 & \dots \\ \dots & . & \dots & 0 & . & . & . & \dots & 0 & \dots \\ \dots & . & \dots & \dots 0 \dots & . & . & . & \dots 0 & \dots \\ \dots & . & \dots & \dots 0 \dots & . & . & . & \dots 0 & \dots \\ \dots & 0 & \dots & \dots 0 & \dots & A_{n-2,s} & B_{n-2,s} & C_{n-2,s} \\ \dots & 0 & \dots & r_{n-1,h} & \dots & 0 & \dots & r_{n-1,s} \end{bmatrix}$$

$$[\Delta r] = \begin{bmatrix} \Delta r_{0,h} \\ \Delta r_1, h \\ . \\ . \\ . \\ \Delta r_{n-2,h} \\ \Delta r_{n-1}, h \\ \Delta r_{0,s} \\ \Delta r_1, s \\ . \\ . \\ . \\ \Delta r_{n-2,s} \\ \Delta r_{n-1}, s \end{bmatrix}, \quad [\Delta P] = \begin{bmatrix} \Delta P_{0,h} \\ \Delta P_{1,h} \\ . \\ . \\ . \\ \Delta P_{n-2,h} \\ \Delta P_{n-1,h} - \Delta P_{n-1,s} \\ \Delta P_{0,s} \\ \Delta P_{1,s} \\ . \\ . \\ . \\ \Delta P_{n-2,s} \\ 0 \end{bmatrix}$$

Equation 78: Final matrix formulation with coefficients

Equation 78 contains the final matrix formulation with all it contents. It seems to have 2 sub matrices in it, where the top left matrix resembles the coefficients related to the hub and the other one, shroud. In the last row, n-1th column (0 being first) is $r_{n-1,h}$ and the 2*n-1th column is $r_{n-1,s}$. That row is for constraining the outlet area change to approximately 0.

3.3.1.1 *Coefficients and multipliers*

The key concept behind Equation 70 is that if the pressure needs to be increased at a certain location on the hub, the wall parallel velocity magnitude has to be lowered, hence the need for a higher flow cross-section area. The algorithm will hence try to bring wall down. This means that the slope, using a backward difference, should also increase but the curvature should increase in the other direction, i.e., decrease. Therefore, β_0 and β_1 should be positive and β_2 negative.

Δt, i.e., the pseudo-time should be very low, to keep the Δr at any iteration not more than a fraction of the radial position.

Some standards values are:

$\beta_0 = 0.1,\ \beta_1 = 0.04 \times \beta_0 \times \Delta x,\ \beta_2 = -100 \times \beta_0 \times \Delta x^2,\ \Delta t = 3 \times 10^{-5}$ and $URF = 0.7$

Using all the above-mentioned equations, the change in the cross-stream coordinate can be calculated by:

$$[\Delta r] = [C]^{-1}[\Delta P] \times \Delta t \times URF$$

Equation 79: Matrix inversion

3.3.2 Inlet slope correction

As noticed in the previous sections, there is no mention of an equation constraining the wall slope at inlet to 0. To preserve the slope continuity at all the inner points (*i= 0 to n-2*), the equation at those points needs to be preserved. One cannot combine 2 of those equations to a single equation, like it has been done in Equation 76, to get room for another constraint equation. This gives excessive freedom and leads to a slope discontinuity at those points. Hence, the only way is to do the slope correction after the matrix inversion has been done.

As the walls curvature is very important to the flow properties, it must be preserved through the correction. Only way is to subtract the tangent at the inlet from the entire curve. This ensures that the slope at the inlet is 0, the inlet positions and the curvature are not affected.

$$r_{i,corrected} = r_i - \left\{ r_0 + (x_i - x_0)\frac{r_1 - r_0}{x_1 - x_0} \right\}$$

Equation 80: Slope correction

Equation 80 corrects the slope but also alters the outlet area, but not enough such that the line at the outlet diverges. This is another reason that the Δt is small, so that the inlet slope at every step before the correction is also very small, leading to a smaller change in the outlet area.

3.3.3 CFD solver

An in-house mesh generating tool and CFD solver have been implemented in the algorithm, namely, *Volmop* and *Trollsol_0.5.1*. *Volmop* consists of standard modules for different types of geometries. Axisymmetric duct is one of the options, which required the hub and shroud line as the radius vs cross-stream coordinate, along with the angle of rotation between the periodic walls. Additionally, *Volmop* required details about the cell spacing. This generated a *Trollsol* compatible grid file.

Trollsol requires the standard metrics and coefficients as other commercial software, along with the boundary conditions, type of turbulence model to use, CFL number and the type of initialization. In models, it has the option for realizable k-ϵ (refer to section 2.7.4 for details) along with standard wall-functions.

The most important post-processing step is to generate files with the pressure distribution along certain walls, averaged in a certain direction. Such a function was added by the author to the existing script of the solver, named *PAVE-CYL*.

4 Results

4.1 Grid independence test and validation of *trollsol_0.5.1*

Table 1 shows the grid independence study for the in-house solver, trollsol_0.5.1, with the first case geometry, discussed further. The 3rd and the 4th column shows the number of cell nodes in the axial and radial direction of the mesh, respectively.

The percentage error for the ΔP/Avg P_{in} is calculated with respect to the next finer mesh. Note that ΔP is the difference between the mean outlet and the mean inlet static pressure.

As the relative error for case 4 is under 0.5%, this mesh has been used for further study.

Case	first wall thickness [m]	Cell no. axial	Cell no. radial	Avg P_{in} [Pa]	Avg P_{out} [Pa]	$\frac{\Delta P}{Avg\ P_{in}}$	% error
1	0.001	25	40	95384.7	98120.3	0.02868	1.096
2	0.001	30	50	95398.85	98104.45	0.02836	1.918
3	0.001	40	65	95429.05	98082.75	0.02781	1.082
4	0.001	50	80	95426.04	98008.25	0.02751	0.421
5	0.001	60	90	95450	98063.95	0.02739	0.116
6	0.0008	70	100	95446.79	98057.7	0.02735	

Table 1 : Grid independence study of trollsol_0.5.1 *solver*

For the first case, a sample hub and shroud line was used to generate the target pressure profiles, which can be seen in Figure 20: (a) Absolute static pressure comparison between Target and Computed geometries (b) Pressure residual in case.

Inlet and outlet boundary condition files were created, taking the static pressure values from the target pressure profiles at inlet and outlet into account.

The exact same conditions of inlet total pressure, static temperature, static pressure outlet, solver model and other settings were used in both Ansys Fluent and *trollsol_0.5.1* to validate it. Comparison between pressure profiles can be seen in the following Figure 16.

The error, calculated using the formula in Equation 82: Pressure residual formula, is 2.89%.

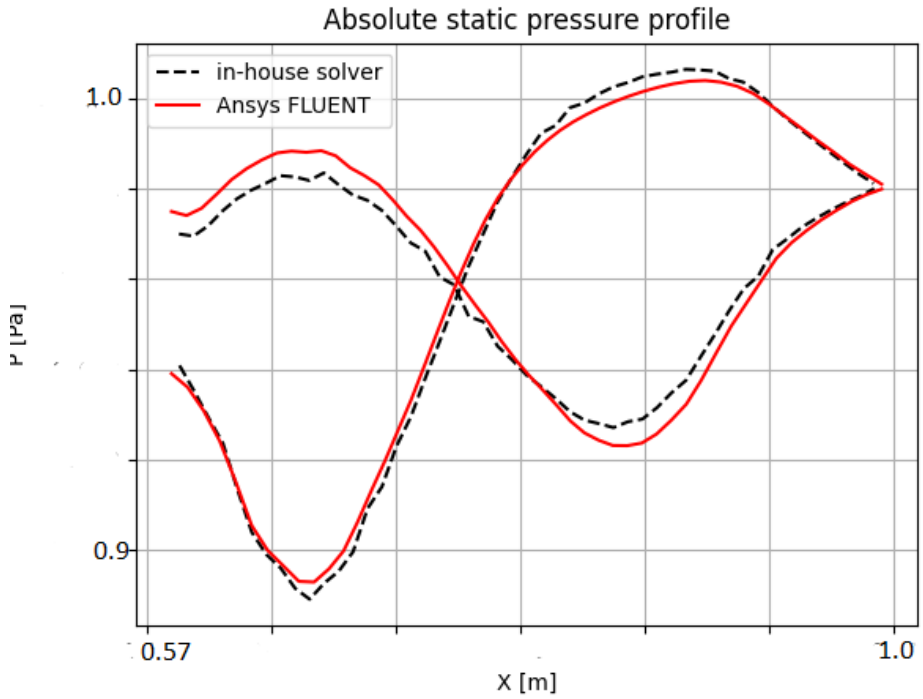


Figure 16: Comparison between the pressure profiles solved by the in-house solver (Trollsol) and Ansys FLUENT

4.2 2D inverse design application for internal flows

Figure 17 depicts an unsuccessful result without an outlet area constraint. The solution, initially, was converging but eventually the hub line was pushed down more than it should have been done. This led to separation and the MGM algorithm was not applicable to separating cases, leading to divergence of solution.

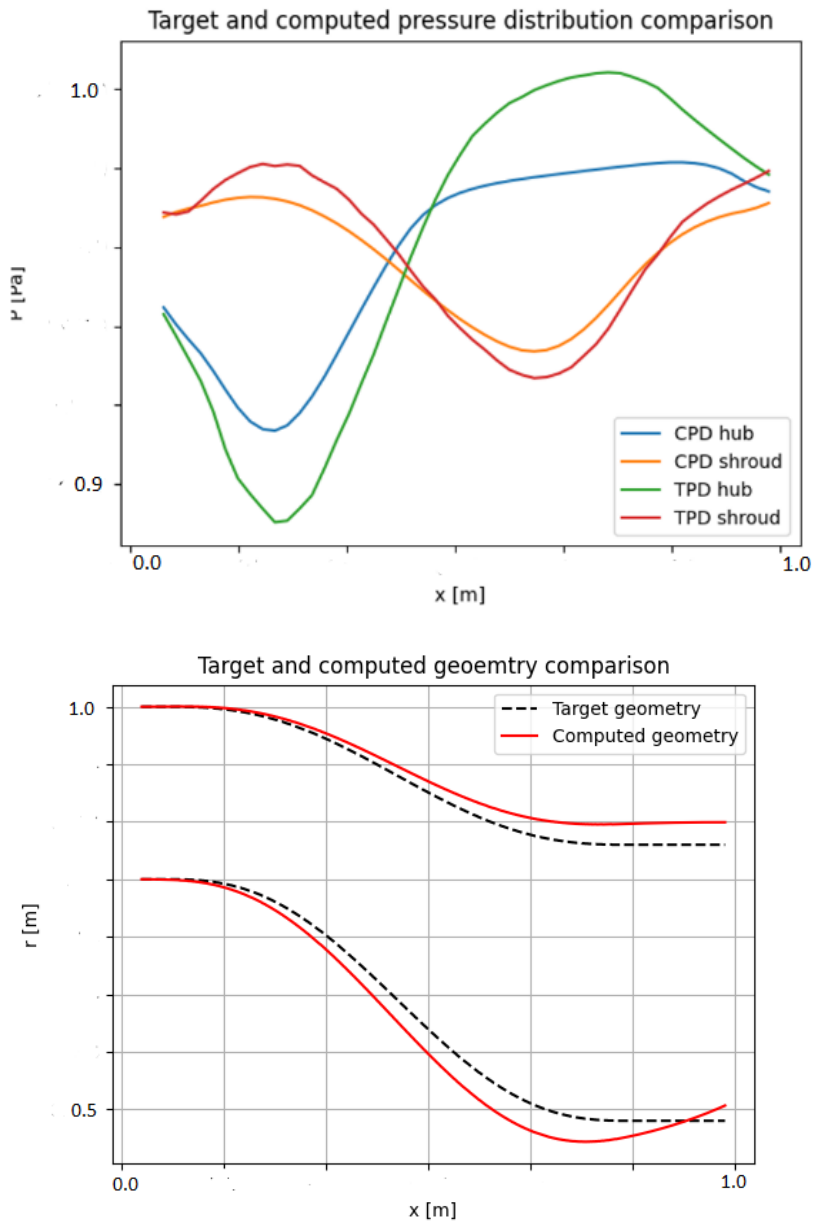


Figure 17: (a) Pressure profile and (b) generated geometry using IDA without any outlet area constraint

The inverse design algorithm, as discussed in section 3.3, has been implemented on a pressure profile along the hub and shroud line of an inter-compressor duct (ICD). The following URF has been used along with the pseudo-time stepping and the 3 β coefficients:

$$URF = 0.3, \qquad \Delta t = 3 \times 10 - 6\, sec, \qquad \Delta r_{clipping} = 10^{-3}$$

$$\beta_0 = 1, \qquad \beta 1 = 0.04 \times \beta_0 \times \Delta x = 2.3 \times 10^{-4}, \qquad \beta_2 = -0.05$$

Equation 81: Coefficients for case 1

β_2 being negative is of the utmost importance as already explained in section 3.3.1.1.

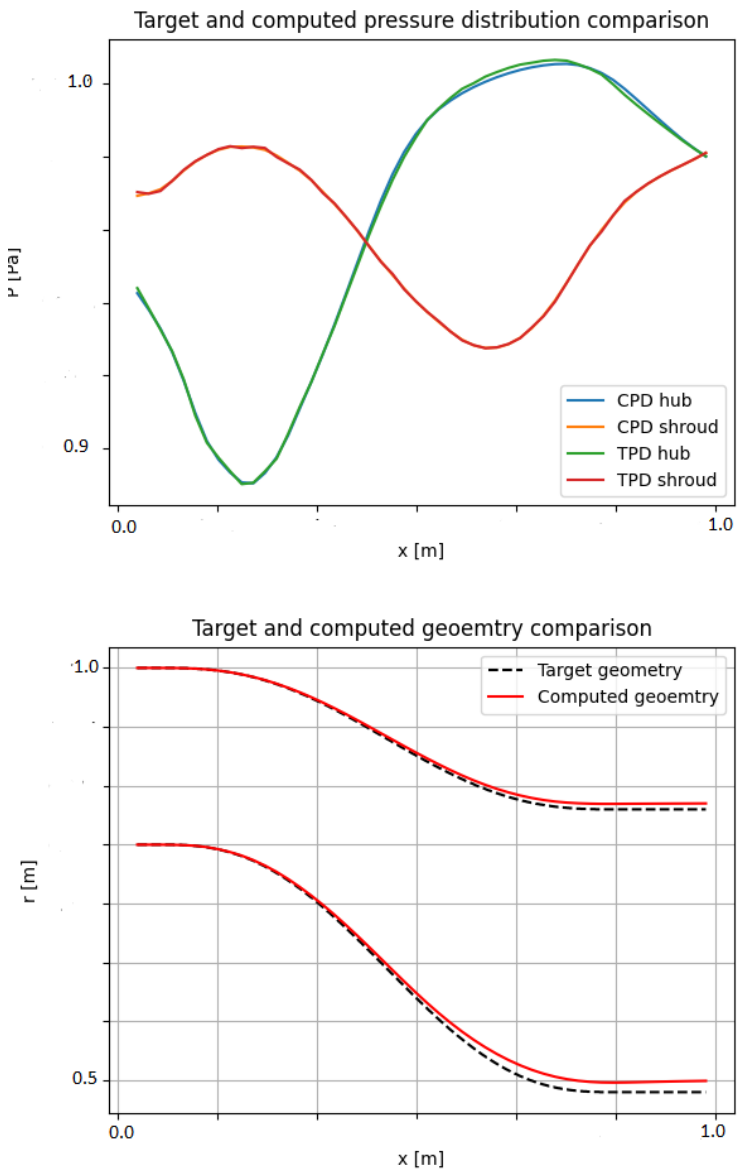


Figure 18: (a) Absolute static pressure comparison between Target and Computed geometries (b) Geometry comparison for the same case

Figure 18 shows an excellent agreement between the case 1 target and computed geometries and profiles, achieved with an URF of 0.3, executed till 100 iterations.

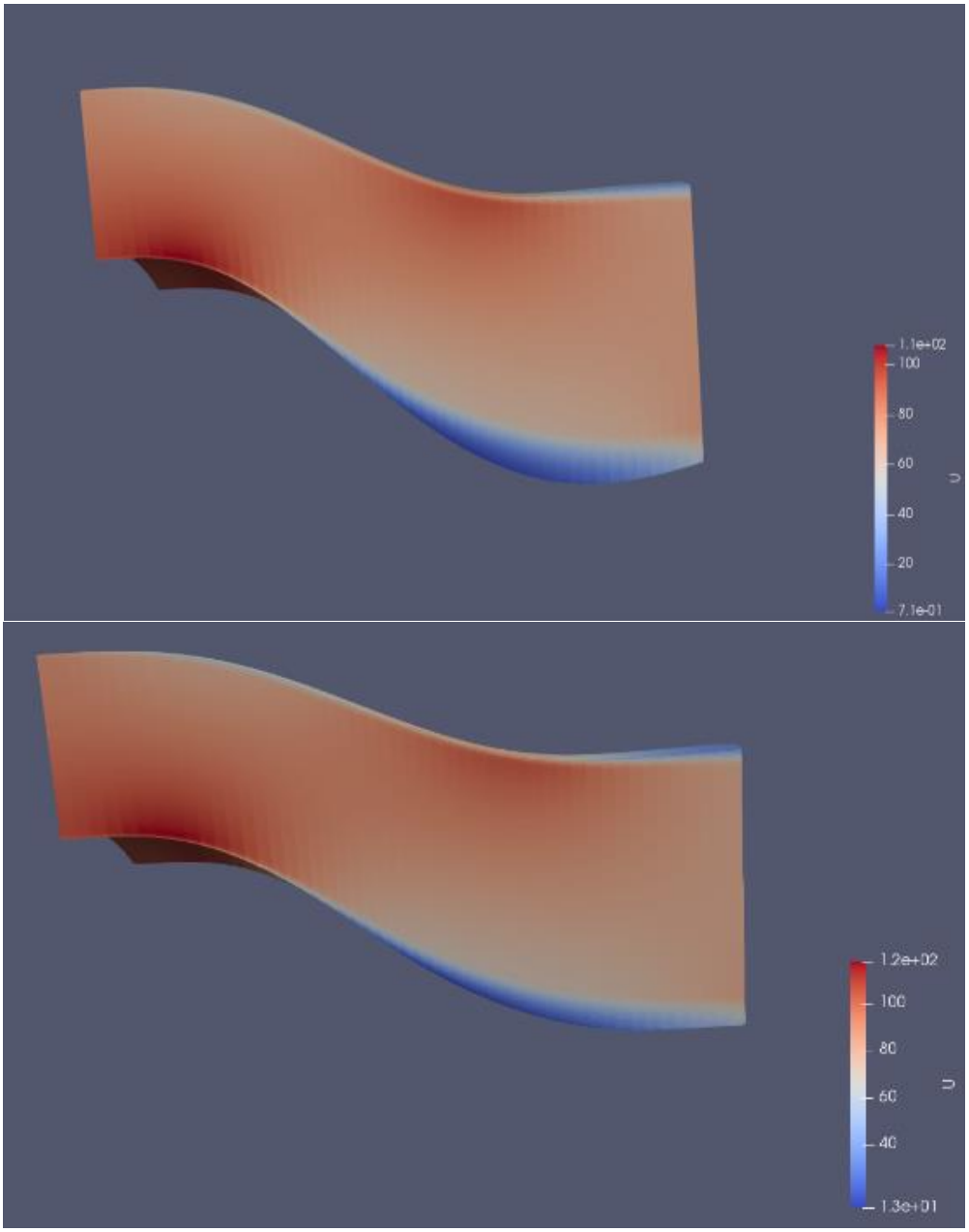


Figure 19 Axial velocity contour for the IDA generated geometries (a) without and (b) with the outlet area constraint

Interestingly, if the algorithm keeps executing for a much longer duration, the residual curve, as shown in Figure 20(b), goes off-trend after the 154th iteration. Residual values start to steadily increase again. This might be caused by separation, as until this case, the mathematical formulation still has not been taken it into account. Hence, as the algorithm keeps on executing and reaches a geometry with separation, it never recovers and the residual starts to diverge.

The existence of separation can also be proved by the subtle flattening of the hub line pressure profile at its maxima in Figure 20(a)

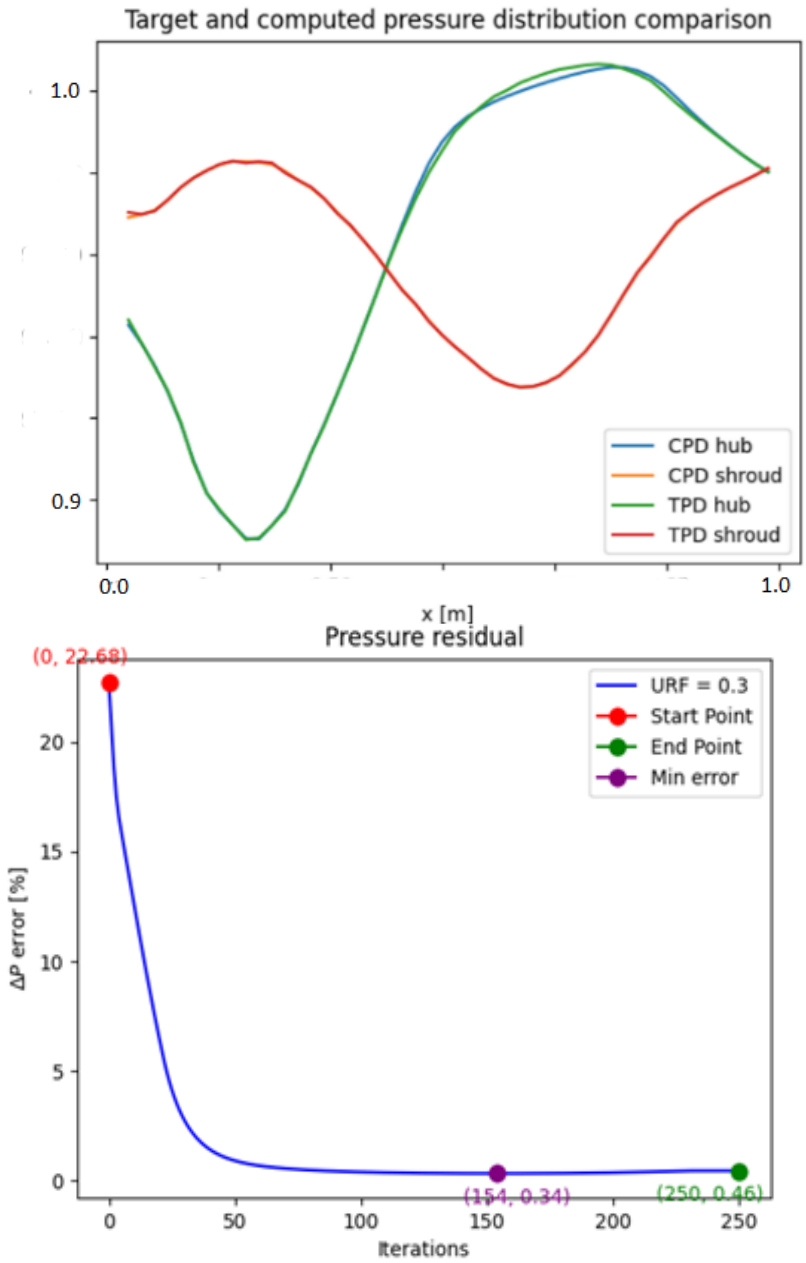


Figure 20: (a) Absolute static pressure comparison between Target and Computed geometries (b) Pressure residual in case 1

Please note that error in the pressure field is calculated using the following formula:

$$\Delta P\ error = \frac{\sqrt{(TPD - CPD)^2}}{TPD_{max} - TPD_{min}} \times 100\%$$

Equation 82: Pressure residual formula

The above results show that despite having a target profile, where flow does not separate, the intermediate iteration computed geometries can lead to separation. These findings led to a stability study, which led to

subtle alterations in the algorithm. The in-house solver takes about 100 to 200 seconds for 3500 iterations of CFD solver, hence the stability analysis could also lead to make the algorithm much faster by reducing the number of iterations of the IDA needed.

4.2.1 Additional algorithm validation cases

Two different turbulence models were employed in the boundary layer code (see section 3.2) to generate a simple duct, i.e. not much aggressiveness. As a result two different pressure profiles were generated by the optimizer. These profiles were further used to validate the robustness of the inverse solver. As shown in Figure 21 the inverse design algorithm converges with significantly different pressure profiles as well.

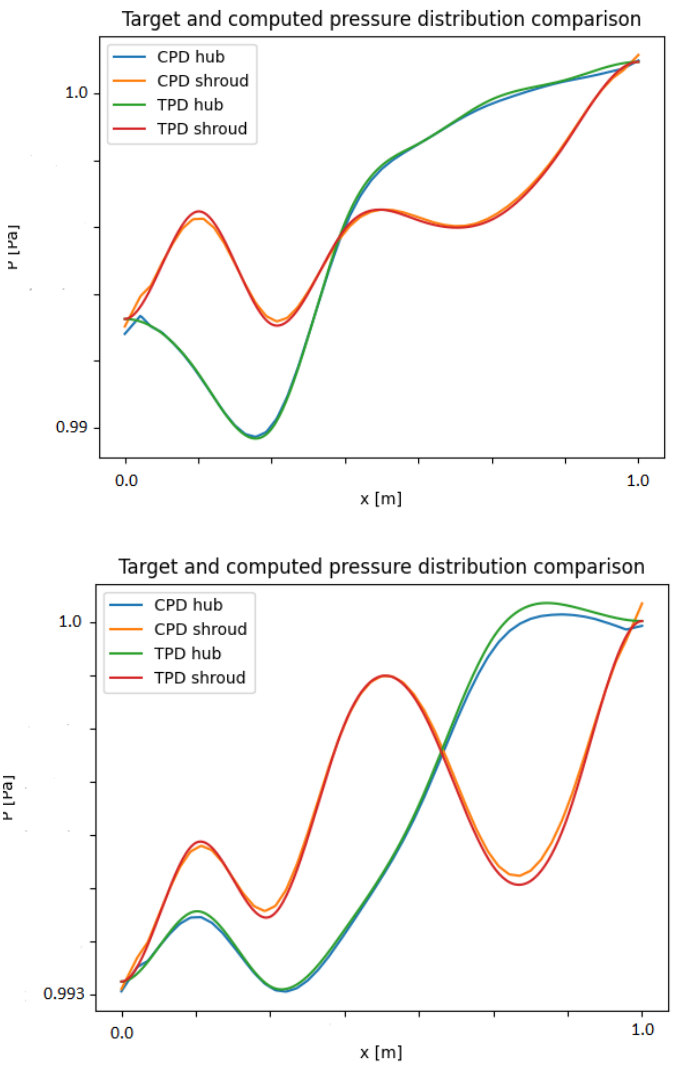


Figure 21: TPD and CPD comparison for TPD optimized using (a) k-ε (b) k-ω as the turbulence model in the boundary layer code

Figure 22 shows the geometry received from the inverse design algorithm with the two TPDs mentioned above as an input, using the same inlet conditions and inlet hub and shroud radii.

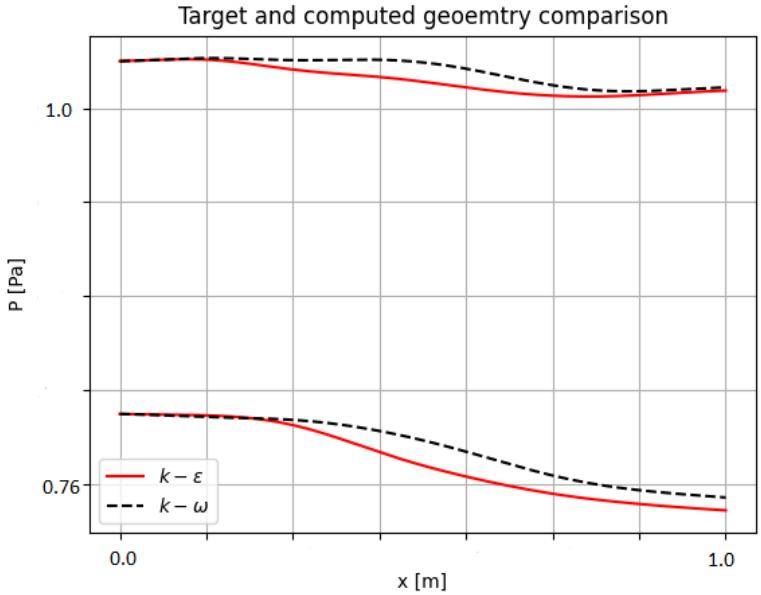


Figure 22: Geometry corresponding to the optimized TPDs using k-ε *and* k-ω

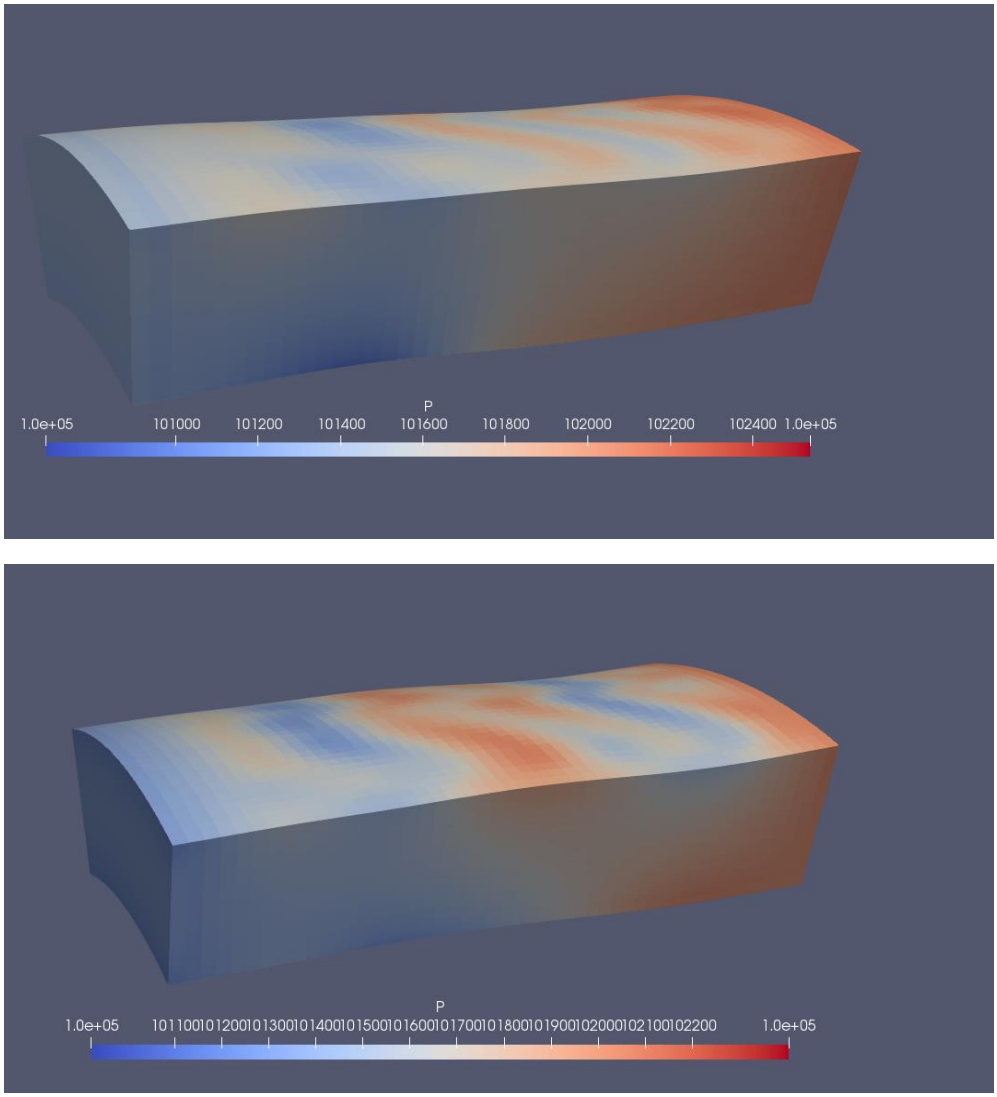


Figure 23: Pressure contour of the geometry corresponding to (a) Figure 21(a) and (b) Figure 21(b)

Figure 23 reveals an issue with the in-house solver used in the inverse design algorithm. For geometries with very low aggressiveness, the solver cannot converge properly, adding spatial fluctuations (in pressure, velocities, etc) about the real solution. To avoid this error, only the pressure profile at the periodic boundaries is used instead of averaging through the circumference.

Additionally, the solver behaves in an unprecedented manner at the first near wall cell, closest to the inlet. Observing the pressure profiles in Figure 21 closely, the inlet pressure values are clearly off-trend. This problem has been avoided by fixing the inlet position, essentially ignoring the effect of the hub and shroud pressure difference at the inlet.

4.2.2 Geometry generation issues detected and potential solutions

As shown in Figure 22, the aggressiveness is much lower compared to that one in Figure 18. Therefore, similar TPDs were generated by constraining the trapped area to a much higher value. The pressure profiles were further parsed into the IDA to generate a new and more aggressive geometry, as shown in Figure 24. According to results from the in-house solver used in the IDA, the radial momentum flux through the outlet was 11.35 N whereas the axial momentum flux was 1612 N. Two important inferences are drawn from this observation:

1. As the contribution to the radial momentum by the shear force was not considered in the trapped area calculation, it led to net addition to the radial momentum from inlet to outlet.
2. The mean TPD of hub and shroud line was very steep till the outlet, with a very sharp curve, leading to a horizontal nature at the outlet. This is essential to ensure that the hub and shroud line ends horizontally. In this case, the mean TPD slope reduced to 0 in a very short axial region, which was less than the cell size used in the IDA. Therefore, the IDA, as can be seen from the same figure, led to a CPD, which did not end horizontally. This led to a geometry where the outlet had a hugely varying radial velocity distribution. This is not an ideally accepted outlet velocity profile for an ICD.

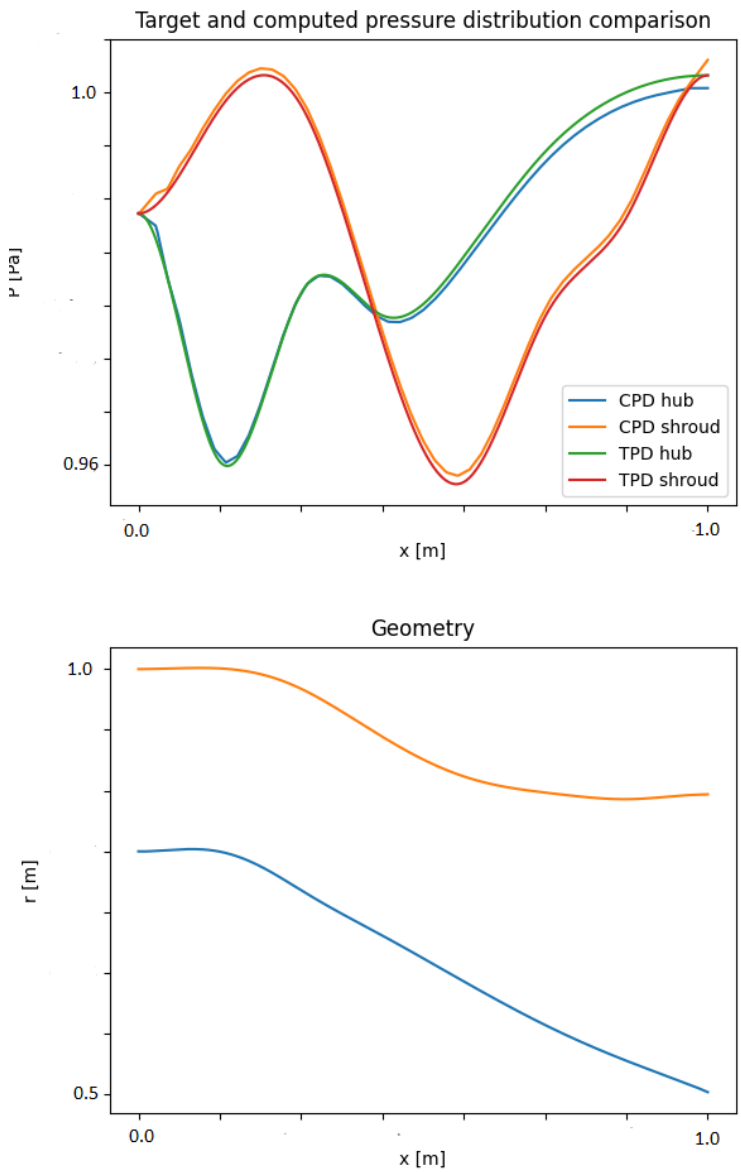


Figure 24: Generated (a) TPD, including CPD and (b) Geometry using k-omega in the BL code

Similarly, a TPD was generated but by using k-ε instead of k-ω as the turbulence model in the boundary layer code, used in the TPD optimizer as shown in Figure 25. Unfortunately, due to the aforementioned reasons, the IDA tried to abruptly change the slopes of hub and shroud line at the end, in a span of 1 cell length. Therefore, due to an unphysical geometry, the CFD solver could never converge. The issue could have been solved by using a much denser mesh, but this would have increased the computation time of the algorithm many-folds.

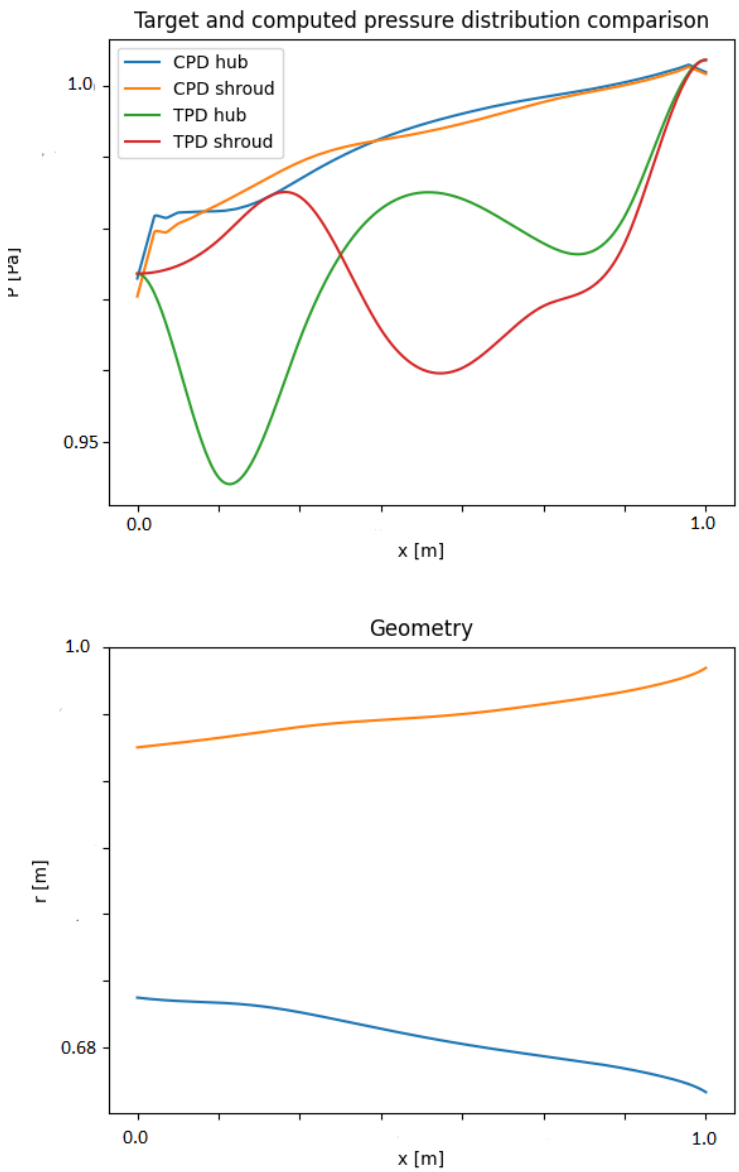


Figure 25: Generated (a) TPD, including CPD and (b) Geometry using k-epsilon in the BL code

Alternatively, the issue can be fixed by increasing the area close to the inlet and outlet, where the slope of the pressure curves has to be zero, allowing the duct turn to become horizontal.

Overall, three different regimes of the MGM applicability were detected based on the nature of the TPD pressure at the outlet:

1. Had a physically possible change in slope, and the change was over a distance of at least few axial cell lengths of the CFD employed in the IDA: Ensured convergence as well correct mimicking of the outlet pressure profile slope

2. Had a slightly unphysical change in slope at the outlet but it was not too unphysical: The IDA algorithm could converge, but the axial cell length was not small enough to capture the change in slope correctly.
3. Had a very unphysical change in slope at the outlet: The IDA could not even converge as the geometry, trying to follow the pressure profile influence, suddenly change its slope from the second last to the last cell.

The potential solution to this issue was to use a mesh being much denser along the axis, near the outlet. However, this would have led to a geometry with a sharp change in slope, compromising the structural integrity. Therefore, the better solution was to change the TPD instead.

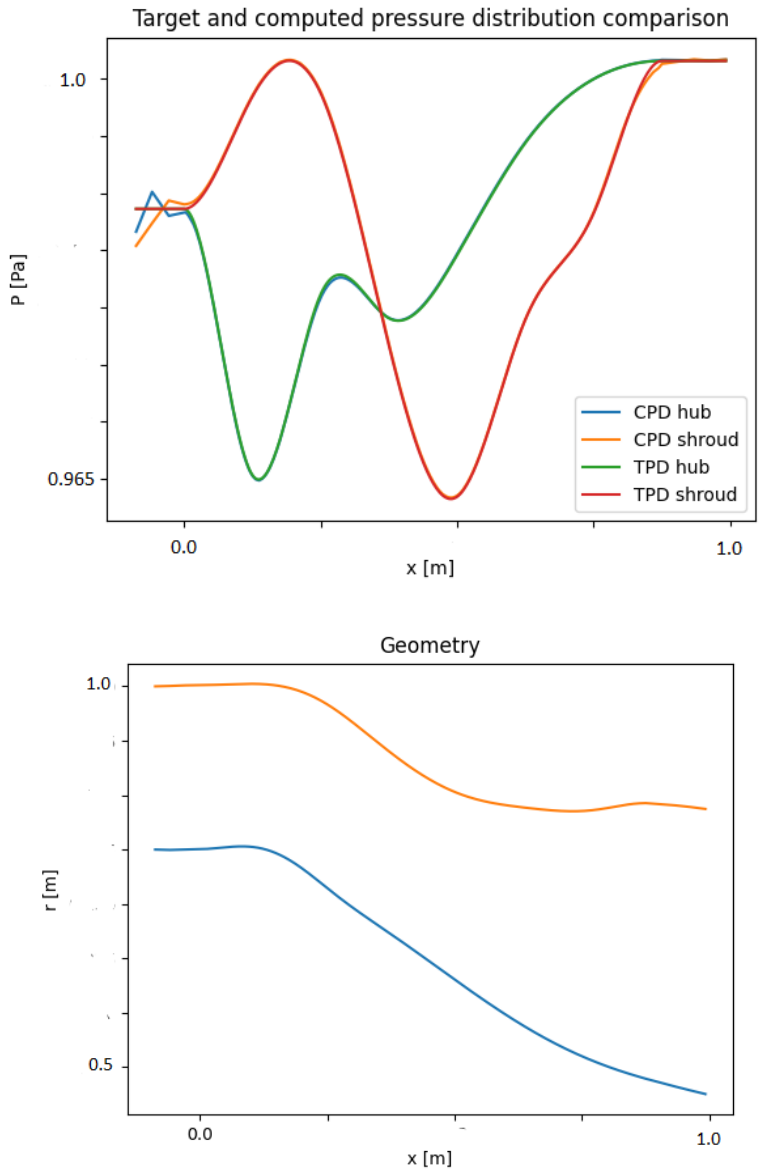


Figure 26: (a) TPD achieved using the k-omega curve by manually added pressure points (b) Generated duct geometry from the mentioned TPD

Figure 26 shows a TPD, where pressure points have been added manually to the inlet and outlet respectively. This was done to increase the axial length over which the slope of the TPD gradually changed. A small change like this help capturing the zero slope TPD at the outlet correctly. The outlet might look off-horizontal but the radial momentum flux was negligible at the outlet, compared to the axial momentum flux.

4.3 Geometries generated for different levels of aggressiveness

Having established the loop between the TPD and corresponding geometry generation, three different test cases are setup. These vary in their aggressiveness, which is being adjusted by altering the initial trapped area between the hub and shroud pressure distributions. Furthermore, these scenarios are modelled using the k-ϵ and the k-ω turbulence models.

4.3.1 Low Aggressive duct design

4.3.1.1 *Standard k-ω*

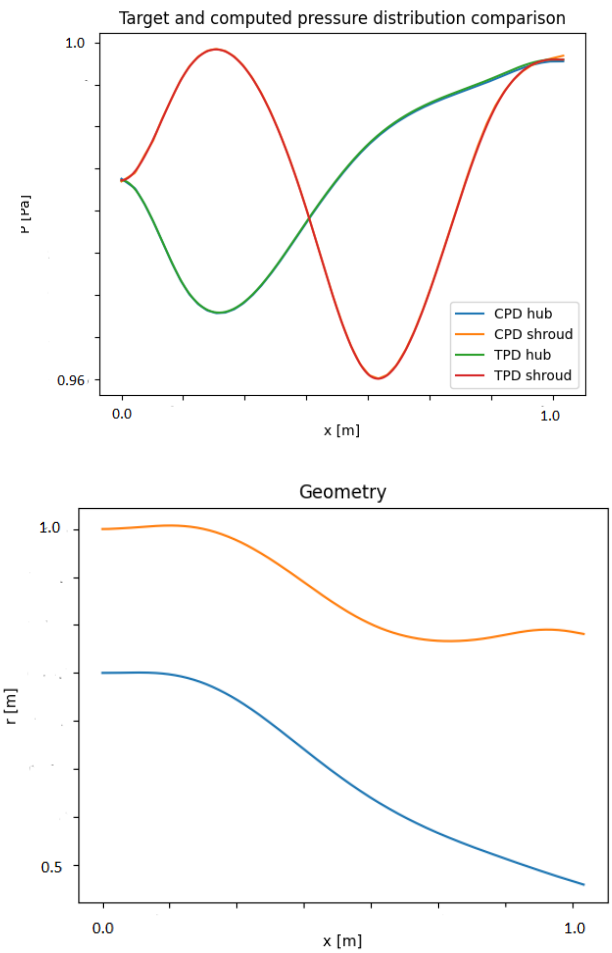


Figure 27: Standard k-ω model, low aggressiveness

Figure 27 shows the TPD as well as the resulting geometry for a low aggressiveness duct. In this case, a “trapped area” of 280 led to a Δr of 0.082534 m. The pressure recovery in this case is 99700 Pa.

4.3.1.2 *Standard k-ε*

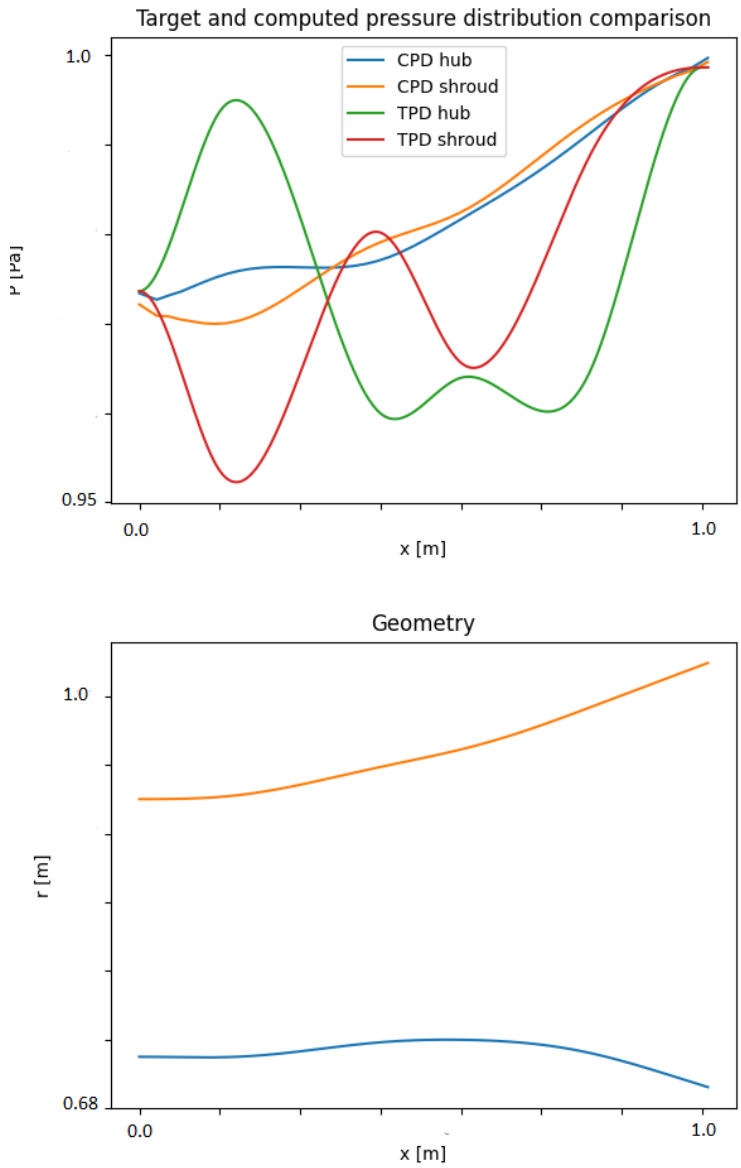


Figure 28: Standard k-ε model, low aggressiveness

As seen from Figure 28, the nature of the TPD was far off a standard pressure curve. This made it particularly difficult for the CFD solver to handle a geometry with unusual curvatures along with high area ratio. For the Standard k-ϵ model, a low aggressiveness duct, with a “trapped area” of 280, led to a pressure recovery of 100860 Pa.

4.3.2 Medium aggressive duct design

4.3.2.1 *Standard k-ω*

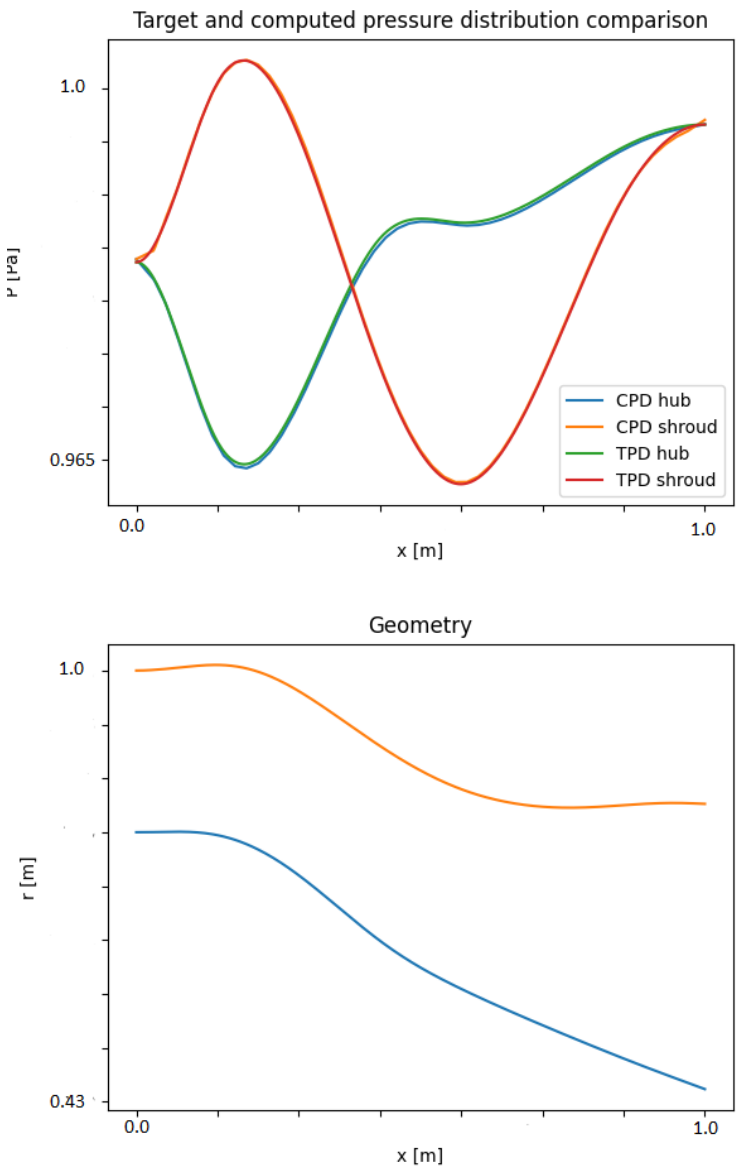


Figure 29: Standard k-ω model, medium aggressiveness

Figure 29 shows the TPD as well as the generated geometry for a medium aggressive duct. Here, a “trapped area” of 380 led to a Δr of 0.090456 m and to a pressure recovery of 99700 Pa.

4.3.2.2 *Standard k-ε*

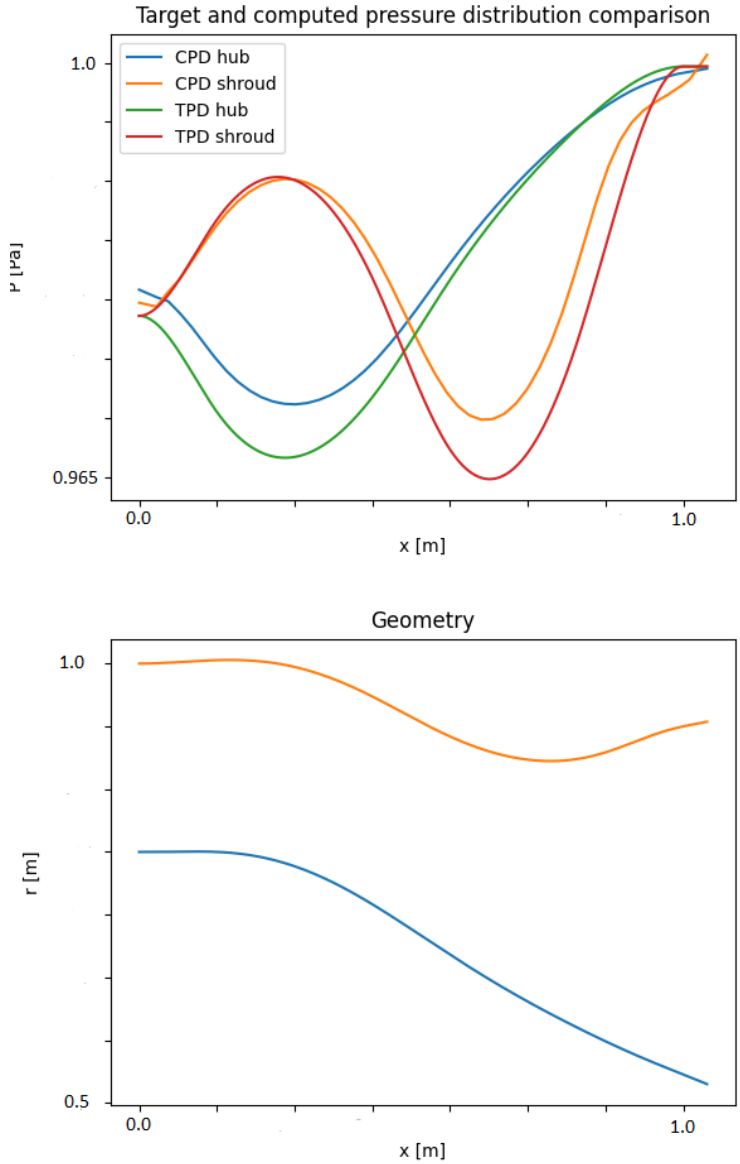


Figure 30: Standard k-ε model, medium aggressiveness

As shown in Figure 30: Standard k-ε model, medium the pressure distribution could not be completely re-created, as the TPD generated by the optimizer shows an extremely high pressure recovery of 100750 Pa, while having a trapped area of 380. The flat nature of the CPD on the shroud end wall near the outlet suggests the presence of separation. Eventually the CFD solver could not converge, leading to an incomplete solution.

4.3.3 Highly aggressive Duct Design

4.3.3.1 *Standard k-ω*

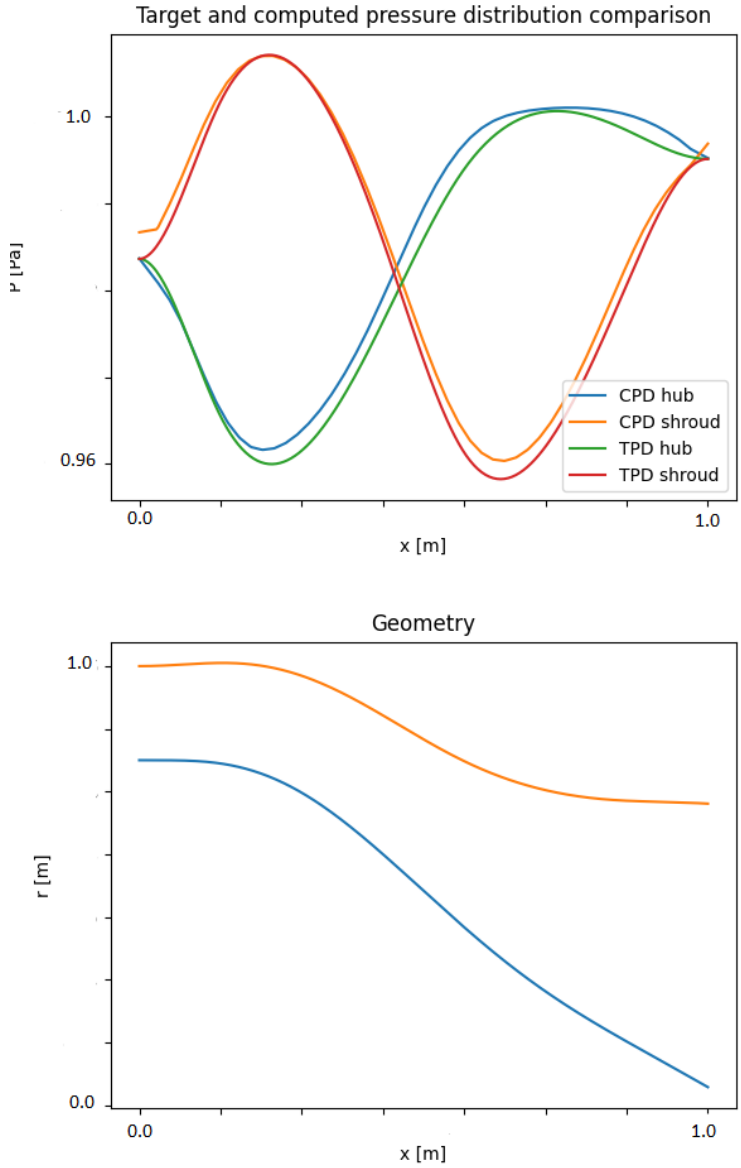


Figure 31: Standard k-ω model, high aggressiveness

Figure 31 illustrates the TPD as well as the generated geometry for a highly aggressive duct. A “trapped area” of 445 hereby led to a Δr of 0.090456 m and to a pressure recovery of 99500 Pa. It should also be noted that a mass flow rate of 14.52 kg/s was achieved instead of 15 kg/s. A lower mass flow than the value upon which the TPD was designed, could be possible reason for separation at the hub wall, near the outlet. This observation is supported by both, Figure 31 and Figure 32.

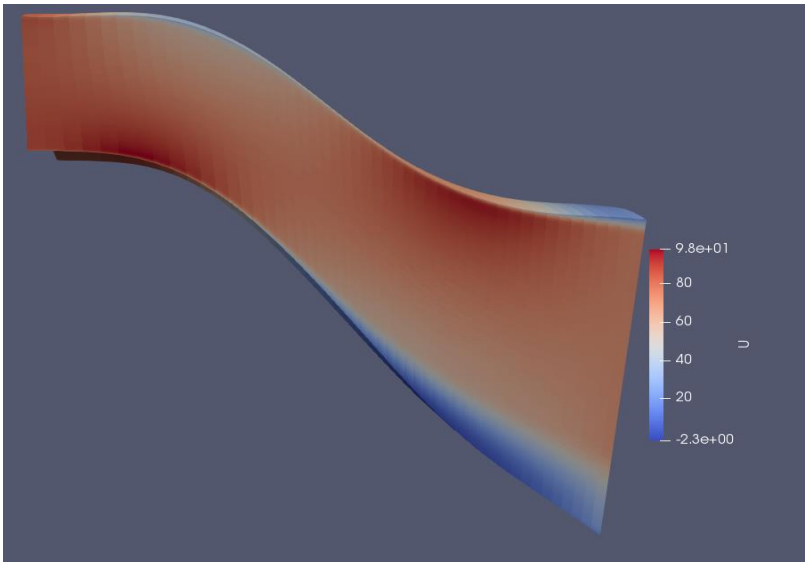


Figure 32: Axial-velocity contour of the geometry achieved by using a k-ω model with high aggressiveness

4.3.3.2 *Standard k-ε*

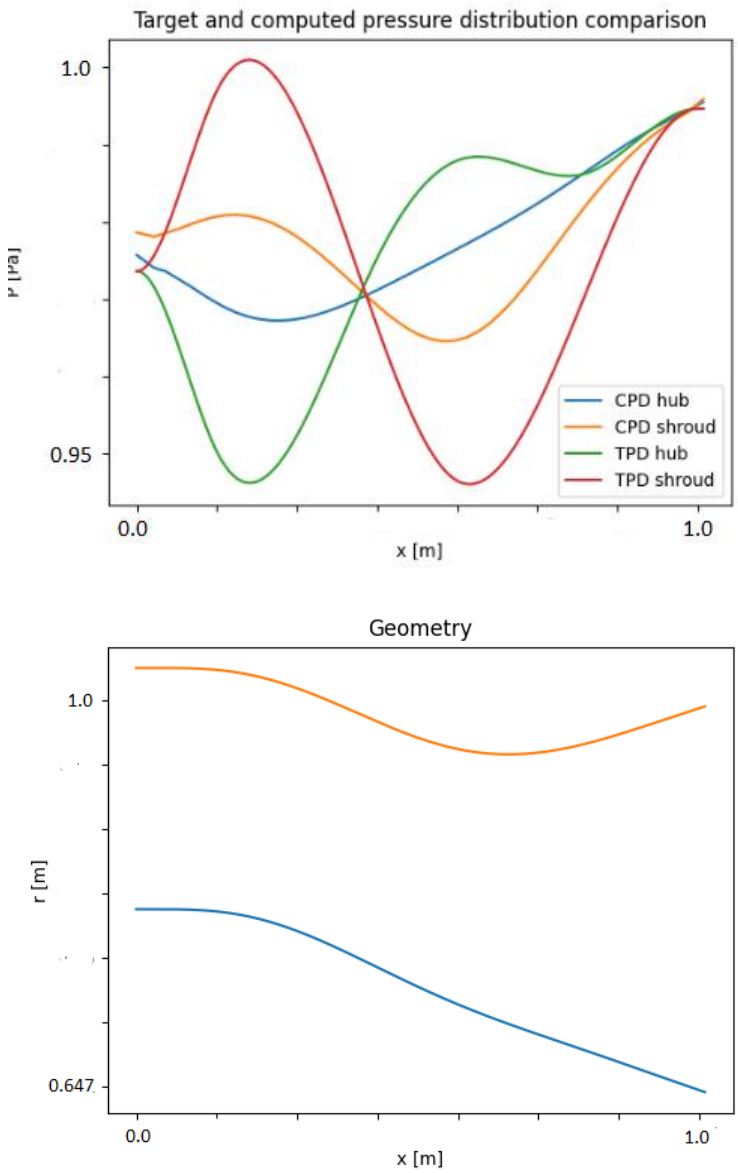


Figure 33: Standard k-ε model, high aggressiveness

The reason for the high difference between the CPD and the TPD shown in Figure 33 was the same as mentioned for the medium aggressiveness k-ε TPD, as it shows a high pressure recovery of 100400Pa with a “trapped area” of 445.

4.3.4 Profile generation with axial outflow

As has been established in the previous sections, a strict area equality constraint does not yield in an axial outflow, because it does not consider any viscous forces. To account for viscous forces the trapped area two (area behind the inflection point) needs to be larger than the first area (section 3.1.3). This results in the following modified area constraint:

$$Area_2 = Area_1 + \int_0^{\Delta H_{up}} \tau_{up}\, dy + \int_0^{\Delta H_{low}} \tau_{low}\, dy$$

, where τ_w can be extracted from the results of the EddyBl program.

However, even including the viscous forces didn't yield in an axial outflow at the duct outlet, as their influence is low in comparison to the pressure existing pressure gradient.

To still generate a TPD including an axial outflow and thus model a feasible s-shaped duct, the "trapped areas" where preset, with area1 being 340, and area2 460. This difference in area, not being the result of neglecting viscous forces, points out a significant change required in the equation for ensuring an axial outlet flow.

Observing the TPD, one can recognize that as the second area increases, the inflection point moves towards the duct inlet. This is to be expected, as it reduces the adverse pressure gradient for the larger second area.

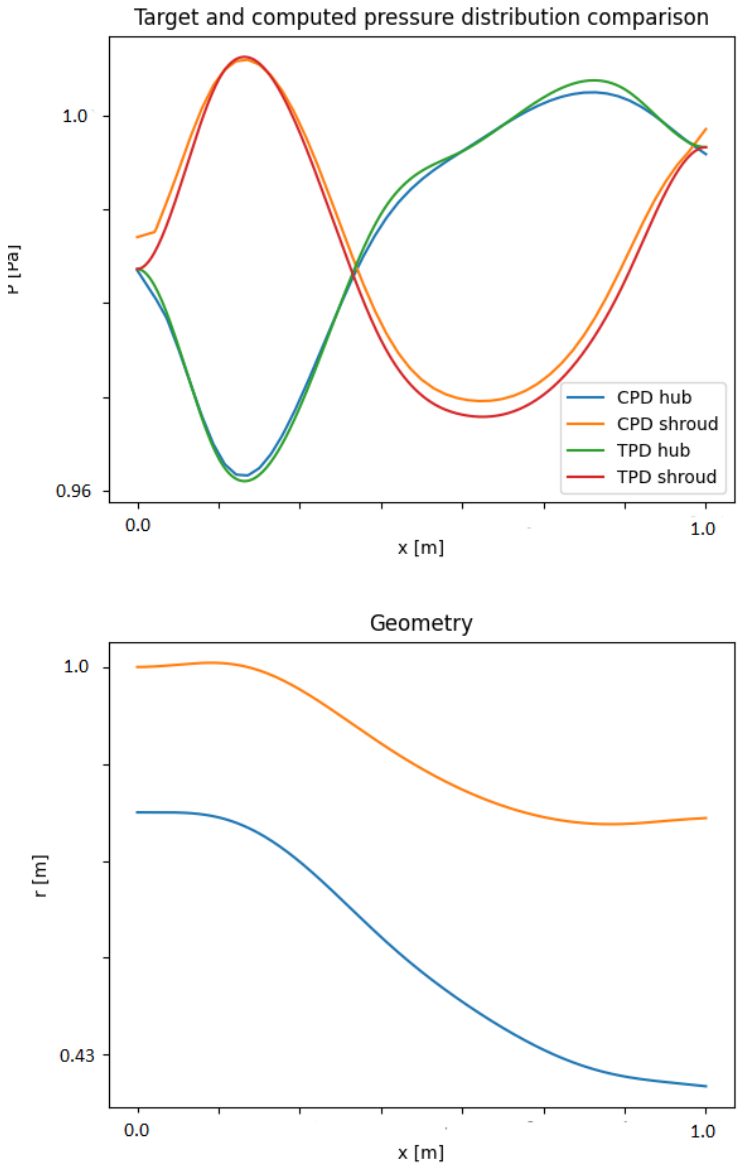
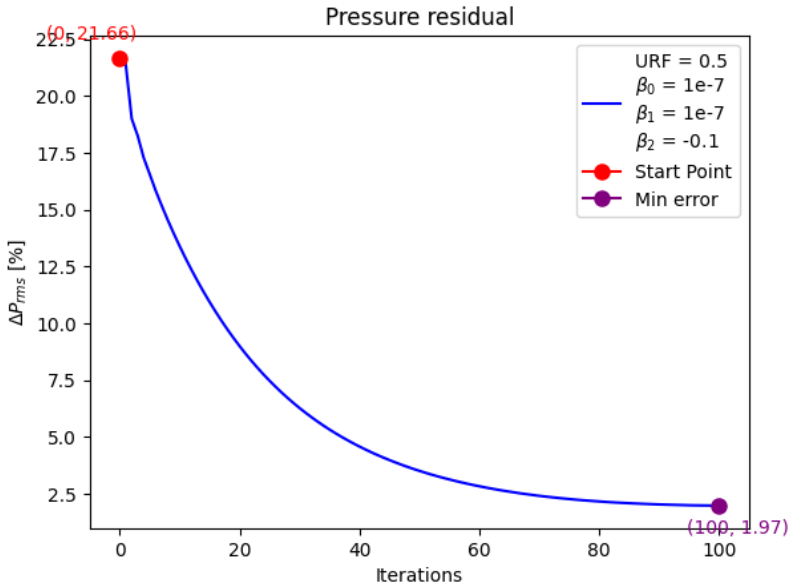


Figure 34 (a) TPD vs CPD and (b) Final geometry achieved with a TPD using a constrained trapped area of 340 and 460 N/m



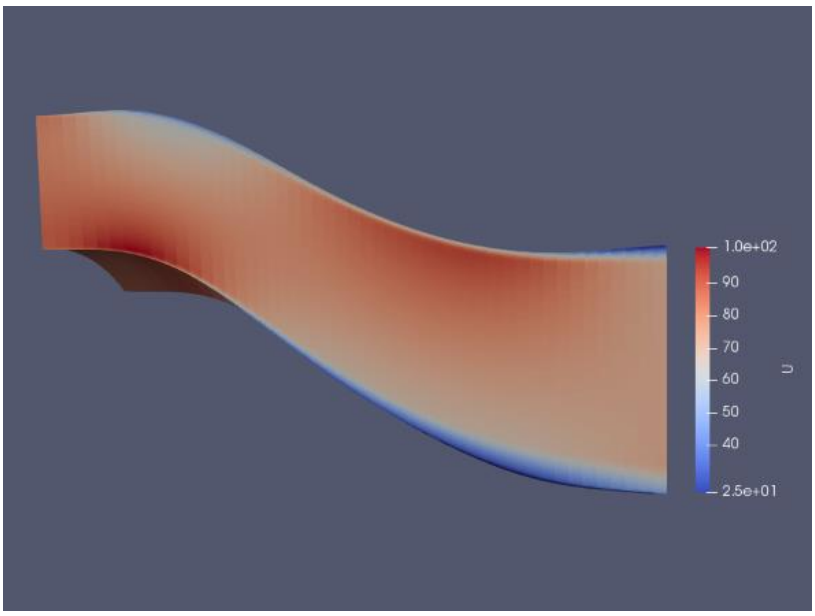


Figure 35 (a) Residual plot and (b) Axial-velocity contour plot achieved with a TPD using a constrained trapped area of 340 and 460 N/m

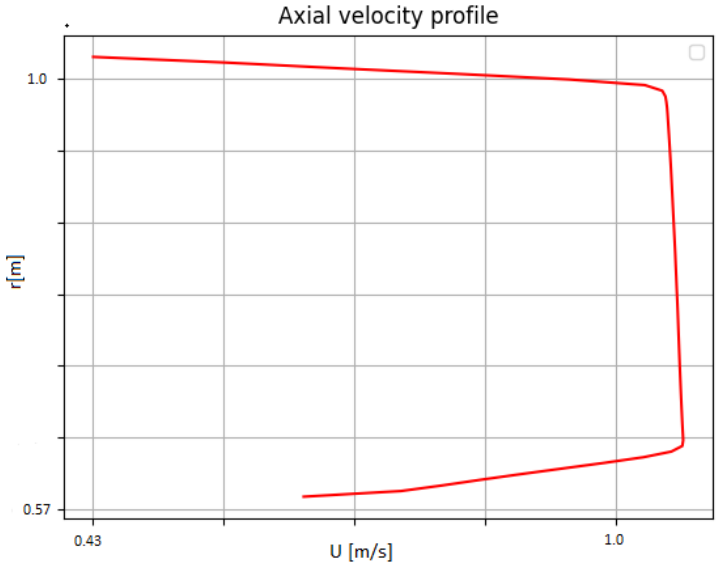


Figure 36: Axial velocity profile at the outlet of the generated duct in Figure 34

The axial and the radial momentum fluxes at the outlet are 86.35 N and 6.75 N, which corresponds to a flux averaged outlet flow angle of 4.47 degrees.

4.3.5 Results discussion

In the above sections one can observe the different generated TPD's varying in two main factors: Turbulence Models and different levels of aggressiveness.

As it has been discussed earlier (Comparison / Choice of turbulence models) the Standard k-ϵ model delays separation in the boundary layer. This trend can also be observed in the generated TPD's as the Standard k-ϵ model allows for a significantly higher pressure recovery. For the same inlet conditions the recovered pressure is around 1000Pa higher for the Standard k-ϵ model than for the Standard k-ω one. The high pressure recovery achieved in the Standard k-ϵ model, lead to convergence issues in the geometry creation (Figure 28, Figure 30, Figure 33). The trend of the Standard k-ϵ model is consistent with expectations: As the aggressiveness increases the pressure recovery decreases. However, the high adverse pressure gradient in the TPD's, which does not yield in separation when considering a flow over a flat plate, indicates separation when generating the geometry based of that. This suggests the further use of the Standard k-ω model instead of the Standard k-ϵ one, because it is more accurate when facing highly adverse pressure gradients (section 3.2.3) – especially when considering that the flat plate assumption underestimates flow separation.

Furthermore, there is also a trend to observe when investigating each turbulence model for different levels of aggressiveness. In both cases, the pressure recovery decreases as the aggressiveness increases. This

is to be expected, since the resulting higher trapped area requires a steeper curvature of both pressure curves. This stronger adverse pressure gradient increases the risk of separation resulting in a lower pressure recovery. Furthermore, the inflection point is increasingly forced to move towards the center of the duct as the aggressiveness increases. When facing a lower aggressiveness, the optimizer still has more room to fulfill all necessary requirements. Hence, it can be observed that the inflection point is not positioned at the center, as it is not constrained.

The last case represents a duct with an axial outflow. For the axisymmetric duct this has been achieved by fixing the areas in between the wall pressure distributions. For a spanwise geometry the standard area equality constraint will already yield in a duct with axial outflow. However, for the axisymmetric case, this constraint needs to be modified and include the centerline radius as an additional variable.

4.4 Nature and scope of the model governing equation

The model governing equation used to govern the change in the cross-stream coordinate has a close resemblance to cross-stream momentum conservation as shown in Equation 84

$$-\frac{\partial p}{\partial r} = \rho\left(\frac{U_x^2 + U_r^2}{R}\cos\left(\tan^{-1}\left(\frac{dx}{dr}\right)\right) + \frac{U_\theta^2}{0.5\,(r_{shroud} + r_{hub})}\right)$$

Equation 83: Inviscid cross-stream momentum conservation equation relating the pressure gradient to the curvature

where *R* is the radius of curvature of the flow in the r-x plane

Assuming no swirl, i.e., U_θ = 0, and the flow at an angle α to the x-axis, the momentum equation simplifies to

$$-\frac{\partial p}{\partial r} = \rho\left(\frac{U_x^2 + U_r^2}{R}\right)\cos(\alpha)$$

$$and$$

$$\frac{1}{R} \cong \frac{d^2r}{dx^2}, for\ R \gg 1$$

Therefore,

$$\frac{d^2r}{dx^2} = -\frac{\partial p}{\partial r}\frac{1}{\rho(U_x^2 + U_r^2)\cos(\alpha)} = -\frac{\partial p}{\partial r}\frac{1}{\rho\, U^2\, \cos(\alpha)}$$

=>

$$\frac{d^2r}{dx^2} \propto -\frac{\partial p}{\partial r}$$

Equation 84: Inviscid cross-stream momentum conservation proportionality equation relating the pressure gradient to the curvature

4.4.1 Effect of including viscous effects

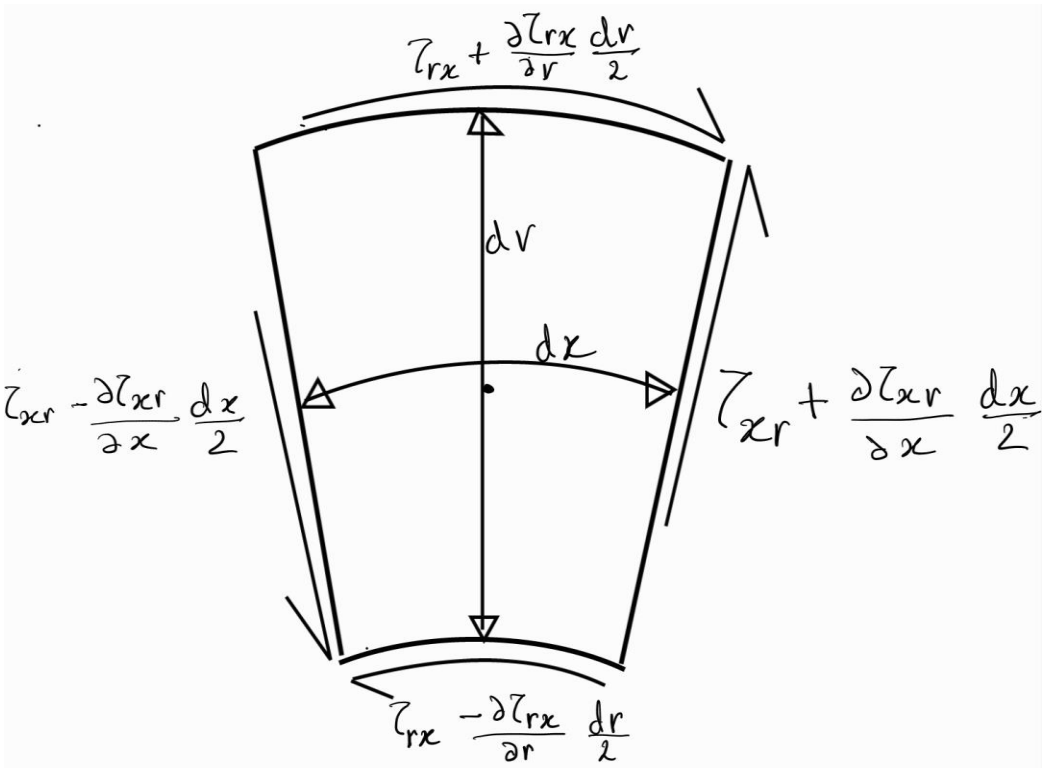


Figure 37: Free body diagram of a differential fluid element, curving under the effect of shear stress

Therefore, only considering the effect of the shear stress,

$$\left(\tau_{xr} + \frac{\partial \tau_{xr}}{\partial x}\frac{dx}{2}\right) dr \; - \left(\tau_{xr} - \frac{\partial \tau_{xr}}{\partial x}\frac{dx}{2}\right) dr$$
$$= \; - \rho \left(\frac{U_x^2 + U_r^2}{R} \cos\left(\tan^{-1}\left(\frac{dx}{dr}\right)\right) + \frac{U_\theta^2}{0.5\,(r_{shroud} + r_{hub})}\right)$$

Equation 85: General cross-stream momentum conservation equation relating the pressure gradient to the curvature derivation

Superimposing the effects of pressure and shear stress gradients,

$$-\left(\frac{\partial p}{\partial r} + \frac{\partial \tau}{\partial x}\right) = \rho \left(\frac{U_x^2 + U_r^2}{R} \cos\left(\tan^{-1}\left(\frac{dx}{dr}\right)\right) + \frac{U_\theta^2}{0.5\,(r_{shroud} + r_{hub})}\right)$$

Equation 86: General cross-stream momentum conservation equation relating the pressure gradient to the curvature

Therefore, on considering the same assumptions, one can get the following

$$\frac{d^2 r}{dx^2} \propto -\left(\frac{\partial p}{\partial r} + \frac{\partial \tau}{\partial x}\right)$$

Equation 87: General cross-stream momentum conservation proportionality equation relating the pressure gradient to the curvature

While the computed geometry is still outside the boundary layer of the target geometry, the inviscid governing equation is applicable. As it enters the boundary layer, the cross-stream pressure gradient starts to reduce and instead, it is the streamwise shear stress gradient that induces curvature in the flow. Therefore, on reaching a very close pressure residual value, the governing equation starts to keep on pushing the computed geometry lines in the same direction to reduce the residual. Unfortunately, the geometry already being in the viscosity dominant region, the physics does not comply with the inviscid model governing equation. Therefore, the wall pressure does not change in the intended direction anymore. This change in physics can be associated to the pressure residuals reaching a minimum value and then eventually increasing gradually, as shown in Figure 38.

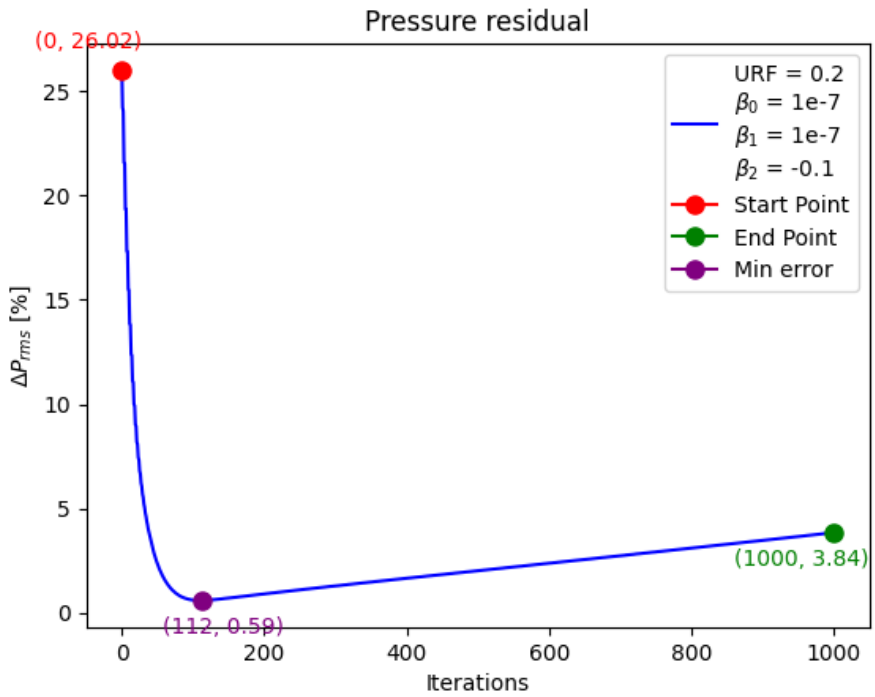
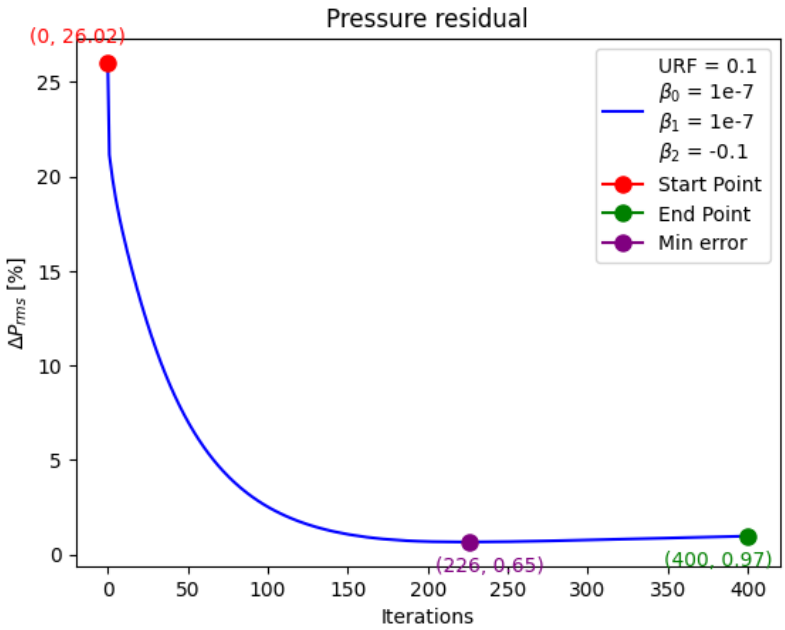


Figure 38: Pressure residual, depicting the inviscid nature of the IDA governing equation

Figure 38 uses the TPD shown in Figure 34 but with a much higher mass flow rate of approximately 25 kg/s. The mass flow rate was gradually reduced till the standard value used in this whole study, to 15 kg/s. With a lower mass flow rate under the same inlet conditions, i.e., with a lower inlet momentum, and the same TPD, the boundary layer at the critical (prone to separation) region started to thicken. This led to an expansion of the viscous dominant region, i.e., where the model governing equation does not comply accurately with physics. Therefore, it was hypothesized that this led to an increase in the minimum residual achieved as shown in the following figures.



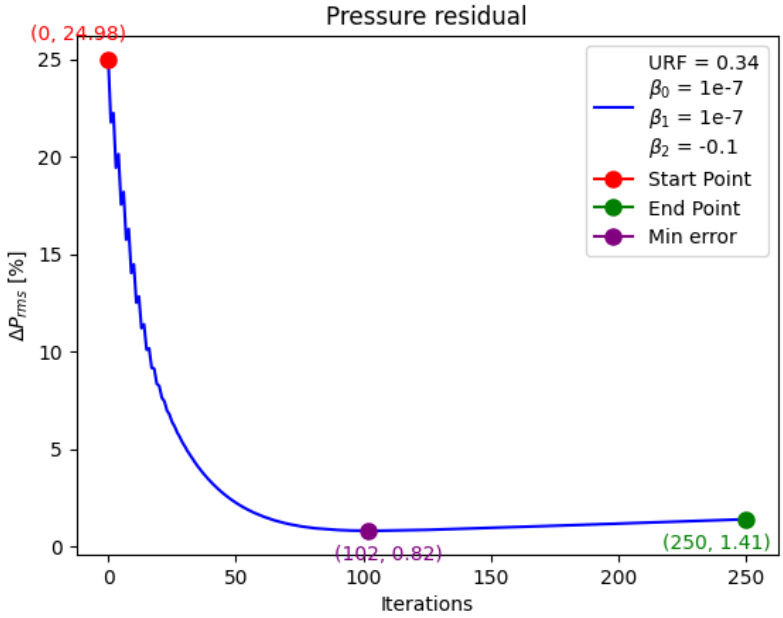


Figure 39: : Pressure residual with a mass flow rate of (a) 24.22 kg/s (b) 19.38 kg/s

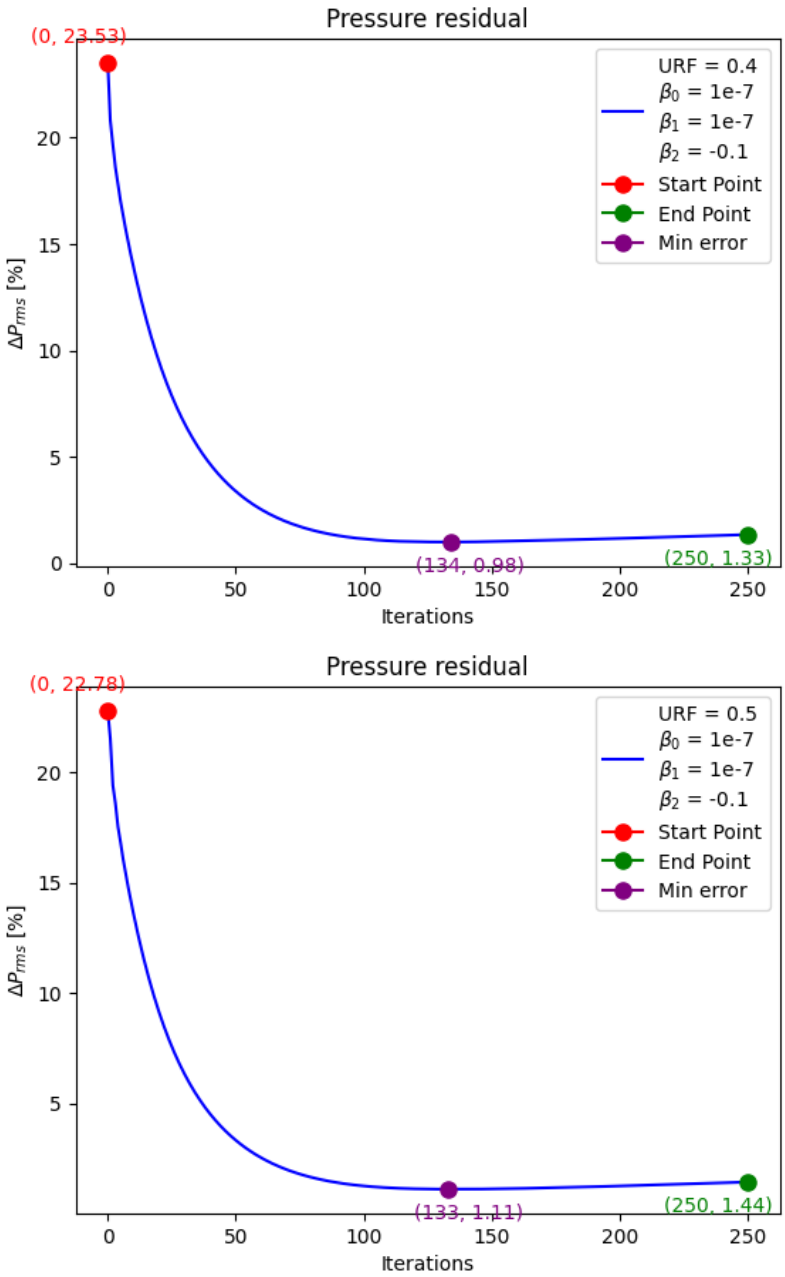


Figure 40: Pressure residual with a mass flow rate (a) 16.35 kg/s and (b) 15 kg/s

5 Conclusions and future work

5.1 Turbulence model investigation

Concluding, a loop to design a geometry based of a generated TPD has been successfully implemented and tested for the case of an ICD. In the first basic scenario, a duct has been generated using three levels of aggressiveness as well as two different turbulence models – namely the Standard k- ϵ and the Standard k-ω one. At a lower aggressiveness, one can observe that the TPD's are more complex with some "wiggles". This can be traced to the fact that at those low aggressiveness scenarios the optimizer has more options to fulfill all constraints while still achieving the highest possible pressure recovery. This results in more unexpected pressure curves for the traditional s-shaped duct. As the aggressiveness increases, the optimizer is forced to move the point of inflection towards the center of the duct and have a smoother overall shape. Otherwise, it would not be possible to implement a higher trapped area, as it yields in a stronger adverse pressure gradient more prone to flow separation.

Investigating the two turbulence models, the TPD reflects the tendency of the Standard k-ϵ model to delay separation in comparison to the Standard k-ω one. Hence, the pressure recovery for all Standard k-ϵ models is significantly higher. Nevertheless, in case of the TPD generation the Standard k-ω model seems to be more suitable. Firstly, literature suggests that the Standard k-ω model is more suitable for highly adverse pressure gradient, which is present in this project, as it captures the near wall region more accurately. Secondly, as the TPD investigates, whether flow over a flat plate separate, it does not account for any curvature effects. Generally, these make the flow more unstable and thus more prone to separation. As a result, it is advised to use the more conservative approach of the Standard k-ω model to check for separation.

5.2 Potential further improvements to TPD generation

Having developed and established a single objective optimizer, the next step in the process could be the development of a multi objective optimizer. This opens a new dimension in the development process, as it not only tries to maximize the pressure recovery but also minimizing the length of the duct – the two main considerations or trade-offs in the design of ICDs. Furthermore, the TPD generation is computationally expensive. For common aero components such as ICD, several TPDs are already available. Therefore, adding a machine learning (PINNs) component to the meta-heuristic optimizer, or using a purely machine learning based approach could significantly reduce computational times. Observing the results for different levels of aggressiveness, one can see patterns in the shapes of the TPD's based off the flow physics of an ICD. These are currently not considered by the metaheuristic Differential Evolution algorithm, as it is a purely stochastic one. However, the

patterns can be exploited by a well-trained neural network to significantly reduce the number of needed iterations.

5.3 Expanding the scope of the inverse design solver

The Inverse design solver constructed and employed for an axisymmetric geometry showed good compatibility with most pressure profiles. For certain TPD cases, the algorithm did not work due to the in-house solver not being able to converge. Changing the CFD software coupled with the IDA from the in-house solver to a more reliable solver would potentially solve this issue.

In our setup, with a stable set of coefficients, it took the inverse solver approximately 100 iterations to go close to a 1% residual region.

The governing equation, if modified into the following, might make it applicable for internal flow in a 3D geometry, *z = f(x, y)*,

$$\beta_2 \times \frac{\partial}{\partial t}\left(\frac{\partial^2 z}{\partial x^2} + \frac{\partial^2 z}{\partial y^2}\right) = \Delta P$$

Equation 88: Potential model governing equation for inverse design solver in 3D

External flows, more prone to instances of separation, would require a change in the governing model equation. Without any change in the IDA governing equation, the algorithm should not work for both 2D and 3D flows.

It was understood that pressure distribution of a separated case, being a less aggressive pressure distribution, could be used as a TPD to generate a geometry without any flow separation but with the same pressure distribution. This suggests that there are different possible solutions for geometry with the same pressure distribution. Without separation and at a low Mach number, the flow approximately follows the isentropic relation, which closes the set of equations, leading to a unique solution. In the instance of separation, this not being valid anymore would require an extra flow characteristic distribution, such as static temperature or shear stress distribution. Currently, the model governing equation only resembles the curvature formulae closely, which was entirely derived from the inviscid radial momentum conservation (Equation 84). A similar governing equation, if derived from the general radial momentum conservation equation (Equation 87), would imply a possible successful routine even with a separating case. This would generalize the solver for all kinds of internal flows, with possible expansion to external flows, with additional or other constraints.

6 Bibliography

Abhishek, K., Guohua, W., Mostafa, Z., Rammohan, M., Pannuthurai, N. S., & Swagatam, D. (04 2020). A test-suite of non-convex constrained optimization problems from the real-world and some baseline results. *Swarm and Evolutionary Computation, 56*, 100693.

Anderson, J. D., Degroote, J., Degrez, G., Dick, E., Grundmann, R., & Vierendeels, J. (2009). *Computational Fluid Dynamics.* Springer.

Baldwin, B., & Lomax, H. (02 1978). Thin Layer approximation and algebraic model for separated turbulent flows. *AIAA Paper*, 78.

Bergh, J. (2018). *On the Evaluation of Common Design Metrics for the Optimization of Non-Axisymmetric Endwall Contours for a 1-Stage Turbine Rotor.*

Bertin, J. J., & Cummings, R. M. (2014). *Aerodynamics for Engineers.* Pearson.

Bertin, J. J., Cummings, R. M., & Reddy, P. V. (2014). *Aerodynamics for Engineers* (6th uppl.).

Blasius, H. (1908). Grenzschichten in Flüssigkeiten mit kleiner Reibung. *Zeitschrift für Mathematik und Physik*, 1-37.

Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization.* Cambridge University Press.

Cebeci, T., & Bradshaw, P. (1984). *Physical and Computational Aspects of Convective Heat Transfer.* Springer.

Dan, Z., Dixon, S. L., & Hall, C. (2025). *Fluid mechanics and thermodynaimcs of turbomachinery* (8th uppl.).

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algroithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*.

Falkner, V., & Skan, S. (1931). Some approximate solutions of theboundary layer equations. *Philosophical Magazine*.

Fletcher, R. (1981). *Practical Methods of Optimization.* WIley.

Gubanova, I., & Gubanova, M. (2014). Inverse methods for aerodynamic design using the navier-stokes equations. St. Petersburg: International Council of Aeronautical Sciences.

Hall, Z., Ahuja, V., Hartfield, R., Shelton, A., & Ahmed, A. (2012). Optimization of a Turbofan Inlet Duct Using Genetic Algorithms and CFD. Applied Aerodynamics Conference.

Jameson, A. (1988). Aerodynamic Design via Control Theory. *Journal of Sceintific Computing*.

Jameson, A. (2004). On improving the Efficiency of Differential Evolution for Aerodynamic Shape Optimization. AIAA/ISSMO Multidisciplinary Analysis and Optimization Conference.

Kalyanmoy, D. (2001). *Multi-Objective Optimization using Evolutionary Algorithms.* Wiley.

Kennedy, J., & Eberhart, R. (1995). Particle Swarm Optimization. International Conference on Neural Networks.

Kundu, P., Cohen, I., & Dowling, D. (2016). *Fluid Mechanics.* Elsevier.

Langtry, R., & Menter, F. (2009). Correlation-Based Transition Modeling for Unstructured Parallelized Computational Fluid Dynamics Codes. *AIAA Journal*.

Langtry, R., Menter, F., Likki, S., Suzen, Y., Huang, P., & Völker, S. (2006). A correlation based transition model using local variables for unstructured parallelized CFD code. *Journal of Turbomachinery*.

Launder, B., & Spalding, D. (1974). A Model of Turbulence: Dissipation Rate in the Turbulent Kinetic Energy Equation. *Computer Methods in Applied Mechanics and Engineering*, 269-289.

Launder, B., & Spalding, D. (1974). The numerical computation of turbulent flows. *Computer Methods in Applied Mechanics and Engineering*.

Madavan, N. K. (2004). On Imporving Efficiency of Differential Evolution for Aerodynamic Shape Optimization Applications.

Malone, J., Vadyak, J., & Sarkar, L. (1985). A technique for the inverse aerodynamic design of nacelles and wing configuration. Colorado Springs: AIAA 3rd Applied Aerodynamics Conference.

Mandal, B., & Mazumdar, H. (2015). The importance of the law of the wall. *International Journal of Applied Mechanics and Engineering*.

Menter, F. (09 1994). Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal, 32*, 1598-1605.

Nili-Ahmadabadi, M., Aghabozorgi, F., Cho, D.-S., & Kim, C. K. (2021). Development and validation of a hybrid hybrid aerodynamic design method for curved diffusers using genetic algorithm and ball-spine inverse design method. *Alexandria Engineering Journal*.

Nocedal, J., & Wright, S. (2006). *Numerical Optimization* (2nd uppl.). Springer.

Pohlhausen, K. (1921). The approximate integration of the differential equation for the lainar boundary layer. *ZAMM - Zeitschrift fuer angewandte Mathematik*, 252-268.

Pope, S. (2000). *Turbulent Flows.* Cambridge University Press.

Price, K., Storn, R., & Lampinen, J. (2005). *Differential Evolution: A prctical Approach to Global Optimization.* Springer.

Rao, S. (2009). *Engineering Optimization Teory and Practice* (4th uppl.). Wiley.

Rashed, N., Ali, Y., & Rashid, T. A. (2024). Advancements in Optimization: Critical Analysis of Evolutionary, Swarm and Behvaior-Based Algorithms. *Algortihms*.

Sagonda, A., & Folly, K. (2022). A comparative study between deterministic and two meta-heuristic algorithms for solar PV MPPT control under partial shading conditions. *Systems and Soft Computing*.

Schlichting, H., & Gersten, K. (2016). *Boundary Layer Theory* (9th uppl.). Springer.

Shih, T.-H., Liou, W. W., Shabbir, A., Yang, Z., & Zhu, J. (1995). A new k-epsilon eddy viscosity model for high reynolds number turbulent flows. *Computers and Fluids*, 227-238.

Spalart, P., & Allmaras, S. (1994). A one-equation Turbulence Model for Aerodynamic Flows . *La Rechere Aerospatiale*, 5-21.

Storn, R., & Price, K. (1997). Differential Evolution - A Simple and Efficient Heuristic for Global Optimization over Continous Spaces. *Journal of Global Optimization*.

Tonsic, B., Bala, G., Lejon, M., Deshpande, S., & Bergh, J. (2024). Investigation of the flow in an aggressive intermediate compressor duct. ISABE.

Wilcox, D. (1998). Turbulence Modeling for CFD. *2nd Edition DCW Industries*.